

基于图论的复杂系统架构模型转换方法研究

郭江敏

2021 年 6 月

中图分类号：TP391

UDC 分类号：621

基于图论的复杂系统架构模型转换方法研究

作者姓名	<u>郭江敏</u>
学院名称	<u>机械与车辆学院</u>
指导教师	<u>王国新 副教授</u>
答辩委员会主席	<u>张之敬 教授</u>
申请学位	<u>工学硕士</u>
学科专业	<u>机械工程</u>
学位授予单位	<u>北京理工大学</u>
论文答辩日期	<u>2021 年 6 月</u>

Research on Model Transformation of Complex System Architecture Based on Graph Theory

Candidate Name:	<u>GUO Jiangmin</u>
School Or Department:	<u>School of Mechanical Engineering</u>
Faculty Mentor:	<u>Associate Prof. WANG Guoxin</u>
Chair, Thesis Committee:	<u>Prof. ZHANG Zhijing</u>
Degree Applied:	<u>Master of Science in Engineering</u>
Major:	<u>Mechanical Engineering</u>
Degree By:	<u>Beijing Institute of Technology</u>
The Date Of Defence:	<u>June, 2021</u>

基于图论的复杂系统架构模型转换方法研究

郭江敏

北京理工大学

研究成果声明

本人郑重声明：所提交的学位论文是我本人在指导教师的指导下进行的研究工作获得的研究成果。尽我所知，文中除特别标注和致谢的地方外，学位论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京理工大学或其它教育机构的学位或证书所使用过的材料。与我一同工作的合作者对此研究工作所做的任何贡献均已在学位论文中作了明确的说明并表示了谢意。

特此申明。

签 名： 日期：

关于学位论文使用权的说明

本人完全了解北京理工大学有关保管、使用学位论文的规定，其中包括：①学校有权保管、并向有关部门送交学位论文的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存学位论文；③学校可允许学位论文被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换学位论文；⑤学校可以公布学位论文的全部或部分内容（保密学位论文在解密后遵守此规定）。

签 名： 日期：

导师签名： 日期：

摘要

复杂系统的开发离不开架构设计，近年来系统规模及系统复杂程度急剧增长，基于模型系统工程（Model-based Systems Engineering, MBSE）被越来越多地应用于复杂系统的架构设计过程。MBSE 方法以形式化的系统架构模型为核心，通过模型驱动开发方法实现模型的（半）自动生成，保证模型信息可追溯性，提高开发效率。模型转换是模型驱动开发的关键方法，然而，在多学科交叉的复杂系统架构设计场景下，目前存在的模型转换方法存在许多不足之处，其具体表现为：1) 缺乏支持多领域异构模型的通用模型转换方法，导致多领域架构模型之间的转换需要应用多种转换方法，开发难度增加 2) 缺乏一种模型转换统一描述语言，在架构模型转换过程中需要用到多种模型转换语言，导致架构师的学习成本增加。针对上述问题，本文提出了面向复杂系统架构设计的模型转换方法和模型转换语言，对以下内容进行研究：

（1）复杂系统架构模型转换理论与框架研究。在架构模型构建方法中主要介绍了基于 GOPRR-E 元模型的元建模框架，实现多领域异构模型的统一表达，为跨域模型转换奠定基础；在模型转换方面总结现有方法并给出本文的模型转换定义；最后提出本文的模型转换框架。

（2）基于图论的复杂系统架构模型转换方法研究。针对复杂系统架构设计场景下多领域异构模型转换方法不统一的问题，基于图论将 GOPRR-E 元建模框架中模型、元模型映射为等价的图、类型图，模型转换规则映射为图转换规则，将异构领域的架构模型转换问题转变为同构的图转换问题，实现系统架构模型的跨域模型转换。

（3）架构模型转换正确性的形式化检验方法研究。针对架构模型转换可能出现缺陷模型的问题，提出模型转换检验方法。首先给出模型转换正确性的形式化定义；接着，基于类型图一致性验证理论，将模型转换正确性问题形式化描述为类型图一致性验证问题。最后，提出模型转换自动验证流程，避免生成缺陷模型，确保模型转换正确性。

（4）架构模型转换统一描述语言研究。在上述研究基础上，提出模型转换统一描述语言，以模型转换规则的形式实现架构模型转换的统一描述。重点引入模型模式的概念并根据架构模型的结构特点，总结出四种典型的模型模式，并设计模型转换语言的抽象语法和具体语法。最后，开发模型转换语言解析器和执行引擎，以实现模型转

换语言的描述能力。

(5) 在上述理论研究的基础上,以无人平台系统架构设计为案例,对本文提出的模型转换方法和模型转换语言进行实例应用和验证。

关键词: 模型转换, 模型转换语言, 模型转换验证, 基于模型系统工程

Abstract

The design of the systems architecture is important for developing complex systems. In recent years, Model-based Systems Engineering (MBSE) has been increasingly used in the architecture design process, as the system scale and system complexity have increased rapidly. The system architecture models are recognized as the core-stone of MBSE, which realizes the (semi-) automatic generation of models through the model-driven development approach. Moreover, the model-driven approach ensures the traceability of the model information and improves development efficiency. Model transformation is the key technology of model-driven development. However, in the multi-disciplinary complex system architecture design scenario, there are still many deficiencies for the existing model transformation approaches. And the specific performance as follows: 1) Lack of general model transformation approach which supports the transformation for multi-domain heterogeneous models. The current model transformation is usually aimed at a specific domain model, so multiple approaches will be required in the model transformation scenarios of complex systems, where multiple domains are referred. 2) Lack of one unified description language for model transformation, which increases the learning cost for architecture engineers. In view of the above problems, one model transformation and a model transformation language are proposed in the paper, the approach and the language mainly for complex system architecture design. The main contents of the paper are as follows.

(1) Research on the theory and the framework of complex systems architecture model transformation. Firstly, the meta-modeling framework based on the GOPPRR-E meta-meta model is introduced to realize the unified expression of multi-domain heterogeneous models. Moreover, the model transformation define are proposed by summarizing the existing methods. Finally, the model transformation framework is proposed.

(2) Research on the model transformation approach of complex system architecture model based on graph theory. Aiming at the inconsistent transformation approaches for multi-domain heterogeneous models in complex system architecture design scenarios, a transformation approach based on the GOPPRR-E meta-modeling framework is proposed.

Firstly, the concept of heterogeneous transformation is introduced. Moreover, a series of mapping methods from architectural model concepts to graph theory concepts are proposed. Thus, the model transformation is formalized as graph transformation of graph theory. Finally, the model transformation process is proposed based on the mapping approach, which realizes the cross-domain transformation of the system architecture model.

(3) Research on the formal verification method for the correctness of the architecture model transformation. In order to ensure the correctness of the model generated by the model transformation, a formal definition of the model transformation correctness was proposed. Secondly, the correctness of the model transformation is verified based on the consistency condition of the type graph, and four consistency conditions are summarized. Finally, an automatic verification process for model transformation is proposed to verify the error model which may be generated during the transformation process.

(4) Research on the unified model transformation language for architecture model transformation. Based on the above research, a rule-based model transformation language is proposed to achieve a unified description of the architecture model transformation. The concept of model pattern is introduced according to the structural characteristics of the architecture model, and four typical model patterns are summarized. Then the abstract syntax and concrete syntax of the model transformation language are designed. Finally, the parser and execution engine is proposed for the model transformation language to enable the description for model transformation.

(5) Based on the above theoretical research, the architecture models of the unmanned platform are developed, in which both the model transformation approach and the model transformation language proposed are verified.

Key Words: Model transformation, Model transformation language, Model transformation verification, MBSE

目录

第 1 章 绪论	1
1.1 研究背景与意义	1
1.1.1 研究背景	1
1.1.2 研究意义	2
1.2 课题来源	2
1.3 国内外研究现状	3
1.3.1 架构模型构建方法	3
1.3.2 架构模型转换方法	4
1.3.3 模型转换语言	5
1.3.4 研究现状分析总结	6
1.4 研究内容及组织结构	7
1.5 本章小结	9
第 2 章 复杂系统架构模型转换理论与框架研究	11
2.1 引言	11
2.2 基于 GOPRR-E 元建模框架的系统架构模型构建方法	11
2.2.1 MOF 元建模框架	12
2.2.2 GOPRR-E 元元模型和元建模	13
2.2.3 架构模型构建流程	15
2.3 架构模型转换方法	17
2.3.1 模型转换的定义	17
2.3.2 模型转换分类	18
2.3.3 模型转换方式	19
2.3.4 模型转换语言	20
2.4 架构模型转换框架	20
2.5 本章小结	21
第 3 章 面向跨域系统架构模型的模型转换方法	23
3.1 引言	23

3.2 跨域模型转换的概念	23
3.3 GOPPRR-E 模型概念到图论概念的映射	24
3.3.1 图转换的基本概念	24
3.3.2 GOPPRR-E 模型的形式化描述	26
3.3.3 GOPPRR-E 元模型到类型图的映射	29
3.3.4 GOPPRR-E 模型到图的映射	32
3.4 基于图转换的 GOPPRR-E 模型转换	34
3.4.1 图转换	34
3.4.2 GOPPRR-E 模型转换到图转换的映射	36
3.5 架构模型转换流程	38
3.6 本章小结	42
第 4 章 架构模型转换正确性的形式化检验方法	43
4.1 引言	43
4.2 模型转换正确性的定义	43
4.3 基于类型图一致性验证的模型转换正确性验证	44
4.3.1 类型图一致性条件	45
4.4 模型转换正确性验证流程	48
4.5 本章小结	50
第 5 章 模型转换统一描述语言设计	51
5.1 引言	51
5.2 模型转换语言抽象语法	51
5.2.1 模式描述语言	52
5.2.2 规则描述语言	57
5.2.3 转换辅助语言	58
5.3 模型转换语言具体语法	59
5.3.1 模型转换语言关键字	59
5.3.2 语言具体语法设计	59
5.4 模型转换引擎开发	66
5.4.1 基于 ANTLR 的语言解析器	66

5.4.2 基于 AGG 图转换引擎的模型转换引擎	66
5.5 本章小结	67
第 6 章 面向无人平台系统架构设计的模型转换验证	69
6.1 引言	69
6.2 无人平台架构设计背景	69
6.3 无人平台架构模型转换流程	70
6.4 架构模型转换分析	74
6.4.1 需求到功能架构的转换	74
6.4.2 功能架构到逻辑架构的转换	78
6.4.3 功能架构到物理架构的转换	81
6.5 架构模型转换结果分析	85
6.6 本章小结	87
总结与展望	89
工作总结	89
展望	90
参考文献	93
攻读学位期间发表论文与研究成果清单	99
参与科研项目	99
发表论文	99
学科竞赛	99
附录 A. 模型转换术语	101
附录 B. 模型转换语言具体语法	103
致谢	109

本文图目录

图 1.1 论文组织结构图	9
图 2.1 OMG 的元建模框架	12
图 2.2 GOPPRR-E 元元模型	13
图 2.3 基于 GOPPRR-E 元建模框架的架构模型构建过程	16
图 2.4 模型转换框架	21
图 3.1 跨域模型转换的模型层级示意图	24
图 3.2 GOPPRR-E 元模型到类型图的映射规范示意图	29
图 3.3 领域元模型到类型图映射实例	31
图 3.4 元模型映射和模型映射的关系示意图	32
图 3.5 GOPPRR-E 模型到图的映射实例	33
图 3.6 基于 SPO 的图规则应用示意图	35
图 3.7 模型转换规则的构成示意图	36
图 3.8 基于图转换规则的模型转换原理图	37
图 3.9 转换元模型 MM_t 构建示意图	38
图 3.10 模型转换流程图	39
图 3.11 转换源元模型和目标元模型的类型图表示	40
图 3.12 转换源模型和生成的目标模型	40
图 3.13 模型转换规则	40
图 3.14 源元模型、目标元模型和转换元模型映射的类型图	41
图 3.15 模型转换规则映射的图规则	41
图 3.16 起始图和生成图	42
图 4.1 模型转换验证示意图	44
图 4.2 模型转换语法正确性验证原理示意图	45
图 4.3 图顶点类型一致性条件示例	46
图 4.4 边一致性条件示例	46
图 4.5 边的目标多重性一致性条件示例	47
图 4.6 边的起始多重性一致性条件示例	48

图 4.7 模型转换正确性验证流程	48
图 4.8 模型转换正确性验证实例	49
图 5.1 模型转换语言抽象语法类图描述	52
图 5.2 模型模式示意图	53
图 5.3 单元素模式示例	53
图 5.4 简单模式示例	54
图 5.5 基数模式示例	55
图 5.6 分组模式和点基数模式示例	57
图 5.7 模型转换规则抽象语法	58
图 5.8 模型转换数据类型	59
图 5.9 模型转换引擎开发架构	66
图 6.1 基于 RFLP 设计流程的无人平台架构设计	70
图 6.2 基于 RFLP 流程的多系统层级架构设计	71
图 6.3 SysML 系统建模语言的九种图	71
图 6.4 无人平台系统级架构模型转换总体流程	73
图 6.5 无人平台系统总体需求模型	74
图 6.6 需求图到用例图转换规则示意图	75
图 6.7 模型转换生成的用例图	77
图 6.8 无人平台系统级用例图	77
图 6.9 用例图到状态机图转换规则示意图	78
图 6.10 模型转换生成的状态机图	80
图 6.11 无人平台系统级状态机图	81
图 6.12 用例图到模块定义图转换规则示意图	82
图 6.13 模型转换生成的模块定义图	84
图 6.14 无人平台系统级模块定义图	85

本文表目录

表 2.1 GOPPRR-E 元元模型含义.....	14
表 2.2 GOPPRR-E 元元模型的衍生概念.....	15
表 3.1 GOPPRR-E 元模型映射规范.....	30
表 3.2 GOPPRR-E 模型映射规范.....	33
表 5.1 模型元素之间存在的二元关系	54
表 5.2 模型转换语言关键字	59
表 6.1 RFLP 设计流程与 SysML 视图对应关系.....	72
表 6.2 需求图到用例图转换的元素对应关系	75
表 6.3 用例图到状态机图转换的元素对应关系	78
表 6.4 用例图到模块定义图转换的元素对应关系	81
表 6.5 无人平台架构模型转换过程中应用的模型转换语言概念	86
表 6.6 目前模型转换引擎对模型转换语言概念支持情况	86

第1章 绪论

1.1 研究背景与意义

1.1.1 研究背景

近年来系统规模及系统复杂程度急剧增长，基于模型系统工程（Model-based Systems Engineering, MBSE）被越来越多地应用于复杂系统的开发过程。复杂系统的开发离不开系统架构设计，架构设计即对系统的实体以及实体间的关系进行描述，在 MBSE 方法中，这一步骤往往通过形式化的系统架构模型描述。传统基于文档的系统工程以文档为中心进行架构设计，开发过程中大量系统信息碎片散布在文档中，难以实现信息追溯，越来越不能满足现代复杂系统开发的需求。为了应对这样的挑战，国际系统工程组织（International Council on Systems Engineering, INCOSE）提出了基于模型的系统工程管理新模式和实现^[1]。MBSE 通过形式化建模方法，将系统全生命周期中所涉及到的需求、设计、分析、验证和确认等活动都以模型的形式表达^[2]，提高了复杂系统信息追溯能力和研发效率，目前已经成为学术界与工业界关注的热点技术^[3]。

复杂系统架构设计是一个多学科交叉过程。在采用 MBSE 方法开发复杂系统架构过程中，想要尽可能详细准确地描述系统，需要从多个角度（系统视图）建立精确细致的模型以刻画系统，视图分别对应不同的专业学科、不同的工作角色及不同的利益相关者^[4]，从不同方面描述系统的特征。系统视图由特定域模型描述，领域工程师利用 SysML（Systems Modeling Language）、UML（Unified Modeling Language）、BPMN（Business Process Modeling Notation）、Modelica、AADL（Architecture Analysis & Design Language）等领域建模语言建立各自领域模型，如需求、功能、逻辑、物理等架构模型、仿真验证模型。系统的各个视图是系统整体视图的一部分，不同视图存在关联关系，对应视图的特定域模型也存在追溯和关联关系，在架构开发过程中需要保证这些信息的一致性。

基于 MBSE 开发系统架构模型是一种模型驱动开发方法，其中模型转换是模型驱动开发的关键方法^[5]。系统架构模型开发以系统模型为核心，建立各个视角的领域模

型，通过模型转换（半）自动生成其他视图的模型，实现模型信息的追溯，提高建模效率。此外，采用模型驱动开发方法，根据高抽象层次架构模型生成低抽象层次架构模型，系统架构师不需要过早考虑平台相关的实现细节，将注意力集中于系统整体的业务逻辑。目前已经存在多种模型转换方法实现某些特定领域模型之间的转换，在多学科交叉的复杂系统开发场景下，异构的领域模型之间的转换需要借助多种模型转换方法，应用多种模型转换语言，导致架构模型开发难度和领域工程师的学习成本增加。

1.1.2 研究意义

针对复杂系统架构设计过程中现有模型转换方法对多领域异构模型转换支持不足，以及缺乏可实现统一描述的模型转换语言的问题，本文提出了基于图论的复杂系统架构模型转换方法，并提出模型转换统一描述语言，实现多领域异构模型的转换和描述，提高系统开发效率，降低系统工程师学习成本。其意义在于：

（1）提高复杂系统架构模型开发效率。本文提出的模型转换方法能够实现模型的（半）自动生成，在架构模型设计过程中，领域专家针对领域建模语言制定模型转换规则，工程师基于已建立的系统架构模型应用模型转换规则（半）自动生成模型，在生成的目标模型基础上进行模型精化，无需从无到有构建模型，提高模型开发效率。

（2）提高模型信息的可追溯性。本文提出的模型转换方法面向基于 GOPRR-E 元建模框架的架构模型，基于图论形式化元建模框架的各模型元素，将异构架构模型转换问题转变为同构的图转换问题，实现跨领域架构模型的转换。在模型转换过程中保留了追溯信息，源模型信息变更后，重新应用模型转换即可同步变更信息，保证模型信息可追溯性。

（3）降低工程师学习成本，减轻语言学习负担。本文在支持跨域模型转换方法基础上，提出模型转换统一描述语言，系统开发工程师无需学习多种模型转换语言即可实现多领域异构模型之间的转换描述，降低系统工程师学习成本，提高开发效率。

1.2 课题来源

本课题来源于国家重点研发计划“复杂产品设计制造服务一体化系统建模理论”，本课题的研究内容是项目研究中的关键组成部分。

1.3 国内外研究现状

复杂系统架构设计模型转换过程中，主要涉及以下三方面研究：架构模型构建方法、架构模型转换方法和模型转换语言，下文从这三个方面的研究现状进行综述。

1.3.1 架构模型构建方法

基于模型系统工程（MBSE）被广泛应用于复杂产品的架构设计。在 MBSE 的开发过程中，需要建立多种领域模型，以描述系统各个视图的架构。这些模型通常是异构的，基于特定域建模语言建立，例如 SysML（Systems Modeling Language）^[6,7]是国际系统工程学会 INCOSE 和 OMG（Object Management Group）在 UML 基础上制定的一套适用于描述系统整体架构的建模语言，现已成为构建系统架构的标准建模语言。UML（Unified Modeling Language）建模语言常用于软件工程领域，描述信息系统架构。AADL（Architecture Analysis & Design Language，架构分析和设计语言）是用于嵌入式领域的标准化架构描述语言，描述嵌入式系统的软件和硬件架构^[8]。

为了元建模过程的规范化，OMG 制订了元对象机制（Meta-Object Facility, MOF），一种由 M3-M0 层次结构描述的元建模框架^[9]。MOF 采用 UML 作为元元模型，但是 UML 作为软件工程领域的描述语言，应用在通用领域的描述过程中太过复杂，且不易理解。目前已经有很多其他元元模型作为 M3 层的元元模型。ARIS（The Architecture of Integrated Information Systems）是用于企业的建模方法^[10]，提供了多种建模符号作为元元模型，用户可以根据现有的语言创建全新的领域建模语言。Ecore^[11]是 EMF（Eclipse Modeling Framework）建模框架使用的元元模型，目前 ECore 是被采用最多的元元模型，很多建模工具都基于 ECore 开发。GME（Generic Modeling Environment）^[12]是一个主要应用于电子领域的特定域建模工具，其定义的元元模型主要用于开发电子领域模型。MetaCase 公司在元建模工具 MetaEdit+^[13]中提出 GOPPRR 元元模型，支持多种建模概念，实现面向通用领域建模语言的描述。

国内关于元建模方法的研究虽较为缓慢，但不少大学机构正在深入研究。麻志毅^[14]在 MOF2.0 的基础上扩展提出一种元元模型，实现建模语言的定义，并在元建模平台 Meta-Modeler 的实现。北理工王宏伟^[15]等在 GOPPRR 基础上扩展了 GOPPRR-E 元元模型，以满足描述 MBSE 中系统全生命周期架构模型的需求。

1.3.2 架构模型转换方法

模型转换是模型驱动开发的核心技术，国内外学者对其进行了深入的研究，尤其在计算机领域目前已有丰富的理论成果及标准，并开发出相应的平台及框架支持模型转换。现有的模型转换方法主要有直接转换方法、结构化驱动型方法、基于图表转换方法、关系型方法以及混合型方法^[16]、基于 XML (Extensible Markup Language) 的模型转换^[17]，其中发展最为成熟且应用最为广泛的标准及框架为 QVT (Query/View/Transformation)^[18]及 EMF (Eclipse Modeling Framework) 建模框架。

(1) QVT 模型转换标准

QVT (Query/View/Transformation，即查询/视图/转换) 是 OMG 建立的用于模型转换的标准化语言^[19,20]。QVT 使用对象约束语言 (OCL)^[21]、元对象工具 (MOF)^[22]，并遵照模型驱动架构 (Model Driven Architecture, MDA)^[23]规范。领域工程师可以根据领域需求制定相应的模型转换规则，同时，可以借助现有的模型转换工具以实现模型之间的转换。QVT 为模型到模型转换定义了 QVT-Relations 声明性语言、QVT-Operational Mapping 命令式语言和 QVT-Core 低层声明性语言。QVT 定义了模型转换语言的标准，很多模型转换语言基于此标准，例如 SmartQVT^[24]、MOLA^[25]语言。

(2) EMF 建模框架

Eclipse 建模框架 (Eclipse Modeling Framework, EMF)^[26]是由 IBM (International Business Machines Corporation) 支持的一种用于建模的框架，也是 Eclipse 平台实现基于 MDA 模型开发的重要组成部分，可将 EMF 集成到 Eclipse 平台中以实现特定的功能。同时，该框架可以将 XML 文档、UML 等文件所描述的特定模型生成高效的、正确的和易于定制的 Java 代码，并以其简单而强大的建模能力^[27]，成为当前主流的开发框架，主要应用在软件工程领域，难以支持异构模型的模型转换。

国内开展模型转换的深入研究主要集中在计算机领域及软件工程领域，大部分研究基于现有模型转化技术进行扩展以此应用于特定领域。例如，邓刘梦^[28]使用扩展的 SysML 模型对需求与产品间的纵向追踪关系进行建模，采用利用 ALT (ATLAS Transformation Language) 转换语言来实现追踪模型的自动生成并实现追踪信息的图形化表达。王宁波^[29]利用 QVT 实现平台无关模型到平台相关模型的转换，并对转换过程中模型的变化进行控制。曾一等人^[30]基于 ECore 和 EMF 框架开发出一套模型转换框架，实现模型转换。李燕^[31]研究了软件工程领域的模型转换问题，采用类图描述

的领域模型 HBase 数据模型的转换。在论文^[32]中作者同样基于 ATL 语言,使用 SysML 和 profile 扩展元模型定义转换规则,实现了 SysML 中需求模型到用例模型及活动模型的转换,另外也使用了 OCL 对元素进行一定限制,实现了模型转换在系统工程领域的简单应用。

1.3.3 模型转换语言

模型转换语言用于描述模型转换,目前存在很多模型转换语言支持模型转换,大部分语言是从学术界研究并提出的,例如用于支持模型到模型转换的 ATL 语言^[33]、GReAT 语言^[34],Fujaba^[35]等。国内在模型转换语言研究方面起步较晚,研究方向主要集中于在现有的语言基础上做扩展,也实现了一些可用的工具。

ATL (ATLAS Transformation Language) 是一种基于规则的模型转换语言,基于 EMF 框架实现模型的单向转换,支持语法检查、转换调试、以及语言解释^[36]。ATL 语言的转化规则定义了源模型元素如何匹配和导航以创建和初始化目标模型元素,在转换过程中,元模型、模型都基于 EMF 描述,目标模型的属性值可以在源模型属性值的基础上通过 OCL (Object Constraint Language) 表达式计算得出。ATL 是声明式和命令性的混合语言,声明式语言简单描述简单的映射关系,对于一些过于复杂而无法通过声明方式处理的映射可以通过命令式的结构指定。

GReAT 语言可描述 GME (Generic Modeling Environment) 环境中基于元模型的转换,GReAT 支持跨域转换以及多个模型转换^[37],GReAT 拥有丰富的模式规范子语言、图转换子语言和高级控制流子语言以满足模型转换领域的特定需求。

Henshin 是 EMF Tiger 转换语言的扩展语言,基于 EMF 建模框架。使用三重图语法 (Triple Graph Grammar, TGG) 进行模型到模型的转换。Henshin 通过转换模型对转换本身进行建模,转换模型由输入图、输出图和对应映射的列表组。因此 Henshin 可以实现高阶转换^[38],即转换模型本身也是可以通过模型转换生成的。

还有很多转换语言是基于 QVT 标准语言的,例如 MT (Model Transformation) 语言、QVT-Operational, SmartQVT 以及 Eclipse M2M。MT 语言是一种轻量级的转换语言,支持模型的单向转换,具有与其他转换语言相同的特点。与其他常见的模型转换语言相比,它还提供了一组附加的新特性,支持模型到模型、模型到代码、代码到模型、模型到文本的转换。MT 语言的输入输出可以是一个或多个模型^[39]。

近年来,国内也对模型转换语言有了深入的研究。何啸等人^[40]提出了针对模型转换程序的图形化建模语言——VisTML,实现对模型转换程序的可视化建模,以降低大规模复杂程序的转换难度,增进软件开发人员之间的沟通与协作,并提出了支持 VisTML 语言的支持工具 TModeler。VisTML 实际上是一种建模语言,对模型转换过程进行建模,面向解决实际工程中的开发问题,本质上是对工程中的常用概念进行抽象,其目标并不是面向通用领域。

王学斌等人^[41]也提出了一种基于 QVT 的模型转换语言 QMTW,通过提高模型转换抽象层次的方法屏蔽多种转换语言的差异性,以降低模型转换在工程应用中的学习成本和开发成本。QMTW 提出了模型编织抽象层,在此抽象层上定义一组转换规则,再生成具体的模型转换语言。

1.3.4 研究现状分析总结

根据上述国内外研究现状分析可知,模型转换的概念是从软件工程领域发展而来,该领域学者们已经在模型转换方面开展了深入的研究,在理论、方法以及工具方面取得了研究成果。系统工程领域结合模型驱动开发方式,借鉴了软件工程中模型转换的概念,多数研究集中在模型转换在系统工程领域的应用的阶段。国内外研究存在的问题主要表现在以下几个方面:

1) 架构模型的构建方法不统一导致模型数据异构,导致跨域模型转换难以实现。虽然 MOF 元建模框架在一定程度上规范了模型的构建流程,但 MOF 采用 UML 作为元元模型,主要面向软件工程领域的模型构建。在涉及多领域的 MBSE 领域,需要基于 MOF 选择一种具备描述多领域模型能力的元元模型,实现异构模型底层数据的统一描述,为跨域模型转换的实现奠定基础。

2) 缺少通用的模型转换方法实现多领域异构模型之间的转换。模型转换主要应用于软件工程领域,在采用 MBSE 方法的复杂系统架构设计场景下,针对不同领域的模型,需要选择多种模型转换方法以支持对应领域的模型转换,领域工程师经过多次模型转换才能得到各领域架构模型,减低开发效率,增加开发难度。

3) 缺乏可实现统一描述的模型转换语言。在复杂系统架构设计过程中各个阶段涉及到多种领域模型,这些异构模型的转换需要应用多种转换语言描述,导致在设计过程中相关模型转换语言不统一。

1.4 研究内容及组织结构

针对目前存在的问题，本文提出一种基于图论的复杂系统架构模型转换方法，并以此为基础提出架构模型转换统一描述语言，主要研究内容如下：

（1）面向跨域系统架构模型的模型转换方法研究

以 GOPRR-E 元元模型扩展的元建模框架为基础，构建系统架构模型，实现多领域异构模型的统一表达，为跨域模型转换奠定基础。以图论中的图转换为理论基础，将 GOPRR-E 各层模型元素形式化为图论概念，将异构模型的转换问题转变为图转换问题。首先 ① 提出跨域模型转换的概念，② 将架构模型中的相关概念映射为类型图、图等图论概念，具体包括元模型的映射、模型的映射，在此基础上，将模型转换规则映射为图转换规则，基于图转换中的图重写实现架构模型的转换。③ 最后，设计模型转换流程，实现模型转换的自动化。

（2）模型转化正确性的形式化检验方法研究

该部分围绕如何避免模型转换生成的缺陷模型展开。① 首先给出模型转换正确性定义，并给出转换正确性的形式化描述。② 其次，基于图论中的类型图概念，将模型转换正确性问题转换为类型图一致性验证问题，并提出四种一致性条件。③ 最后，提出模型转换自动验证流程，对一致性条件进行验证，检验转换过程中生成的缺陷模型，保证模型转换的正确性。

（3）模型转换统一描述语言研究

该部分在上述两个研究点的基础上，重点围绕如何解决模型转换统一描述问题展开。① 提出面向 GOPRR-E 架构模型的基于规则的模型转换描述语言，重点引入模型模式的概念对模型结构进行描述，并提出四种典型的模型模式。② 此外，从模型转换语言的抽象语法和具体语法两方面对模型转换语言进行设计。③ 最后，开发模型转换语言解析器和执行引擎架构，实现模型转换语言的描述能力。

本文的研究思路如图 1.1 所示。本文共分为 6 章，各章具体内容安排如下：

第 1 章，绪论。本章从复杂系统架构模型转换的背景、意义出发，综述并总结国内外对系统架构模型构建方法、模型转换方法、模型转换语言的研究现状，通过分析目前研究所存在的问题，引出本文的主要研究内容以及研究思路。

第 2 章，复杂系统架构模型转换理论与框架研究。分析基于 GOPRR-E 元建模框

架的架构模型构建方法，以及模型转换技术、模型转换规则、模型转换语言等技术背景，并提出本文模型转换框架，作为后续工作的理论基础。

第 3 章，面向跨域系统架构模型的模型转换方法。本章面向 GOPPRR-E 元建模框架支持的系统架构模型，定义跨域模型转换概念，阐述在图论支持下的架构模型转换方法。提出模型转换到图论的映射方法，将架构模型转换问题转变为图转换问题，包括架构模型到图的映射、元模型到类型图的映射、模型转换规则到图转换规则的映射，利用图转换中的图重写实现架构模型的转换。最后，基于上述映射方法，提出复杂系统模型转换流程。

第 4 章，架构模型转换正确性的形式化检验方法。本章主要介绍模型转换的正确性验证，给出模型转换正确性的形式化定义。在第 3 章的模型映射方法的基础上，通过类型图一致性检查方法实现模型转换正确性验证，分析并总结出四种类型图一致性条件。最后，提出模型转换自动验证流程，以自动检验模型转换过程中可能生成的缺陷模型。

第 5 章，模型转换统一描述语言研究。在上述研究基础上，提出基于规则的模型转换描述语言，从抽象语法和具体语法两个方面对语言进行介绍，重点引入模型模式的概念对模型结构进行描述，在分析 GOPPRR-E 模型结构基础上提出四种典型的模型模式。最后开发模型转换语言解析器和执行引擎，实现模型转换语言的描述能力。

第 6 章，以无人平台系统架构设计为例，对上述研究进行实例应用和验证。重点验证模型转换方法支持工程问题中系统架构模型转换的有效性，以及模型转换语言对系统架构模型转换的描述能力。

最后，在总结与展望部分，对本课题的研究工作进行总结，归纳了研究成果，阐述了本文的主要创新点，并针对研究过程中的不足，提出后续的研究方向。

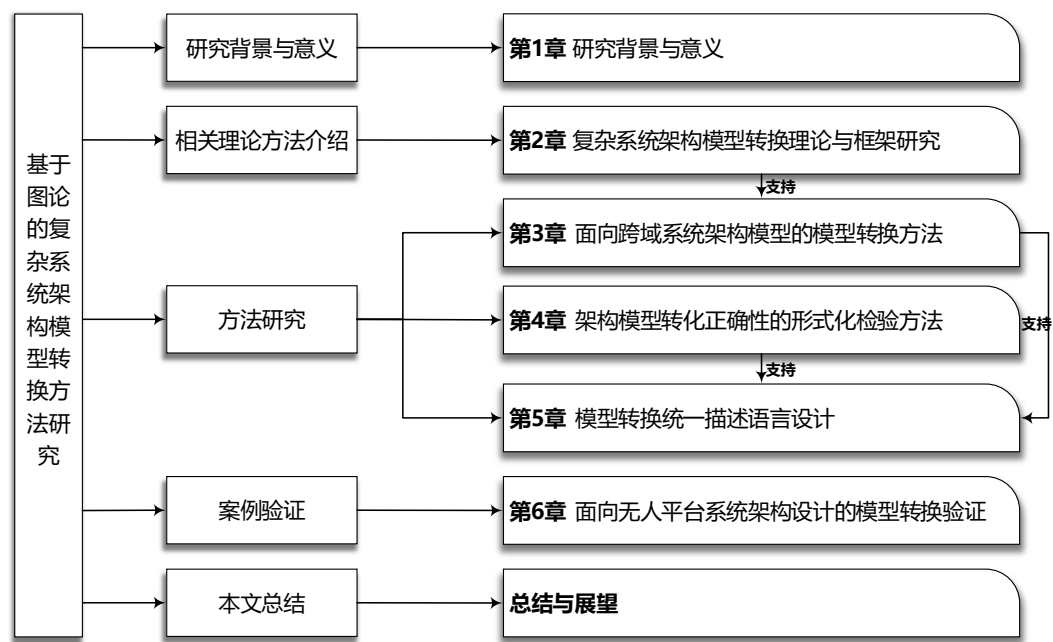


图 1.1 论文组织结构图

1.5 本章小结

本章分析了课题研究的背景和意义，分析总结了国内外关于架构模型转换与模型转换语言的研究现状。通过总结目前研究现状存在的问题，确定本文的研究内容，并进一步对研究思路进行梳理，提出论文的结构框架。

第2章 复杂系统架构模型转换理论与框架研究

2.1 引言

采用 MBSE 方法开发复杂系统架构是一个模型驱动开发的过程，系统架构模型的构建和模型转换是其中两项关键内容。前者通过构建特定领域模型，对整体系统的某个视图进行描述，后者通过模型转换方法，将现有的领域模型自动转换为用于描述其他视图的领域模型，或者同视图下更细粒度的模型。

架构模型的构建是模型转换的基础。复杂系统的开发往往具有多系统层级和多领域的特点，在架构设计过程中涉及大量不同系统层级，不同领域的异构模型，这些异构模型基于不同建模语言建立，底层数据结构不同，导致架构设计过程中难以统一管理和保证数据一致性。基于 MOF 的元建模方法是解决异构模型统一描述问题的有效方法，通过 M0-M3 元建模框架实现模型的统一描述。

模型转换是模型驱动开发的关键。模型转换是指从源模型按照一定的转换规则，自动转化为目标模型的过程。在系统架构设计过程中，根据模型转换的应用场景可以分为横向模型转换和纵向模型转换。横向模型转换的源模型和目标模型处于同系统层级的不同领域，模型转换用于建立源模型和目标模型之间的数据映射关系，关联不同视图的模型数据。纵向模型转换中源模型和目标模型通常处于同一个领域的不同系统层级，模型转换实现从高层次系统层级（例如系统级）自动生成低层级（子系统级）细粒度模型。目前存在多种模型转换方法，其中基于图转换的声明式的模型转换方法较为直观，且相关的理论研究已经很成熟。因此，本文主要对基于图转换的模型转换方法进行研究。

2.2 基于 GOPRR-E 元建模框架的系统架构模型构建方法

复杂系统的开发是一项涉及多系统多学科的复杂系统工程，在架构设计开发过程中涉及大量不同系统层次，不同领域的异构模型，这些异构模型通常基于多种图形化建模语言（下文中的建模语言特指图形化建模语言）建立^[42]。例如在软件工程领域，常使用 UML（Unified Modeling Language）建模语言描述信息系统；在系统工程领域，使用 SysML（System Unified Modeling Language）语言描述系统需求、架构、行为等；

在嵌入式领域，AADL（架构分析和设计语言）描述嵌入式系统的软件和硬件。这些面向各个学科领域的建模语言统称为特定域建模语言（DSML，Domain Specific Modeling Language），在开发过程中，由于特定域建模语言的异构性，难以实现架构模型的统一转换。元建模技术是解决这个问题方法之一，在分析领域问题的基础上，定制适合于特定领域的领域建模语言，进而使用领域建模语言构建领域模型，实现架构模型的统一构建，MOF 元建模框架是元建模技术中被广泛应用得的元建模框架。

2.2.1 MOF 元建模框架

为了实现特定域建模语言的统一管理，OMG 制订了元对象机制（Meta-Object Facility, MOF）^[43]，通过这套建模框架规范建模流程。由 M3-M0 层次结构描述的元建模框架如图 2.1 所示。

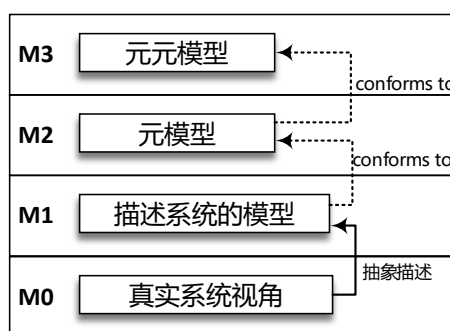


图 2.1 OMG 的元建模框架

MOF 中提出的 4 层模型结构，分别是系统视角（M0 层）、模型层（M1 层）、元模型层（M2 层）和元元模型层（M3 层）。该框架中，每一层的元素严格对应上层模型元素，即上层模型是下层模型的抽象，下层模型是上层模型的实例。MOF 每层有如下含义：

（1）M0 是指真实系统的视角，是模型的实例，用于描述利益相关人视角。例如需求分析师关注的系统需求视角，系统设计师关注的系统架构视角。

（2）M1 表示模型层，以抽象的方式描述真实物理世界，通过建模语言描述。模型是元模型的实例，用于支持系统开发，例如基于 SysML 需求图的需求模型，基于 SysML 包图的物理架构模型等。

（3）M2 表示元模型层，即用于构建模型的模型，元模型是用元建模语言描述的。元模型是元元模型的实例，构建领域元模型的过程实际上就是在定义领域语言。元模

型和特定的领域相关，例如 SysML，BPMN 和 Capella 建模语言是基于各自领域元模型定义的，其中 SysML 中的各种“图”就是 SysML 语言的一种元模型。

(4) M3 表示元元模型层，即用于构建元模型的模型。在 MOF 的规范中，M3 层一般是通过自身描述的，即 M3 层具有反身性。例如一种以 UML 定义元元模型的方式，采用 UML 类图描述元元模型本身。

2.2.2 GOPPRR-E 元元模型和元建模

在 MOF 元建模框架中，M3 层是整个建模框架的基础。MOF 采用 UML 作为元建模语言，但 UML 作为软件工程领域的描述语言，不适合描述通用领域架构模型。因此，有多种 M3 层的元元模型被扩展以实现通用领域模型的描述。Kelly^[44]从现有的元建模语言中总结并提取出目前被广泛使用的几种元元模型，例如 (ARIS, Ecore, GME, GOPPRR, MS DSL Tools 和 MS Visio)，其中 GOPPRR 元元模型描述能力强，能够支持更加复杂的模型概念，在建模描述方面更具优势。本文基于 GOPPRR-E 元元模型扩展 MOF 框架，GOPPRR-E^[15]元元模型在 GOPPRR 元元模型基础上添加了一定扩展 (如图 2.2 所示)，在架构模型的构建中能够更清晰描述领域模型元素之间的关联和约束。GOPPRR-E 是 G (Graph) O (Object) P (Point) P (Property) R (Relationship) R (Role) E (Extension) 的缩写，即包括图、对象、关系、点、角色、属性六种基本的元素以及扩展的衍生概念。

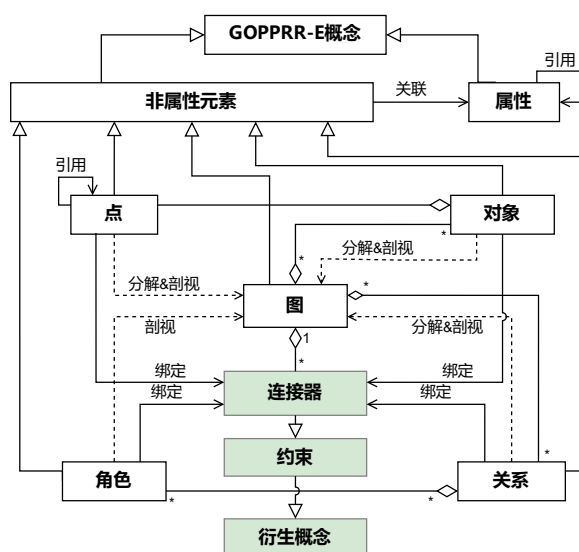


图 2.2 GOPPRR-E 元元模型

GOPRR-E 元元模型包含属性元素和非属性元素两类。非属性元素包括图、对象、点、角色、关系，描述领域的关键概念，刻画元模型中不同类型的元素，例如实体或连接关系。属性元素用于描述元模型的指标特性。GOPRR-E 元元模型的元素概念如图 2.2 所示，表 2.1 对各元素进行解释。

表 2.1 GOPRR-E 元元模型含义

元元模型	名称	描述
Graph	图	表示模型的某个视图，是“对象”、“关系”和“角色”类型的集合。图类型相当于一个容器，可以包含若干个对象、关系和角色类型，图类型可以具有“属性”。
Object	对象	用于表示元建模语言中的概念。是包含在图中的主要元素，可以独立于关系和角色单独存在的实体。对象可以包含若干个属性，其具体语法通常表现可以连接的实体
Point	点	描述对象中与关系（或角色）连接的端口，点用于限定某个对象可以被连接的关系（角色）类型。点不能够单独存在，必须通过绑定在对象的形式存在。
Relationship	关系	包含在图类型中，表示对象之间的关联，通过角色和对象相连
Role	角色	描述对象或对象上的点如何与关系相连接
Property	属性	不能单独存在，必须依附在其他五种元元模型上才有意义，用于描述语言概念的属性。属性可以具有不同的数据类型（字符串，文本，数字，布尔值，集合等）

GOPRR-E 元元模型作为构建领域建模语言的基础，也遵循一定的语法以约束六种元素之间的关联关系，在 GOPRR-E 框架中，这些规则和约束是通过扩充的衍生概念描述的，E（Extension）扩充的衍生概念主要有如表 2.2 所示几种：

表 2.2 GOPPRR-E 元元模型的衍生概念

关联规则	描述
包含	描述元元模型之间的组合关系，其中，图可以包含对象、关系，对象可以包含点
属性连接	描述图、对象、关系、角色、点五种类型元素和属性的所属关系
剖视	描述一个对象（关系、点、角色）实例可以映射为多个图实例
分解	描述一个对象（关系、点、角色）实例可以一一映射为一个图实例
绑定	描述对象、点、角色等之间的约束方式，通过绑定的方式可以定义元建模语言的规则。其中，一个对象的端点对应多个角色，每个角色至少对应一个对象的端点，每个关系绑定两个角色。
关联	描述对象（或点）和角色之间的连接关系

2.2.3 架构模型构建流程

MOF 通过 4 层模型的概念规范了领域模型构建过程，通过在 M3 层的统一描述实现不同领域架构模型的统一描述。基于 GOPPRR-E 元元模型构建系统架构模型过程中，领域系统工程师根据领域相关知识、规范、约束及标准，利用 GOPPRR-E 元元模型设计领域元模型，形成建模语言，领域工程师利用设计好的元模型构建领域模型，从而实现不同的建模需求，因此，基于 GOPPRR-E 元元模型的元建模框架支持对异构模型的统一描述^[45]。系统架构模型构建流程如图 2.3 所示，包含以下步骤：

- （1）基于 GOPPRR-E 元元模型，定义元建模语言；
- （2）在元建模语言基础上描述领域元模型，该过程也称为元建模过程；
- （3）基于领域元模型，定义领域建模语言（DSL），例如定义 SysML，UML 等建模语言；
- （4）使用领域建模语言开发领域架构模型，描述某一个系统视图，该过程称为建模过程。

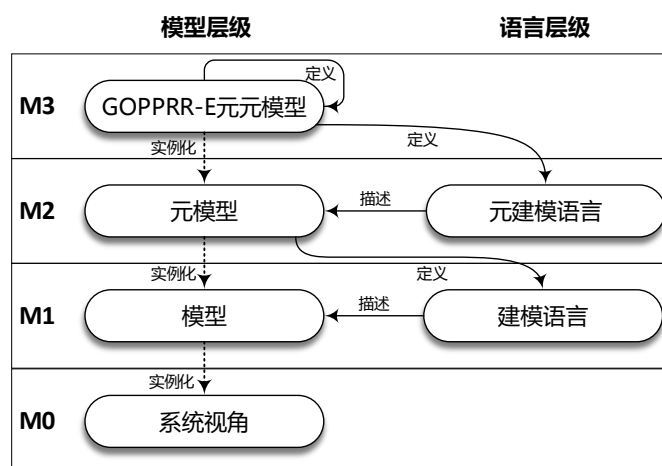


图 2.3 基于 GOPPRR-E 元建模框架的架构模型构建过程

2.2.3.1 元模型的构建过程

元模型的构建过程被称为元建模，即元建模的产物是元模型。元建模过程涉及到一种特定域语言，这类语言被称为元建模语言，例如本文中的 GOPPRR-E 元建模语言，GOPPRR-E 元建模语言是通过 GOPPRR-E 元元模型定义的^[46]。

元元模型具有与特定域无关的抽象描述能力，不包含领域学科相关信息，具有通用描述能力。元模型是领域概念的抽象描述，包含特定领域相关的信息，定义一个领域模型所需要的构造元素和规则，受到领域相关标准、规范等的限制。因此，元模型构建过程中，需要参考领域概念、标准、规范等，使用 GOPPRR-E 元建模语言对其进行描述，形成领域元模型。

2.2.3.2 模型的构建过程

模型的构建过程称为建模，即建模的产物是模型。建模语言是特定域建模的基石，特定域模型是通过建模语言描述的，例如常使用的 UML，SysML 语言都属于特定域建模语言，建模过程包括建模语言定义和模型构建两个步骤。

从语言学的角度分析，建模语言包含具体语法和抽象语法。元模型定义了建模语言的抽象语法。抽象语法描述了建模语言中能够包含的概念、关系和约束，并对建模语言元模型要素之间的连接规则及约束进行定义，为建模的正确性、模型约束检查提供参考。具体语法定义该领域下模型元素的表示形式，即在建模环境中模型元素的图形化样式，例如模型通过图形化形式或文本形式描述。在抽象语法的基础上，参照领

域相关的规范、标准等定义建模语言的具体语法，最终形成建模语言。

基于定义好的领域建模语言，系统工程师根据业务场景构建对应系统层级、对应领域的架构模型。复杂系统架构设计通常遵守 RFLP（需求-功能-逻辑-物理）建模流程。首先对利益相关人的需求进行整体分析，然后进行整体功能分析、逻辑分析和物理架构分析，并构建对应的架构模型。通过反复迭代、逐层分析的方法，完成从系统及到分系统等各层级的架构模型构建，且在建模过程中各系统层级涉及到机械、电气、液压、控制等特定域模型。整个系统架构模型的构建涉及大量特定域建模语言，基于 GOPRR-E 元元模型，能够实现各视图架构模型的统一描述，为多领域异构模型转换奠定基础。

2.3 架构模型转换方法

在基于元建模方法开发复杂系统架构模型过程中涉及大量领域模型，手动建立模型效率低下，且难以追溯模型之间的关联信息。模型驱动架构（MDA）思想正在越来越多地运用到复杂系统架构模型的构建过程，以提高开发效率。模型转换是其中的关键方法，模型转换提供了操作模型的方法，可视为与一种特殊的程序，以架构模型作为输入，自动创建模型作为输出，使得模型可以被操作^[47]，保证模型之间的信息传递。模型转换语言用于描述模型转换，对输入模型以及如何生成输出模型进行描述，通过声明式或命令式的计算机语言描述输入、输出模型之间的映射关系。

2.3.1 模型转换的定义

模型转换被广泛应用于软件开发领域，实现模型驱动的软件设计开发。模型转换术语有多种定义方式，Tratt^[48]将模型转换定义为“将一个模型转换为另一个模型的程序”。OMG（对象管理组织）将模型驱动体系结构（MDA）中的模型转换定义为“将一个模型转换为同一系统另一个模型的过程”。Kleppe 等人^[49]将模型转换定义为“根据转换描述，从源模型自动生成目标模型”。Mens 等人^[50]扩展了这一定义，允许使用多种模型作为输入或输出，将模型转换定义为“根据转换描述，从一个或多个源模型自动生成一个或多个目标模型”。

本文面向复杂系统架构模型设计场景，给出如下模型转换定义：模型转换是指根据模型转换语言描述，在一个源模型基础上应用一系列模型转换规则，生成并输出一

个目标模型的过程，其中允许源模型和目标模型基于不同元模型建立。本文研究的模型转换方法主要针对由图形化建模语言构建的架构模型，且架构模型基于 GOPPRR-E 元建模框架构建，源模型和目标模型的元模型均为 GOPPRR-E 元模型的实例，因此，本文提出的模型转换能够实现跨域模型转换。

2.3.2 模型转换分类

模型转换以其高效的模型驱动开发方式，被广泛应用于软件开发领域和系统架构开发领域，目前模型转换技术有多种分类方式。根据模型转换目标不同可以分为模型到模型的转换，模型到代码的转换^[51]，后者常用于软件工程领域实现代码的自动生成^[52]。本文所研究的模型转换特指源模型到目标模型的转换。

根据源模型和目标模型的元模型是否相同，可以分为内生模型转换和外生模型转换^[53]。外生模型转换是指源模型和目标模型所遵守的元模型不同，允许在不同元模型之间建立映射关系，实现不同领域模型之间进行转换，即跨域模型转换。

内生模型转换是指源模型和目标模型的元模型相同，在某一种建模语言内部进行转换，例如，优化模型的性能，同时保留其语义。内生模型转换通常被应用于同领域不同系统层级的转换，在源模型基础上，不改变模型语义，将高抽象层次的模型转换为更具体的低层次抽象模型。架构模型的开发往往是不断迭代的，从最初的顶层架构粗粒度模型出发，不断迭代优化，最终形成细粒度、低抽象层次架构模型，内生模型转换可以最大程度地实现顶层架构模型信息的重用，且保证架构模型信息向下传递的一致性。

根据模型转换场景中源模型和目标模型抽象层级是否改变，可以分为横向的模型转换和纵向的模型转换^[54]。复杂系统架构开发遵循需求-功能-逻辑-物理的开发流程，流程中的每个阶段涉及大量多领域模型。这些模型按照抽象层级可以分为横向和纵向两类，不同开发阶段涉及到的模型处于不同抽象层级，从需求到物理抽象层次依次递减。同一个阶段不同领域的模型具有相同的抽象层级。

纵向转换是指源模型和目标模型属于两个不同的抽象级别。通常模型转换过程中模型抽象层次不断降低，高抽象层级模型定义接口和规范，具体的架构定义由低抽象层级模型实现。

横向转换是指源模型和目标模型的抽象层级相同。横向模型转换通常被应用于模

型信息同步，同抽象层级包含多个视角的架构模型，通过水平模型转换关联不同视图之间的模型数据，以实现其中某个视图模型信息改变时，其他视图模型同步修改。

2.3.3 模型转换方式

模型转换的实现需要具体的转换理论指导，目前存在的模型转换方法主要有以下几种。

（1）直接转换方式

直接转换即通过预先提供的 API（应用程序接口）对模型进行操作^[55]，需要具备一定的编程基础，通常被应用于面向对象的框架中。直接转换方式更多面向软件开发领域，在实际应用过程中需要掌握一门或多门编程语言实现转换。

（2）基于关系代数的模型转换

该种转换方法基于代数关系理论，通过关系代数定义模型之间的映射关系^[56]。其思想是定义一种模式，通过“关系对”表达源模型元素到目标模型元素之间的一个映射，通过关系代数中的“关系”形式化模型转换的定义，关系代数的性质用于形式化模型转换中的约束。

（3）基于元模型的模型转换

该方法是 MDA 规范中使用的模型转换方法，其核心思想和基于关系代数的模型转换类似。通过在 M2 元模型层定义映射关系，在进行模型转换时，针对 M1 层具体的模型实例，以 M1 层的映射关系作为转换规则执行，生成目标模型^[57]。在元模型层次上建立对应模型元素实例之间的映射 f ，当进行模型转换时，根据 f 的定义从源模型生成目标模型。

（4）基于图的转换方式

该方法基于图论和图转换理论，将模型抽象为类型图、属性图、标记图等图概念，通过图重写技术操作图形，实现模型的转换^[58]。一个图转换由一组图转换规则描述，图转换规则由左图、右图和约束条件组成，左图对源模型中满足约束条件的部分进行模式匹配，源模型实例匹配成功后被右图模型实例取代，生成目标模型。图转换方式是一种声明式的模型转换，架构师在开发过程中无需关注模型转换实现细节，只需要定义模式对应关系，降低使用难度。

2.3.4 模型转换语言

模型转换语言用于描述模型转换的执行，指定如何将一个源模型转换为一个目标模型。模型转换语言具有一个词汇库、一组语法以及定义良好的语义来执行模型转换。模型转换语言是基于某种模型转换范式，如果转换描述的语言是基于规则的，则模型转换通过一组模型转换规则描述^[59]，规则指定如何将源模型的片段转换为目标模型的片段^[60]。模型转换语言可以基于文本式或图形式，前者具有形式化的语言语法，后者具有简单清晰的描述方式。按照语言范式区分，可以将模型转换语言分为命令式、声明式和基于图转换方式。命令式模型转换语言将转换描述为一系列动作的序列，在转换过程中，需要明确地控制模型转换规则执行顺序。声明式模型转换语言描述源元模型到目标元模型之间的对应关系，但没有显式地指出规则执行流程，这种描述方式更为简洁直观。图转换语言建立在图论基础上，通常应用于形式化方法和证明，模型被解释为图。图转换语言是声明式语言的子集，通过图描述转换规则^[61]，包括规则左图（LHS），规则右图（RHS）以及 LHS 和 RHS 之间的映射关系。

2.4 架构模型转换框架

基于目前模型转换理论的研究，本文提出一种基于图论中图转换的模型转换方法，实现支持跨领域的多领域系统架构模型转换。此外，并提出一种基于文本式、声明式的模型转换统一描述语言实现模型转换流程的描述。模型转换框架如图 2.4 所示。其中，模型转换规则定义在元模型层，模型的转换执行在模型层，使得模型转换描述针对领域元模型而不是具体的模型，实现模型转换规则的重用。

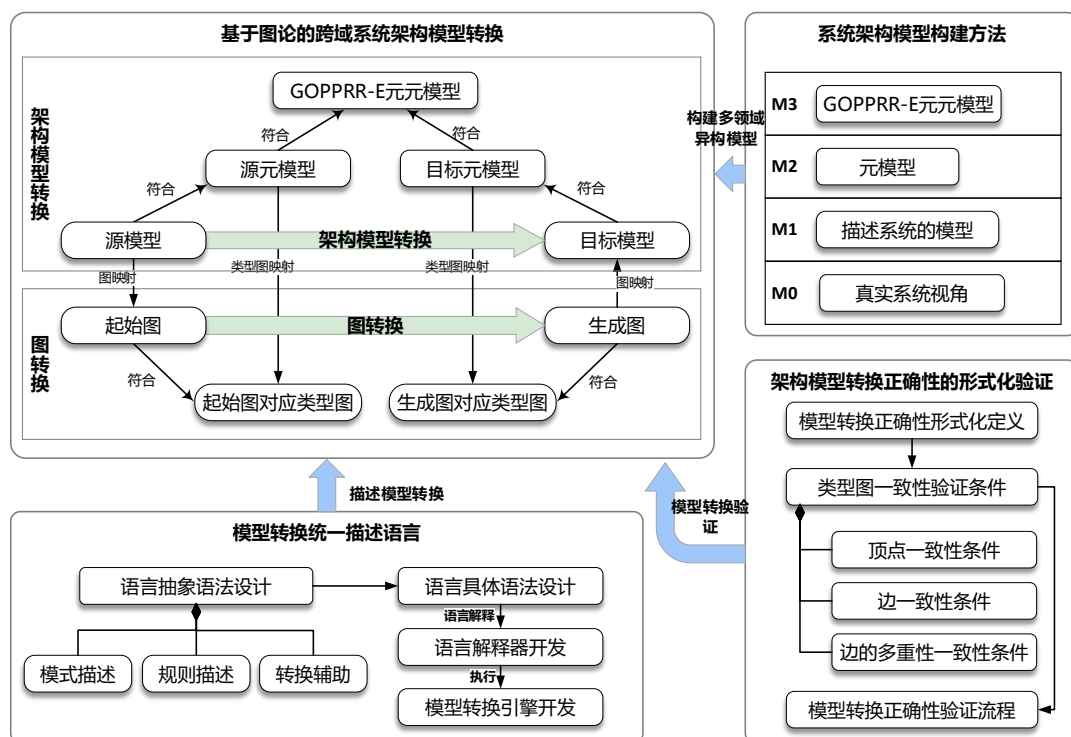


图 2.4 模型转换框架

2.5 本章小结

本章主要介绍了复杂系统架构设计方法和模型转换方法，为后续研究提供理论依据。在架构模型构建方法中主要介绍基于 GOPRR-E 元元模型的元建模框架，该框架从元元模型层实现异构多领域架构模型统一，且相比其他元元模型，GOPRR-E 元元模型支持更多建模概念，更适合于复杂系统架构模型设计场景，统一的架构模型描述为跨域模型转换奠定基础；在架构模型转换方法中重点介绍模型转换的定义和模型转换语言，在总结现有模型转换方法的基础上给出本文的模型转换定义。最后结合复杂系统架构模型转换场景和现有模型转换方法提出本文的模型转换框架，为下文研究提供指导。

第3章 面向跨域系统架构模型的模型转换方法

3.1 引言

采用模型驱动方式开发复杂系统架构模型过程中，模型转换的典型应用场景是跨域模型转换。本文基于图论中的代数图转换实现架构模型的转换，以图论为形式化理论基础主要考虑到以下几个原因：（1）图是架构模型的自然表示，本文所提模型转换方法主要针对图形化建模语言构建的架构模型，基于图论形式化架构模型较为直观；（2）图论中的代数图转换已形成成熟的理论基础和相关工具，现有的技术体系为模型转换应用提供了形式化理论和技术支撑；（3）基于图转换的模型转换规则是一种声明式的编程范式，描述更加简单、直观，图转换规则的复杂性被隐匿，工程师易于学习理解。

3.2 跨域模型转换的概念

在系统架构模型开发过程中的模型转换往往是不同领域模型之间的转换。复杂系统架构设计是一项涉及多学科的复杂系统工程，例如在无人平台架构开发过程中，涉及到结构方案设计、电气方案设计、电讯方案设计以及控制系统方案设计等，其架构模型的开发涉及到机械、电子、液压、控制等多领域多学科的交叉融合，在开发过程中，初始的顶层需求被一步步分解到不同领域的子系统中，由对应领域设计人员构建领域模型。这些不同层级不同领域模型之间的数据存在追溯和关联关系，在实际设计过程中往往借助模型转换的技术，自动或半自动生成领域模型，在生成模型基础上构建完整模型，保证模型信息可追溯性，同时提高开发效率。

跨域模型转换是指不同领域的特定域模型之间的转换，其源模型和目标模型基于不同的领域建模语言构建。跨域模型转换的一个关键特点转换过程中元模型的改变，由于领域建模语言由领域元模型定义，因此跨域模型转换中源元模型和目标元模型不同^[62]。实现跨域模型转换的前提是将异构的源元模型和目标元模型统一到同一个域中，提出转换模型和转换元模型的概念。转换模型（Mt）用于对模型转换过程建模表示，转换模型由转换元模型（MMt）定义，描述如何根据源模型元素生成目标模型元素，是一个由源元模型和目标元模型构成的临时域，如图 3.1 所示。

Tab 是一个模型转换引擎，在执行过程中根据转换模型 Mt，自动根据源模型 Ma 创建目标模型 Mb，这三个模型分别符合 MMt、MMa 和 MMb 元模型，元模型规定了对应模型的抽象语法，在跨域模型转换场景中 $MMa \neq MMb$ 。在元建模框架下，MMt、MMa 和 MMb 元模型都是通过元元模型 MMM 定义的。转换模型 Mt 的构建目的是描述 Ma 和 Mb 元素之间的语义映射，这种描述建立在元模型层，在描述过程中参考 MMa 和 MMb。由于 Mt 本身的抽象语法通过 Mt 定义，因此要求 Mt 能够同时具备两个领域语义，即 MMt 同时涵盖 MMa 和 MMb 的抽象语法，MMt 也称为转换元模型。

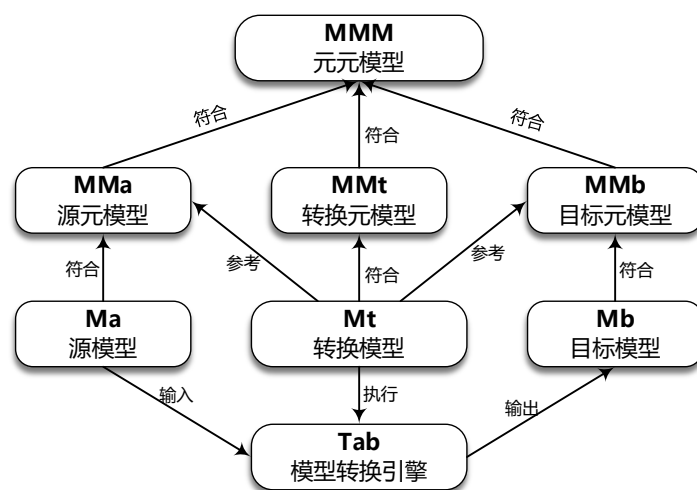


图 3.1 跨域模型转换的模型层级示意图

3.3 GOPRR-E 模型概念到图论概念的映射

本文基于图论中的图转换方法实现跨域的系统架构模型转换，将模型转换问题转变为图转换问题。模型、元模型都可以通过带属性的标记图表示，针对模型的操作通过图文法表示。图的概念被广泛应用于软件设计中，结合具体应用场景具有多种表现形式，例如有向图、属性图、标记图等，但基本上由顶点和边两部分组成。本文基于有向的，带属性的标记图实现元模型和模型的形式化描述^[63,64]。

3.3.1 图转换的基本概念

3.3.1.1 标记图

一个有向的、带属性的标记图，由一个带标记的节点有限集，一个带标记的有向边有限集，两个映射函数 `source`, `target` 以及标签类型映射函数 `label` 组成^[65]，定义为：

$$G = \langle V, E, source, target, label \rangle \quad (3.1)$$

- V 是由图中顶点 v 组成的有限集, $v \in V$ 。
- E 是由图中边 e 组成的有限集, $e \in E$, 边 e 具有方向性, 方向从源顶点指向目标顶点, 通过 $source$ 和 $target$ 两个函数与源顶点以及目标顶点链接。
- $source: e \rightarrow v_s$ 和 $target: v_t \rightarrow e$ 是两个映射函数, 分别表示边 e 和相连的源顶点 v_s 以及目标顶点 v_t 之间的映射关系。
- $label: v \rightarrow L_V \vee e \rightarrow L_E$ 为标签映射函数, 为图中的所有顶点 v 和边 e 赋予类型标签, 其中 L_V 和 L_E 分别表示顶点类型集合和边类型集合。
- 其中, 节点 v 和边 e 都可以定义属性和属性值, 具有标签 T_a , $T_a \in N$, N 是用字符串表示的名称的集合, 标签用于标识顶点和边的类型。

3.3.1.2 图重写

在输入图 G_0 上应用一次图文法规则, 并产生一个输出图 G_1 的过程称为图重写, 记作:

$$G_0 \xRightarrow{R} G_1 \quad (3.2)$$

其中 $G_0 \in G \wedge G_1 \in G$ 。

对于文法规则 $R: LHS \rightarrow RHS$, 在主图 G_0 中寻找 LHS 图匹配, 匹配成功则通过 RHS 替换匹配的图元素, 通过重写生成一个输出图 G_1 。限于篇幅, 文章没有对图匹配、图替换等原理做深入介绍, 详细介绍可以参考文献^[66]。

3.3.1.3 属性图文法

一个属性图文法定义为:

$$GG = \{T, S, R^*, C^*\} \quad (3.3)$$

- T 表示类型图, 定义了图文法中顶点和边的类型, 以及边和顶点之间的拓扑关系和多重性约束。
- S 表示起始图, S 是一种标记图, 描述图文法的初始状态, 针对图文法的图操作是在 S 的基础上进行的。
- R^* 表示文法规则 R 组成的集合, 描述如何对图进行操作。其中, 文法规则 R 记作 $R: LHS \rightarrow RHS$, 其中 $LHS \in G \wedge RHS \in G$, LHS 和 RHS 分别被称作左图和右图。

- C^* 表示约束集, 描述了图文法中属性约束等约束条件, 用于限定文法规则 R 的执行条件。约束的类型包括 NAC (否定应用条件) 和 AC (属性检查条件)。

图文法是一种针对图操作的形式化描述, 图文法重点研究图的生成(图推导)和图结构分析(图规约), 统称为图转换^[67]。图文法将图转换形式化为一系列图重写过程, 采用顺序方式, 在 S 的基础上应用规则集 R^* 中的文法规则 $R_1, R_2, \dots, R_n \in R^* \wedge R_2 \in R^* \dots \wedge R_n \in R^*$, 最终得到图 G' , 记作 $S \xRightarrow{R^*} G'$, 即:

$$S \xRightarrow{R^*} G' : S \xRightarrow{R_1} G_1 \xRightarrow{R_2} G_2 \dots \xRightarrow{R_n} G' \quad (3.4)$$

在应用文法规则集 R^* 对图进行重写的过程中, 图重写的执行受约束集合 C^* 限制, 只有满足约束条件 $C_n, C_n \in C^*$ 的图文法才会进行匹配。此外, 重写的顺序也可能被指定, 例如在 AGG 图转换引擎中, 有一套针对规则执行顺序的控制机制, 可以指定文法规则的优先级。

相比于基于文本的低抽象层级编程语言, 使用图文法形式的计算模型具有以下几个优点: (1) 图是一种自然的、图形化的、高抽象层级的形式化语言, 能够以直观简洁的方式描述图形化的建模语言; (2) 图文法的理论背景有助于证明计算模型的终止性和正确性。然而, 图文法也存在一定的问题, 最明显的是计算效率, 因为在一般情况下, 图匹配过程中的子图同构测试是 NP 完全的, 重写过程计算复杂性过高。不过, 这些劣势可以通过增加的一些优化方法避免, 例如在文法规则的 LHS 上使用小的子图, 检查顶点和边类型以及属性约束条件等, 通过优化手段可以大大减少匹配过程中的时间复杂度。

3.3.2 GOPPRR-E 模型的形式化描述

基于上述对图论的研究, 将元模型抽象为类型图, $GOPPRR-E$ 模型实例抽象为图, 模型转换抽象为图转换, 以图转换解决模型转换问题。从带属性的标记图角度看, 模型可以抽象为图, 模型的元素可以抽象为图的元素(顶点和边), 模型的属性可以抽象为图的属性。领域元模型可以表示为图的约束, 即对图中元素的连接关系进行限制。不同图转换方法对于这种约束有各自的描述方法, 例如 $PROGRES$ 采用 $schema$ 表示约束, AGG 采用类型图表示约束^[68]。本文采用类型图的方法表示约束, 将元模型抽象为类型图以对模型的语义进行约束。在此基础上, 将模型转换抽象为图转换, 因此可以使用图论形式化模型转换。

本文基于图论，在有向的、带属性的类型图基础上，对顶点和边概念进行扩展，结合 GOPPRR-E 模型元素特点给出如下形式化描述。

3.3.2.1 模型图

模型图是一个有向的，类型化的属性图。有向表示顶点之间的边是有方向的；类型化图指的是图中的元素都包含类型信息，图本身以及图的顶点和边都有相应的类型；属性图指图本身以及图的顶点和边具有属性 *attribute*。图定义如下：

$$G = \langle gtype, V, E, attrs \rangle \quad (3.5)$$

其中，*gtype* 表示图的类型，*V* 表示顶点的集合，*E* 表示边的集合，*attrs* 表示图的属性集合，每个属性是由属性类型和属性值组成的键值对。

为了便于表示，对于 $\forall e \in E, \forall v \in V$ ，定义了如下符号：

- $source(e) \in V$ ，表示边 *e* 的源顶点；
- $target(e) \in V$ ，表示边 *e* 的目标顶点；
- $ms(e) \in I$ 表示边 *e* 的起始连接多重性，其中多重性用于类型图，表示 *e* 的源顶点的数量，*I* 是一个整数区间或无穷大；
- $mt(e) \in I$ 表示边 *e* 的目标连接多重性，表示可以 *e* 的目标顶点的数量；
- $name(v), name(e) \in N$ 表示顶点和边的标签名称，其中 *N* 表示字符串集合。

唯一确定图中顶点和边的条件，即唯一性条件，定义为：

- $\forall v_1, v_2 \in V, name(v_1) = name(v_2) \Rightarrow v_1 = v_2$ ，即顶点的唯一性由标签标识；
- $\forall e_1, e_2 \in E, source(e_1) = source(e_2) \wedge name(e_1) = name(e_2) \wedge target(e_1) = target(e_2) \Rightarrow e_1 = e_2$ ，且边 *e* 的起始连接多重性和目标连接多重性分别为 $ms(e_1) \cup mt(e_2)$ 和 $ms(e_1) \cup mt(e_2)$ 。

3.3.2.2 图顶点

图顶点的定义在图论基础上进行扩展，用于表示模型中 *Object* 概念。提出包含“映射类型”顶点的概念，即顶点具有映射类型，顶点类型和属性，定义如下：

$$V = \langle mapType, vtype, attrs \rangle \quad (3.6)$$

其中，*mapType* 表示顶点的映射类型标识，*vtype* 表示顶点的类型，*attrs* 表示顶点的属性集合。映射类型的提出是为了形式化 GOPPRR-E 模型元素，模型中 *Object*、

Point、Role 概念都抽象为顶点，这三种不同类型的顶点通过映射类型标识区分。而 Object、Point、Role 模型元素对应的元模型抽象为顶点类型。具体地，对于 Object、Point、Role 实例进一步细化为 V_O , V_P , V_R 三类顶点，形式化描述为：

$$V_O = \langle objectType, vtype, attrs \rangle \quad (3.7)$$

$$V_P = \langle pointType, vtype, attrs \rangle \quad (3.8)$$

$$V_R = \langle roleType, vtype, attrs \rangle \quad (3.9)$$

3.3.2.3 图的边

图的边连接图中的顶点，GOPRR-E 模型中的 Relationship 可以通过边表示，定义如下：

$$E = \langle etype, src, tar, attrs \rangle \quad (3.10)$$

其中 $etype$ 表示边的类型， src 和 tar 分别表示参照的源顶点目标顶点， $attr$ 表示边的属性集合。

3.3.2.4 类型图

类型图本质上是一种特殊的模型图，区别在于类型图用于约束图中顶点和边的连接关系，同类型顶点只能在类型图中出现一次。其次，类型图中属性只能定义属性类型，而不能包含属性值^[69]。

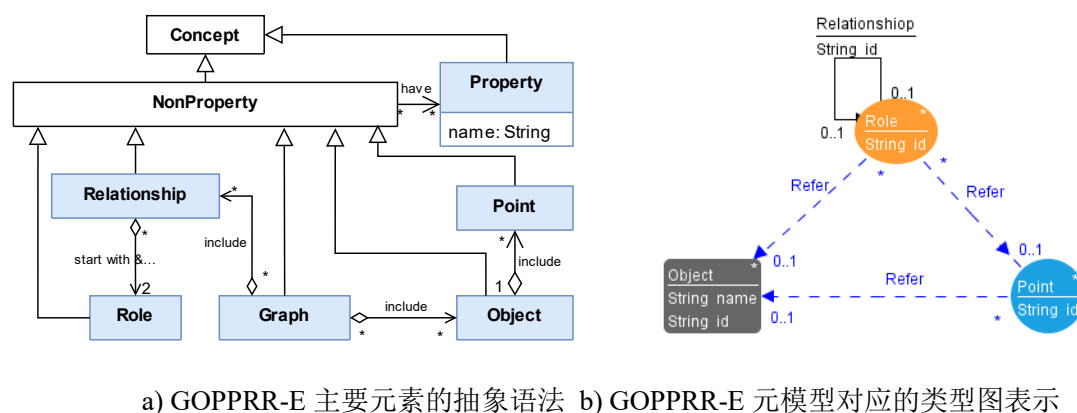
$$TG = \langle V, E, attrs \rangle \quad (3.11)$$

其中， V 表示顶点的集合，且 $\forall v \in V, |v| \leq 1$ ； E 表示边的集合， $attrs$ 表示图的属性集合，类型图以及类型图中得顶点和边，其属性均不能包含属性值。

此外，为了表示类型图和图之间的派生关系，定义了 $type$ 函数，对于模型图 G 的类型图 TG ，有 $\forall v_G \in V_G, e_G \in E_G, \exists v_{TG} \in V_{TG}, e_{TG} \in E_{TG}, type(v_G) = v_{TG} \wedge type(e_G) = e_{TG}$ ，即对于图 G 的顶点 v_G ，其对应的类型图顶点为 $type(v_G)$ ；对于图 G 的边 e_G ，其对应的类型图边为 $type(e_G)$ 。

类型图和模型图的不同之处在于：类型图用于表达图的类型信息，两个顶点之间的边表示源顶点和目标顶点之间允许通过该种类型的边连接。此外，边的两端可以定义“多重性”，约束连接的源顶点及目标顶点的数量。

3.3.3 GOPPRR-E 元模型到类型图的映射



a) GOPPRR-E 主要元素的抽象语法 b) GOPPRR-E 元模型对应的类型图表示

图 3.2 GOPPRR-E 元模型到类型图的映射规范示意图

元模型定义了建模语言的抽象语法，对模型元素组成、具有的属性以及元素的静态语义进行约束。在图论中，约束概念通常通过类型图表达，因此，可以通过元模型映射为（模型对应的）类型图的方式保留建模语言的语义。GOPPRR-E 元模型到类型图映射主要思路如下：

- (1) 图元模型 (**Graph**) 映射为一个空的类型图 (即此时类型图中没有顶点和边)；
- (2) 对象元模型 (**Object**)、角色元模型 (**Role**)、点元模型 (**Point**) 分别映射为不同“映射类型”的顶点；
- (3) 关系元模型 (**Relationship**) 映射为边；
- (4) 属性元模型映射为顶点或边的属性。

上述映射过程中，对象、角色、点三类元模型都抽象为类型图的顶点，为了区分这三类不同的元模型，对应的顶点具有各自的“映射类型”，通过添加前缀“obj_”，“role_”和“point_”的方式标识。在图可视化工具中，不同标识的顶点可通过颜色和形状区分，“灰色圆角矩形”顶点表示对象元模型，“蓝色圆形”顶点表示点元模型，“橙色椭圆”顶点表示角色元模型。

在 M3-M0 三层元建模框架的中，元模型基于元建模语言构建，是 GOPPRR-E 元模型的实例，在领域元模型映射为类型图过程中，应遵守相应的映射规范以保留元建模语言语义。下文对元模型映射规范和元模型映射的实现进行详细介绍。

3.3.3.1 GOPPRR-E 元模型的映射规范

GOPPRR-E 元建模框架以图、对象、点、关系、角色、属性为基本建模元素定义建模语言，构建领域元模型，此外，元建模语言规定了抽象语法。GOPPRR-E 元建模语言的抽象语法定义了构建元模型过程中的约束，例如，点包含于对象，不允许单独存在于图；对象和关系通过绑定的角色间接连接，不允许直接连接。图 3.2 中 a 图表示 GOPPRR-E 元元模型主要元素抽象语法的描述，参考了 Kern^[44]文献中 GOPPRR 元建模语言的描述方式。因此，在元模型映射过程中，需要遵循一定的映射规范，以保证映射之后的元模型仍然保留元建模语言的约束语义。在研究元建模语言抽象语法的基础上，定义了如表 3.1 所示的映射规范其中，在映射过程中引入了特殊边“Ref”以描述对象、点、角色之间的绑定和包含关系。

表 3.1 GOPPRR-E 元模型映射规范

GOPPRR-E 元建模语言抽象语法	元模型语义映射规范
1 图元模型	空的类型图
2 对象元模型	“对象类型”（灰色）顶点，标识为 “Object[{对象类型名}]”
3 点元模型	“点类型”（蓝色）顶点，标识为 “Point[{点类型名}]”
4 角色元模型	“角色类型”（橙色）顶点，标识为 “Relationship[{角色类型名}]”
5 关系元模型	边（非 Ref 边），标识为“Relationship[{关系类型名}]”
6 属性元模型	图、顶点或边上定义的属性，类型图中属性只包含属性名和类型，不包含属性值
7 图元模型上可定义多个对象元模型	灰色顶点的多重性为“无限制”
对象元模型“包含”点元模型，一个对象	蓝色顶点通过“Ref”边与灰色顶点连接，
8 元模型可以包含多个点元模型，但每个点	“Ref”边起始端、终止端多重性分别为 “无限制”和“0..1”
元模型只能含于一个对象元模型	
9 关系不能直接与对象（或点）连接，通过	边（非 Ref 边）两端只能为橙色顶点，多

GOPRR-E 元建模语言抽象语法	元模型语义映射规范
角色间接连接，关系两端分别能且只能连接一个角色	多重性为“0..1”
角色可以与对象（或点）直接绑定，对象（或点）上可以绑定多个角色，但同一个角色只能绑定于一个对象（或点）	橙色顶点通过“Ref 边”与蓝色顶点或灰色顶点连接，“Ref”边起始段和终止端的多重性分别为“无限制”和“0..1”

3.3.3.2 领域元模型映射

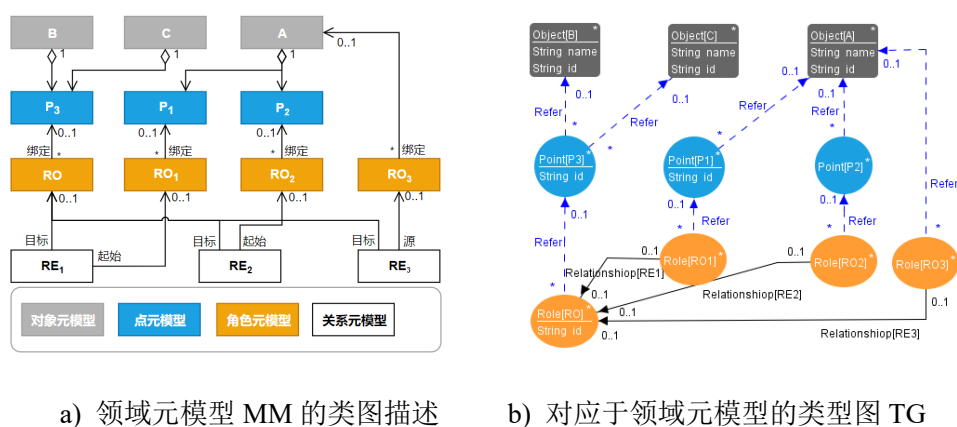


图 3.3 领域元模型到类型图映射实例

依据上述映射规范，针对具体领域元模型，按照如下流程映射为类型图，以图 3.3 为例进行解释：

(1) **图元模型的映射：**图元模型映射为一个空的类型图，图 3.3 图元模型 MM 映射为类型图 TG；

(2) **对象元模型及点元模型的映射：**将不同类型的对象映射为具有对应类型的灰色顶点；对象元模型上的点元模型映射为具有对应类型的蓝色节点。对象元模型和点元模型之间的“包含”关系映射为 Ref 边，该 Ref 边映射遵循表 3.1 第 2、3、8 项。例如图 3.3 中 A、B、C 对象分别映射为 Object[A]、Object[B]和 Object[C]三个顶点，P₁、P₂、p₃ 点元模型分别映射为 Point[P₁]、Point[P₂]和 Point[P₃]三个顶点，对象 B 和点 P₃ 之间的包含关系映射为从 Point[P₃]指向 Object[B]的 Ref 边，Ref 边的多重性限制对象可以包含的点的个数。

(3) **关系元模型及角色元模型的映射：**关系元模型映射为对应类型的边，由于在

类型图中边不能单独存在，因此需要先映射关系元模型所连接的角色元模型。将关系始末端连接的角色元模型分别映射为对应类型的橙色顶点，接着将关系映射为橙色顶点之间的边，映射原则遵循表 3.1 中的第 4、5、9 项。图 3.3 中，领域元模型 MM 中 RO、RO₁、RO₂、RO₃ 角色分别映射为类型图 TG 中的 Role[RO]、Role[RO₁]、Role[RO₂]、Role[RO₃]，关系 RE₁、RE₂、RE₃ 分别映射为 Relationship[RE₁]、Relationship[RE₂]、Relationship[RE₃]边，从顶点 Role[RO]到顶点 Role[RO₁]的有向边 Relationship[RE₁]表达角色的方向（初始角色和目标角色）语义，Relationship[RE₁]边的多重性为 0..1。

（4）绑定关系的映射：对象（或对象上的点）经过角色，和关系链式间接连接，这种关系称为对象和关系的“绑定”。为了描述绑定关系，基于映射规范，通过 Ref 边，将边两端的橙色顶点分别与灰色顶点（或蓝色顶点）连接，实现绑定的映射。映射原则遵循表 3.1 中第 10 项。图 3.3 领域元模型 MM 中对象 B 通过点 P₃ 和角色 RO 和 RE₁ 间接绑定，在步骤 3 映射的基础上，将点 P₃ 和角色 RO 的绑定映射为 TG 中顶点 Point[P₃]和顶点 Role[RO]之间的 Ref 边，并通过多重性描述绑定的数量约束。

（5）属性的映射：GOPRR-E 元建模语言中，除属性外，其他五种元素都可以拥有属性，本文基于属性图，图、顶点及边都可以定义属性。因此，元模型的属性直接映射为图模型元素的属性，图 3.3 中属性的映射没有可视化。

3.3.4 GOPRR-E 模型到图的映射

3.3.4.1 GOPRR-E 模型实例映射规范

在上述元模型映射为类型图 TG 的基础上，将模型实例映射为图 G^[70]。元模型规定了领域建模语言的抽象语法，模型实例中的元素是相应元模型元素的实例，因此，模型映射之后产生的图 G 也需要符合映射的类型图，如图 3.4 所示。为了保证映射之后的图符合对应的类型图，需要遵循一定的映射规范。

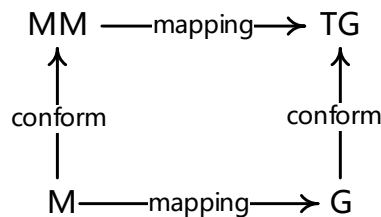


图 3.4 元模型映射和模型映射的关系示意图

GOPPRR-E 模型映射为图的映射规范如下表所示：

表 3.2 GOPPRR-E 模型映射规范

GOPPRR-E 模型元素	模型语义映射规范
1 图模型	空图
2 对象模型	对象类型顶点，其类型符合对应类型图的顶点
3 点模型	点类型的顶点，必定有一条 Ref 边与某个对象类型顶点相连，不能单独存在于图中
4 角色模型	角色类型顶点，该顶点必定的是某条边的起始端点或终止端点，且必定通过 Ref 边与其他（对象类型或点类型）顶点连接。
5 关系模型	边，且两端必须是角色类型顶点
6 属性实例	顶点或边的属性

3.3.4.2 GOPPRR-E 模型实例映射流程

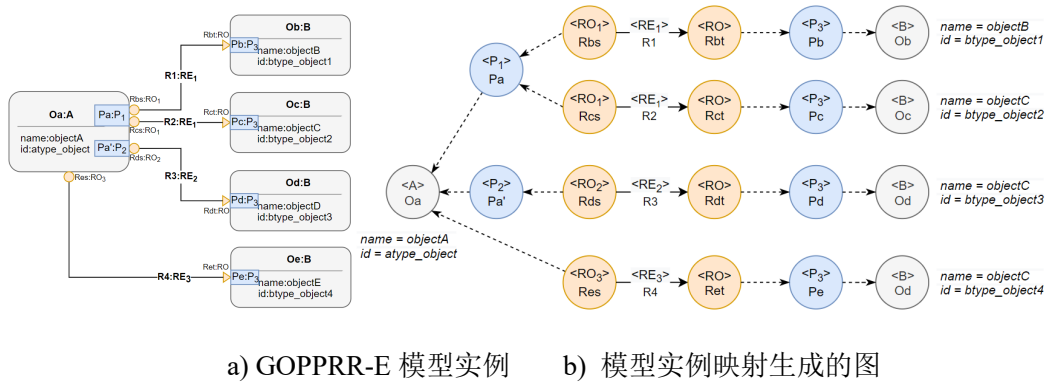


图 3.5 GOPPRR-E 模型到图的映射实例

在 GOPPRR-E 模型映射规范的基础上，针对某一个具体的元模型 MM 的模型实例 M，将 M（及模型实例元素）映射为对应的图 G，映射流程如下：

- (1) GOPPRR-E 模型本身实例映射为一个空图；
- (2) 模型的对象实例映射为图的顶点 V，其类型符合相应类型图中对象类型的顶点，即 $\text{type}(V) \in V_{\text{TG}}^{\text{Object}}$ ；
- (3) 模型的点实例映射为图的顶点 V_p ，和对象类型顶点类似， V_p 顶点的类型对应于类型图中点的类型。为了描述对象实例可以包含若干个点实例语义， V_p 通过参

考边链接到对应的类型顶点 V ，参考边是类型图中 Ref 边的实例；

(4) 模型的角色实例映射为图的顶点 V_r ， V_r 节点和 V_p 节点类似。GOPRR-E 模型中对象的连接关系是通过角色实例和对象实例直接（或通过点实例间接）绑定描述的，因此， V_r 可以通过 Ref 边参考其他对象类型点类型顶点，以描述绑定关系。

(5) 模型中的关系实例映射为图的边。GOPRR-E 模型中对象实例和关系实例是通过引用的角色间接连接的，关系实例两端与角色实例进行绑定，通过引用不同类型的角色描述不同连接规则。因此，关系映射为边，边的源顶点和目标顶点分别表示引用的角色实例；

(6) 模型元素的属性直接映射为图、顶点及边的属性。基于上述的映射规则，图 3.5 中 GOPRR-E 模型和对应的映射图是等价的。

3.4 基于图转换的 GOPRR-E 模型转换

3.4.1 图转换

在图转换研究领域，图转换问题以图文法形式化描述，在起始图的基础上应用图转换规则，利用图重写技术对图进行操作，最终得到目标图。图转换规则是图转换过程的关键，其中一个重要的概念是图态射，首先给出其形式化定义。

1) 图态射

给定两个图 G 和 H ，形式化描述为 $G = \langle V_G, E_G, source_G, target_G, label_G \rangle$ ， $H = \langle V_H, E_H, source_H, target_H, label_H \rangle$ ，一对映射函数 (f_V, f_E) ， $f_V: V_G \rightarrow V_H$ ， $f_E: E_G \rightarrow E_H$ 构成了图态射 $f: G \rightarrow H$ ^[71]，图态射具有如下属性：

1. $\forall e \in E_G: f_V \circ source_G(e) = source_H \circ f_E(e)$
2. $\forall e \in E_G: f_V \circ target_G(e) = target_H \circ f_E(e)$

2) 图匹配

从起始图 S 中找到与规则的左模式 LHS 结构相符的一个实例称为匹配，匹配是一个图态射，是从 L 图的顶点和边到 S 图顶点和边的映射，记作 $m: L \rightarrow S$ 。

3) 图转换规则

图转换规则 R 是由左图 (LHS) 和右图 (RHS) 组成的二元组 $R = \langle LHS, RHS \rangle$ ， R 等价于术语重写规则^[72]。重写是指，如果左图在源模型上找到匹配，则将匹配到的

元素按照右图代替。更具体地，图转换规则的形式化定义如下：

$$R: LHS \xrightarrow{f} RHS \quad (3.12)$$

其中 $LHS \in G \wedge RHS \in G$ ，左图（也称为左模式）和右图（也称为右模式）都属于标记图， f 称为图映射指明 LHS 和 RHS 图元素之间的对应关系，例如 LHS 中的顶点 v ，在重写过程中没有被删除或修改，存在顶点 v' ，满足 $v' \in RHS$ 而且 v 和 v' 具有相同的编号，那么 $(v, v') \in f$ 。

4) 图规则应用

应用图转换规则 $r \in R$ 对起始图 S 进行一次转换，形式化表示为：

$$t: S \xRightarrow{r, m} H \quad (3.13)$$

如图 3.6 所示，规则的应用过程包含如下流程：

1. 在起始图 S 中找到一个 LHS 的匹配实例 $m(LHS)$ （图模式匹配）；
2. 在起始图 S 上删除 $m(LHS - RHS)$ ，产生上下文图 D ；
3. 根据匹配到的元素，根据映射规则生成规则右边图 R ，胶合 R 和 D ，添加 $m^*(RHS - LHS)$ ，获得生成图 H 。

其中 p^* 建立起始图 S 与生成图 H 的追溯关系，保证转换前后图元素之间可追溯。图映射 m 是一个匹配 $m: LHS \rightarrow G$ ， m^* 是 m 的同匹配， m^* 本质上是一个图同态 $m^*: RHS \rightarrow H$ ， m 和 m^* 一起构成推出。根据在重写过程中对悬空边的处理方式，可以将推出分为一次推出^[66]（SPO, Single pushout）和二次推出（SPO, Double pushout）^[73]。两者之间的主要区别在于二次推出采用保守方案处理悬空边，通过两次图胶合才能得出结果，在胶合过程中需要考虑胶合条件，以保证不会出现悬空边，该策略可实现可逆转换。SPO 处理悬空边采取激进方案，无需检查胶合条件，直接删除悬空边，这种方法容易引起其他转换副作用，需要显式地控制。相比于 DPO，SPO 支持更多转换场景，可以处理 DPO 无法处理的转换问题。因此本文采用基于 SPO 方法的图转换。

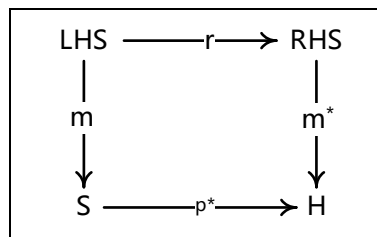


图 3.6 基于 SPO 的图规则应用示意图

3.4.2 GOPRR-E 模型转换到图转换的映射

在模型映射为图的基础上，模型转换问题可通过图转换形式化描述。与图转换概念相对应，如图 3.7 所示，模型转换也包含若干个模型转换规则 Rule，转换规则为 LHS（左模式）、RHS（右模式）以及 NAC^[74]（否定应用条件）组成的三元组 $\text{Rule} = \langle \text{LHS}, \text{RHS}, \text{NAC} \rangle$ ，LHS 描述源模型中能够被匹配的元素拓扑结构；RHS 描述如何根据匹配到的元素生成新的元素；否定应用条件和左模式对应，描述源模型中不允许被匹配到的元素拓扑结构和约束条件。LHS 和 RHS 之间映射关系可定义为函数 f ，函数 $f: \text{LHS} \rightarrow \text{RHS}$ 描述了从模式图到匹配图的元素映射关系，从函数角度看，LHS 为定义域，RHS 为值域。

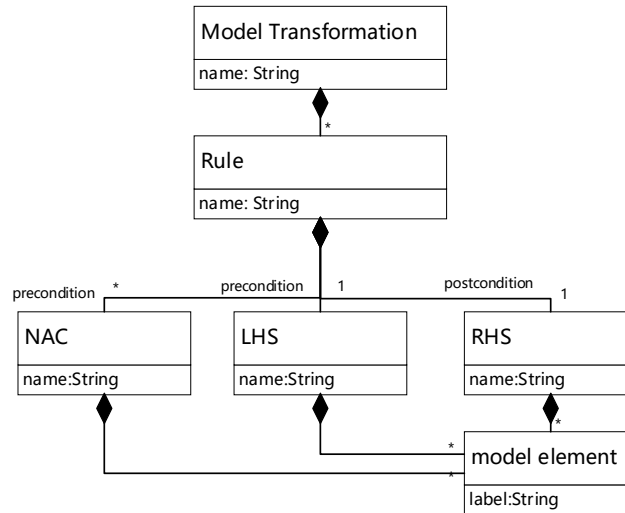


图 3.7 模型转换规则的构成示意图

本文基于图转换实现模型转换，模型转换原理如图 3.8 所示。其中 M_a 和 M_b 图分别形式化描述了源模型和目标模型， MM_a 和 MM_b 类型图形式化描述了对应的源元模型和目标元模型，通过 **type** 建立类型图和图之间的遵循关系，图转换规则 R 形式化模型转换规则。模型转换规则中 LHS、RHS 和 NAC 定义的模式本质上也是一个模型实例，模式的定义需要参考对应的元模型，例如 LHS 参考 MM_a 以描述源模型中何种模型结构应该被匹配，RHS 参考 MM_b 以描述如何生成目标模型元素，由于 NAC 是相对于 LHS 的应用条件约束，因此 NAC 也参考 MM_a 。

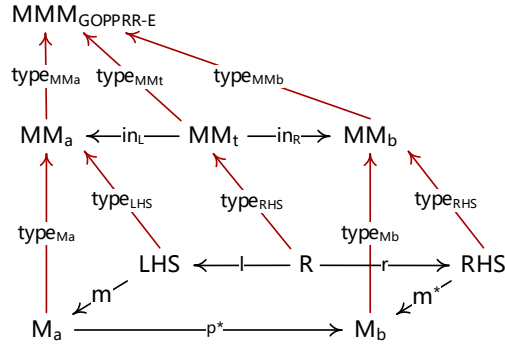


图 3.8 基于图转换规则的模型转换原理图

在 3.2 节已经介绍，为了实现跨域模型转换引入了转换元模型 MM_t 。为了尽可能重用领域概念，避免从头创建一种新的元模型，转换模型的元模型 MM_t 重用两种特定域建模语言的元模型 MM_a 和 MM_b 。跨域模型转换的另外一个问题是如何维护跨领域模型元素之间的追溯关系，因此转换元模型 MM_t 不能通过简单地复制 MM_a 和 MM_b 将两者融合^[75]，需要在两个元模型基础上扩展，引入额外的元模型元素以描述追溯链接。

综上所述，为了解决跨域模型转换问题，本文通过额外构建一个元模型 MM_t 实现对转换过程的建模。如图 3.9 所示，以简单状态机模型到分层并发状态机模型的转换为例，说明其转换元模型 MM_t 的构建流程：

(1) 拷贝领域建模语言的元模型：模型转换规则在元模型层建立跨域模型元素之间的映射关系，因此需要基于源元模型 MM_a 和目标元模型 MM_b 构建一个额外的域 MM_t ，通过拷贝将 MM_a 和 MM_b 中的领域概念融合到同一个领域，形成一个临时的额外的元模型 MM_t 。临时域的原因在于 MM_t 仅仅为了方便描述跨域元素映射而存在，并不能表示某个具体的学科领域，在模型转换结束后便没有意义。例如图中 MM_a 表示有限状态机元模型， MM_b 表示分层并发状态机元模型，将两个元模型拷贝形成元模型 MM_t （分别用蓝色线框和黄色线框示意）。

(2) 元模型标记：模型转换的追溯链接同样通过转换模型描述，为了建立追溯链接，需要区分 MM_t 中各个元模型元素的领域语义，通过标记元模型的方法实现，在上步融合过程中，来源于 MM_a 的元模型元素标记为“LHS_”，来源于 MM_b 的元模型元素标记为“RHS_”，追溯链接只能在拥有不同标记元模型的实例间建立，如图 3.9 中 MM_t 所示，所有元素分别通过对应的前缀标注。

(3) **转换元模型扩展**: 转换模型 M_t 包含了源模型元素和目标模型元素交叉链接信息, 通过上述拷贝获得的 MM_t 并不能定义出链接语义。因此需要在上述两步的基础上扩展 MM_t , 在 MM_t 中添加“追溯”的概念, 以描述追溯关系, “追溯”是一种只能定义在源元模型元素和目标元模型元素之间的关系, 如图中定义在 $LHS_Finite\ State$ 和 RHS_State 以及 $LHS_Finite\ Transition$ 和 $RHS_Transition$ 之间的 $trace$ 类型关系, 其实例可以描述转换前后 $State$ 和 $Transition$ ^[76]实例的追溯关系。

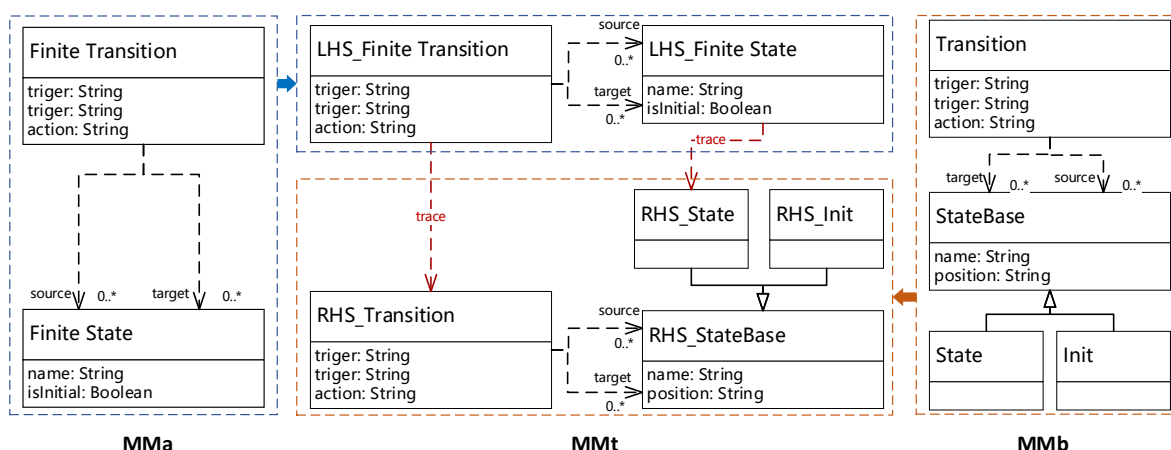


图 3.9 转换元模型 MM_t 构建示意图

通过上述步骤构建一个包含所有域和追溯链接的转换元模型, 将不同的域联系在一起, 实现异构模型转换的描述。基于 MOF 元建模框架, 在元元模型层通过 GOPRR-E 统一描述领域元模型和领域概念, 结合元建模方法, 将跨领域的异构数据通过统一的形式表达, 使得跨域转换成为可能, 也是利用图转换实现模型转换的前提。

3.5 架构模型转换流程

在上述模型、元模型映射、模型转换规则映射的基础上, 以图重写为理论依据实现模型转换。架构模型转换流程如图 3.10 所示。

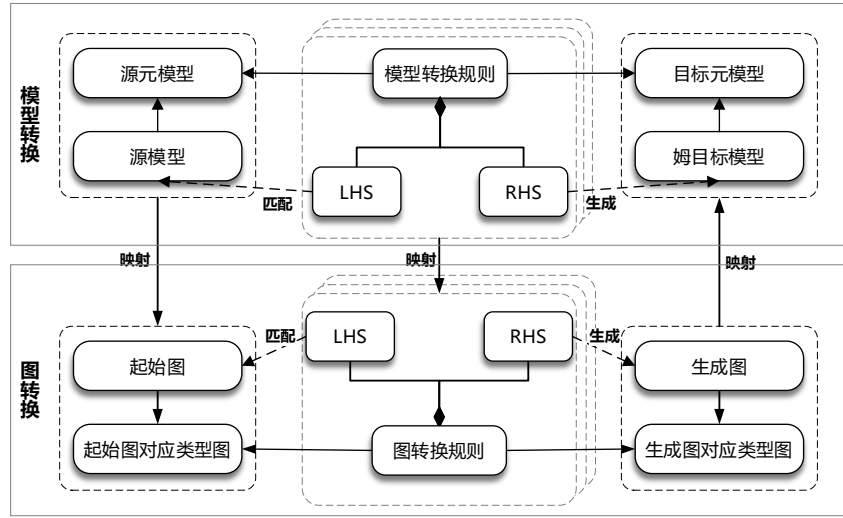


图 3.10 模型转换流程图

(1) 转换元模型构建：分析源元模型 MMa 和目标元模型 MMb ，判断是否为跨域模型转换（即分析是否满足 $MMa = MMb$ ）。对于非跨域模型转换，选择源元模型或目标元模型其中一方作为转换元模型即可。否则基于上节介绍的转换元模型构建方法，融合源元模型和目标元模型，构建转换元模型 MMt ，并依据节中所述元模型映射方法，将转换元模型映射为类型图；

(2) 起始图映射：将源模型依据 3.3 节中所述方法映射为对应的起始图；

(3) 转换规则集映射：构建图转换规则，并分别将模型转换规则的 LHS 图和 RHS 图按照映射规范映射为图转换规则的 LHS 和 RHS，同时将 NAC 映射为图。映射规则集合中的每条规则，最终形成图转换规则集合；

(4) 构建图文法：完成以上映射后，将映射后的转换元模型，起始图以及图规则构建为图文法，模型转换规则中 NAC 映射后的图作为约束条件添加到图文法中；

(5) 图重写操作：基于图重写对图文法进行推导，主要基于 SPO 方法，对图转换规则集中的规则进行推导，经过推导得到生成图；

(6) 生成图到目标模型的映射：将生成图按照模型映射方法，反向映射得到目标模型。

依据上述流程，以如图 3.12 所示模型为案例进行跨域模型转换，其中源元模型和目标元模型如图 3.11 所示。源元模型中包含 A、B、C 三种对象，其中 A 类型对象可包含 P 类型点；包含 RE1 和 RE2 两种关系，并通过相应的角色和对象绑定。目标

元模型包含 a、b 两种对象，re 一种关系，通过角色和对象绑定。

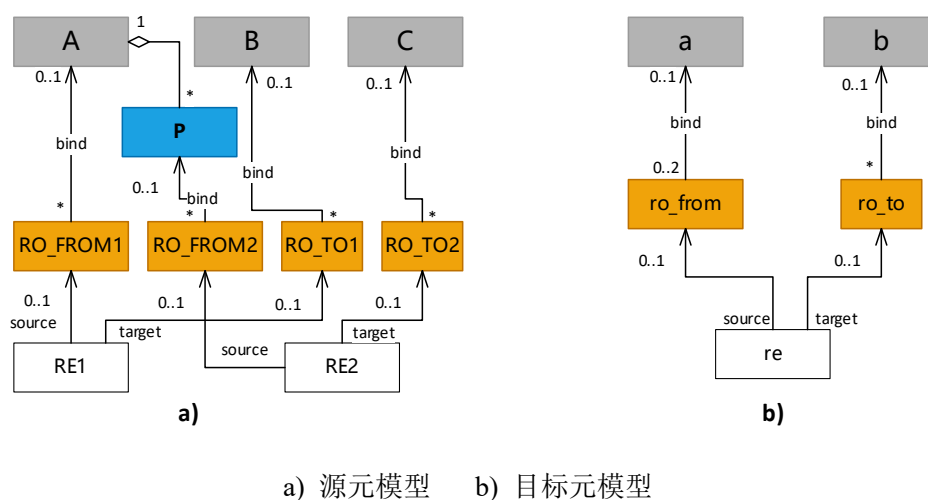


图 3.11 转换源元模型和目标元模型的类图表示

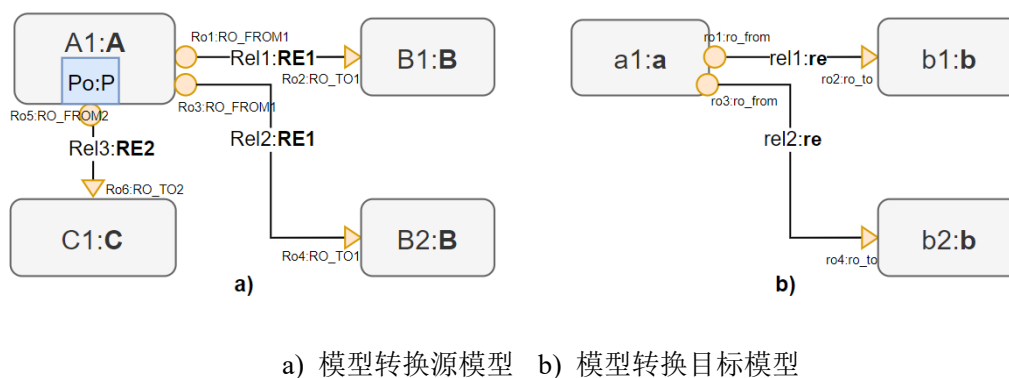


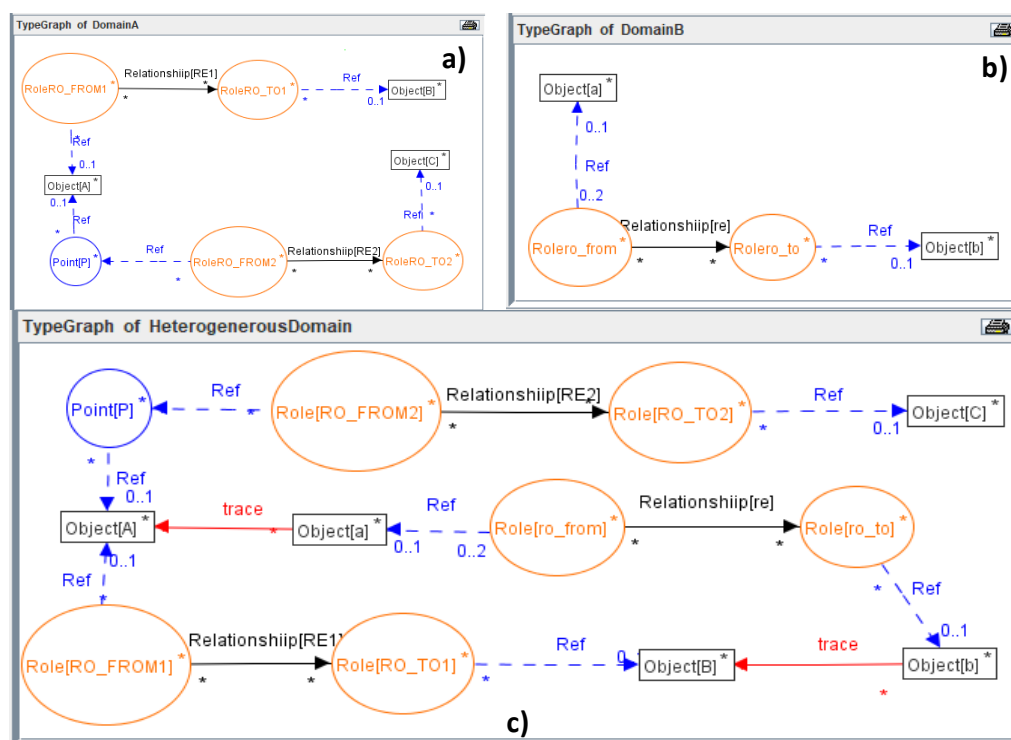
图 3.12 转换源模型和生成的目标模型

模型转换规则如图 3.13 所示，LHS 用于匹配源模型中这样的模型结构：A 类型对象通过 RE 类型关系连接到 B 类型对象，且绑定的角色类型也需要匹配。RHS 表示，对于 LHS 匹配到的模型实例，生成的模型结构为：a 类型对象通过 re 类型关系连接到 b 类型对象，且生成相应的角色完成绑定。



图 3.13 模型转换规则

按照本章提出基于图转换的模型转换方法，首先将源元模型和目标元模型分别映射为类型图 MMa（图 3.14-a），类型图 MMb（图 3.14-b），并构建转换元模型 MMt（图 3.14-c），转换元模型中 trace 关系用于描述“追溯”关系。



a) 源元模型映射类型图 b) 目标元模型映射类型图 c) 转换元模型类型图

图 3.14 源元模型、目标元模型和转换元模型映射的类型图

在元模型映射为类型图基础上，分别将源模型映射为起始图（如图 3.16-a 所示），模型转换规则映射为图转换规则（如图 3.15 所示）。起始图和图转换规则形成图文法，在图文法上应用图转化规则，生成如图 3.16-b 所示的生成图。经过验证符合对应类型图的生成图，将映射为目标模型，最终经过模型转换得到的目标模型如图 3.12-b 所示。

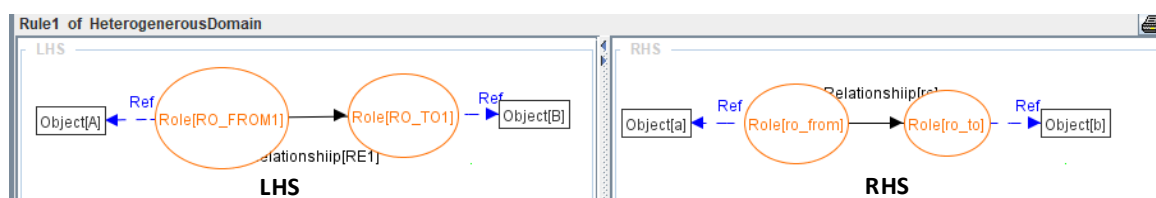
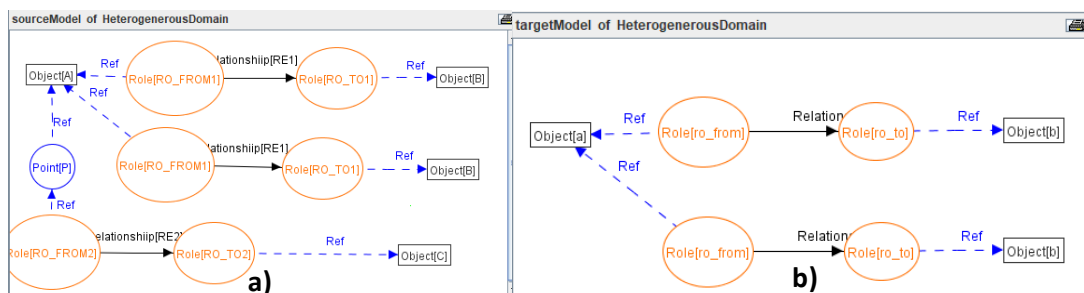


图 3.15 模型转换规则映射的图规则



a) 源模型映射的起始图 b) 图推导产生的生成图

图 3.16 起始图和生成图

3.6 本章小结

本章介绍了基于图论中的图转换实现跨域模型转换的原理，本章内容是模型转换正确性检验以及模型转换统一描述语言设计的基础。首先通过分析复杂系统架构模型转换特点，提出跨域模型转换的概念；接着，针对 GOPPRR-E 元建模框架构建的系统架构模型，通过图论形式化架构模型、元模型、模型转换等概念，提出架构模型到图、元模型到类型图、模型转换规则到图转换规则等一系列映射方法，将模型转换问题映射为图转换问题，并基于图转换的一次推出方法实现模型转换；最后，提出模型转换流程，实现系统架构模型的跨域模型转换。

第4章 架构模型转换正确性的形式化检验方法

4.1 引言

模型转换技术提供了自动化（或半自动化）生成复杂系统架构模型的方法，领域工程师根据业务场景制定模型转换规则，模型转换引擎辅助自动生成目标模型。但基于图转换的模型转换方法并不能保证生成模型的语法符合工程师期望，在未经验证的模型转换生成的目标模型基础上继续完善模型，将给系统架构模型设计引入误差，由此经过多次模型转换造成的模型误差将导致最终结果与需求严重不符。因此在模型转换过程中，必须保证自动生成的模型不会引入新的缺陷，精确地证明转换生成的模型语法正确。

4.2 模型转换正确性的定义

模型转换验证是模型转化中的关键步骤。模型转换作为模型驱动开发的基础，贯穿系统架构模型开发全生命周期，在模型转换中出现的误差或错误将传播到下一级模型转换中，如此不断传播引起整个系统模型产生累积的巨大误差和错误^[77]。目前针对模型转换验证的研究主要集中于以下几方面^[78]：

- （1）目标模型语法正确性：转换生成的目标模型符合目标建模语言的语法规则，即保证目标模型是目标元模型的实例。
- （2）模型转换可终止性：转换过程中源模型应用模型转换规则，在有限步转换后可得到目标模型，而不会无限进行下去。
- （3）转换结果唯一性：转换结果唯一性描述模型转换的非确定性问题，所谓非确定性是指模型转换过程中，由于模型转换匹配机制是随机的，同样的规则按照不同的匹配顺序应用到源模型，得到的目标模型不同，则表明模型转换是非确定的。模型转换结果唯一性要求源模型无论经过怎样的模型转换，都得到唯一的结果。

对于模型转换验证问题也存在多种方法，最好的办法是通过数学形式化方式证明模型转换规则^[79]，形式化方法指在数学、逻辑和计算机科学中，通过形式语言的方式将过程描述出来，形式语言是用精确的数学或机器可处理的公式定义的语言。但是形式化验证方法对形式化、推理等数学基础要求过高^[80]，而且证明过程繁琐，领域工程

人员难以应用该方法解决业务问题。因此，通常使用验证工具辅助实现自动验证，例如通过模型检查^[81]、静态分析^[82]、定理证明^[83]等方式。从解决工程问题的角度来看，无需从推理的角度证明模型转换本身的正确性，只需要证明经过转换生成的目标模型的正确性便能满足工程需求。因此，本文重用现有的代数图转换正确性证明已有成果（例如已有图转换相关理论研究能够保证转换可终止性和转换结果唯一性），通过图转换验证方法证明系统架构模型转换的正确性，避免证明模型转换方法本身的正确性。此外，本文关注的是某个具体的源模型转换生成的目标模型的正确性，而不是整个模型转换系统的正确性，模型验证的目的在于发现模型转换目标模型中的错误，而不是证明模型转换本身的正确性。

可推出的定义：模型 M 基于领域建模语言 L 构建，满足建模语言 L 的语法和语义约束，称为语言 L 可推出模型 M ，记作 $L \vdash M$ 。

模型转换正确性定义：设 $t: M_A \xRightarrow{T} M_B$ 是一个模型转换， L_B 是目标模型对应的领域建模语言，对于某个输入模型 M_A ，经过模型转换 t 得到目标模型 M_B ，当且仅当目标模型 M_B 符合目标建模语言 L_B ，模型转换 t 是正确的（如图 4.1 所示），形式化描述为：

$$M_A \xRightarrow{T} M_B \Rightarrow L_B \vdash M_B \quad (4.1)$$

从公式中可以看出，验证模型转换正确性的关键在于保证模型 M_B 满足可推出条件，需要满足两个方面的推出条件，一是保证模型 M_B 符合建模语言 L_B 的语法，而是 M_B 满足 L_B 的语义约束。对于纵向模型转换，模型转换可实现目标模型的半自动生成，本文主要针对语法正确性进行验证。

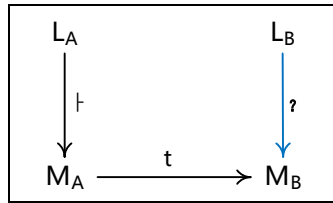


图 4.1 模型转换验证示意图

4.3 基于类型图一致性验证的模型转换正确性验证

模型转换的语法正确性是指，模型转换生成的目标模型符合对应建模语言语法。

如前面章节所述，建模语言的抽象语法是通过元模型定义的，因此语法正确性要证明目标模型（及其模型元素）是对应元模型（及其元模型元素）的实例。目标模型语法正确性是模型转换验证的最基本要求，即首先要保证生成的模型是目标语言的一个语法结构良好的实例。

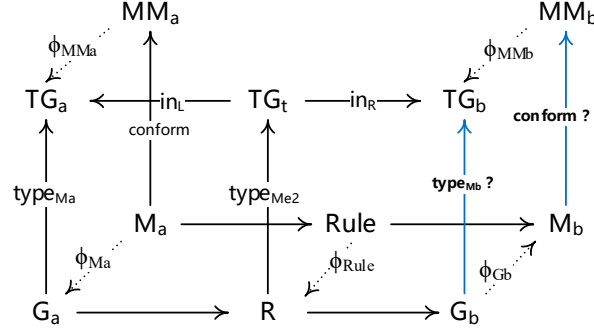


图 4.2 模型转换语法正确性验证原理示意图

在基于图转换的模型转换方法中，领域元模型的语义是通过映射为类型图表示，如图 4.2 所示，按照第 3 章中 GOPRR-E 元模型的映射方法，分别将领域元模型 MM_a 和 MM_b 映射为对应的类型图 TG_a 和 TG_b ，源模型 M_a 映射为图 G_a ，模型转换规则 $Rule$ 映射为图转换规则 R ，映射关系记作 $\phi(x)$ ，经过上述映射得到的图文法，保留了模型转换的全部语义，模型转换问题描述为图转换问题。基于图重写理论对映射后图文法进行推导，图转换完成后生成目标图 G_b ，按照 ϕ_{G_b} 映射得到 M_b 。当且仅当 $type(G_b) = TG_b$ 成立，使得 $M_b \xrightarrow{conform} MM_b$ 成立，证明模型得到的模型是语法正确的，即生成模型符合对应元模型。综上，本文的验证思路是将一个模型转换语法正确性问题转换为类型图一致性检查问题，通过等效的图转换验证代替模型转换语法的正确性验证。

4.3.1 类型图一致性条件

对于图 G 和类型图 TG ，当且仅当一组一致性条件 $conform(G, TG)$ 成立时，认为图 G 满足类型图 TG ：

$$conform(G, TG) = \bigwedge_{i=1}^n c_i \quad (4.2)$$

c_i 是一个逻辑表达式，表示形式化的一致性条件，具体来说，类型图一致性条件包括图顶点类型一致性条件、边一致性条件、顶点多重性一致性条件、边多重性一致性条件。定义函数 $\tau_v: V_G \rightarrow V_{TG}$ 和 $\tau_e: E_G \rightarrow E_{TG}$ 分别将图 G 中的顶点 v 和边 e 对应到类

型图 TG。

1) 图顶点类型一致性条件 c_1

$$c_1 \triangleq \forall v_G \in V_G, \exists v_{TG} \in V_{TG}: v_{TG} = \tau_v(v_G) \quad (4.3)$$

c_1 表示对于图 G 中任意节点，在类型图 TG 中存在对应的顶点以定义它的类型。在图 4.3 中展示了图顶点类型一致性条件示例，类型图 TG 定义了 A、B 两种类型的顶点，由于图 G_1 中包含 TG 中未定义的 C 类型的顶点，因此 G_1 未满足 TG 定义的顶点类型一致性条件， G_2 中所有的顶点类型都在 TG 定义，满足顶点类型一致性条件。

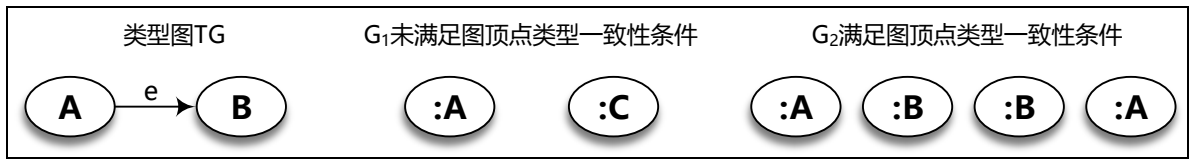


图 4.3 图顶点类型一致性条件示例

2) 边一致性条件 c_2

$$c_2 \triangleq \forall e_G \in E_G, \exists e_{TG} \in E_{TG}:$$

$$\tau(e_G) = e_{TG} \wedge source(e_{TG}) \in \tau(source(e_G)) \wedge target(e_{TG}) \in \tau(target(e_G))$$

c_2 表示对于图 G 中任意边 e_G ，在类型图中存在对应的边 e_{TG} 以定义边的类型，并且边 e_G 的源顶点 $source(e_G)$ 、目标顶点 $target(e_G)$ 的类型也分别和 e_{TG} 源顶点 $source(e_{TG})$ 和目标顶点 $target(e_{TG})$ 定义的类型相同。如图 4.4 所示，类型图 TG 定义了四类顶点以及顶点之间的边，由于 G_1 中边两端连接的顶点类型和 TG 中定义的不同，故未满足边一致性条件； G_2 中包含 TG 未定义的边类型 ec ，也不满足边一致性条件； G_3 中所有边类型都已在 TG 中定义，且所有边两端的顶点类型与 TG 中定义的类型相符，因此满足边一致性条件。

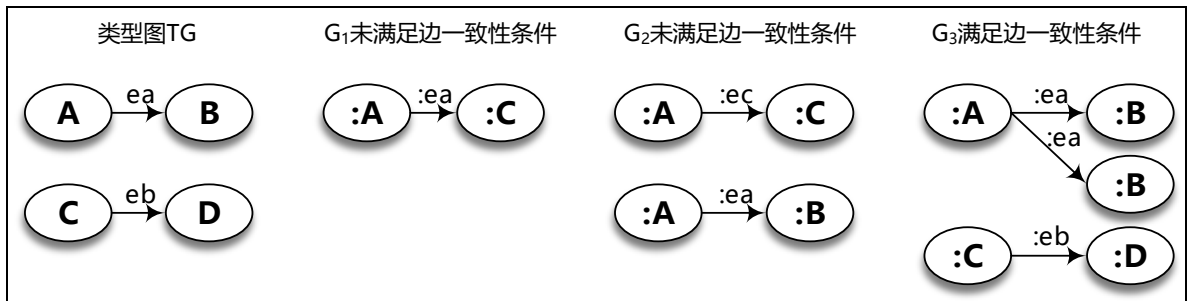


图 4.4 边一致性条件示例

3) 边的多重性一致性条件 c3、c4

边的目标多重性用于限制同一个源顶点的情况下，可以通过该边连接的目标顶点的个数；边的起始多重性用于限制同一个目标顶点的情况下，通过该边连接的源顶点的个数。

$$\begin{aligned}
 c_3 &\triangleq \forall e_{TG} \in E_{TG}, \exists v_A \in V_G, \exists e_G \in E_G, \forall V_{SG} \\
 &= \{v_B \mid v_B \in V_G \wedge v_B \neq source(e_G) \wedge target(\tau_e(e_G)) \in \tau_v(v_B)\} : \tau_e(e_G) \\
 &= e_{TG} \wedge source(e_G) = v_A \wedge |V_{SG}| \in md(e_{TG})
 \end{aligned}$$

c_3 表示对于类型图中的所有边 e_{TG} ，其在图中的边实例 e_G 都满足目标连接多重性约束条件。上式中，图顶点 v_A 通过边 e_G 正确连接到其他顶点 v_B ，这些（其他）顶点组成一个顶点子集 V_{SG} ，其中子集的元素满足目标多重性约束。如图 4.5 所示，类型图 TG 中边 e 的目标多重性定义为[2,4]，表示其实例图中对于同一个 A 类型顶点，通过 e 类型边与之连接的 B 类型顶点的个数需介于 2 和 4 之间，故图 G_1 不满足目标多重性一致性条件， G_2 满足。

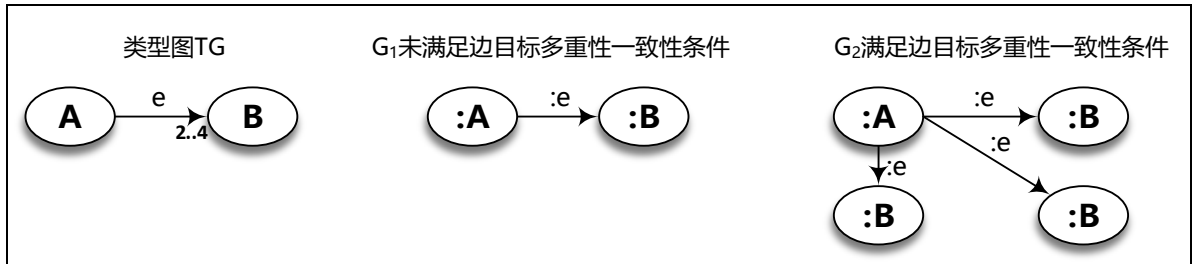


图 4.5 边的目标多重性一致性条件示例

$$\begin{aligned}
 c_4 &\triangleq \forall e_{TG} \in E_{TG}, \exists v_A \in V_G, \exists e_G \in E_G, \forall V_{SG} \\
 &= \{v_B \mid v_B \in V_G \wedge v_B \neq target(e_G) \wedge source(\tau_e(e_G)) \in \tau_v(v_B)\} : \tau_e(e_G) \\
 &= e_{TG} \wedge target(e_G) = v_A \wedge |V_{SG}| \in ms(e_{TG})
 \end{aligned}$$

c_4 表示对于类型图中的所有边 e_{TG} ，其在图中的实例边 e_G 都满足起始连接多重性约束条件。上式中，其他顶点 v_B 通过边 e_G 正确连接到图顶点 v_A ，这些 v_B 顶点组成一个顶点子集 V_{SG} ，其中子集的元素满足起始多重性约束。如图 4.6 所示，类型图 TG 中边 e 的起始多重性定义为[2,4]，表示其实例图中对于一个 B 类型顶点，通过 e 类型边与之连接的 A 类型顶点的个数需介于 2 和 4 之间，图 G_1 中边的起始多重性为 1，不满足起始多重性一致性条件。 G_2 中通过 e 类型边与 B 类型顶点连接的顶点个数为

3, 介于 2 和 4 之间, 满足边的起始多重性一致性条件。

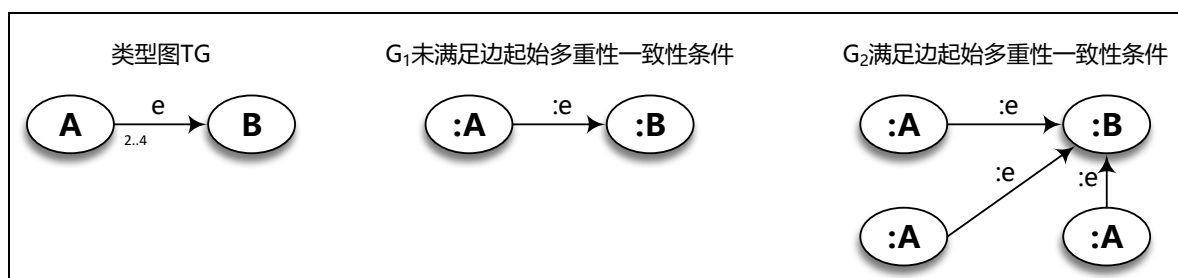


图 4.6 边的起始多重性一致性条件示例

4.4 模型转换正确性验证流程

本文的模型转换验证主要针对目标模型语法的正确性验证, 以类型图一致性验证为理论基础, 在 3.4 节中图转换框架上增加验证功能, 验证生成图的有效性 (是否符合对应的类型图)。如上述介绍, 主要从图的顶点类型一致性条件、边一致性条件以及边多重性一致性条件几个方面验证, 验证成功的生成图按照 3.3 节中生成图到目标模型的映射方法得到正确的目标模型。详细的验证流程如图 4.7 所示。

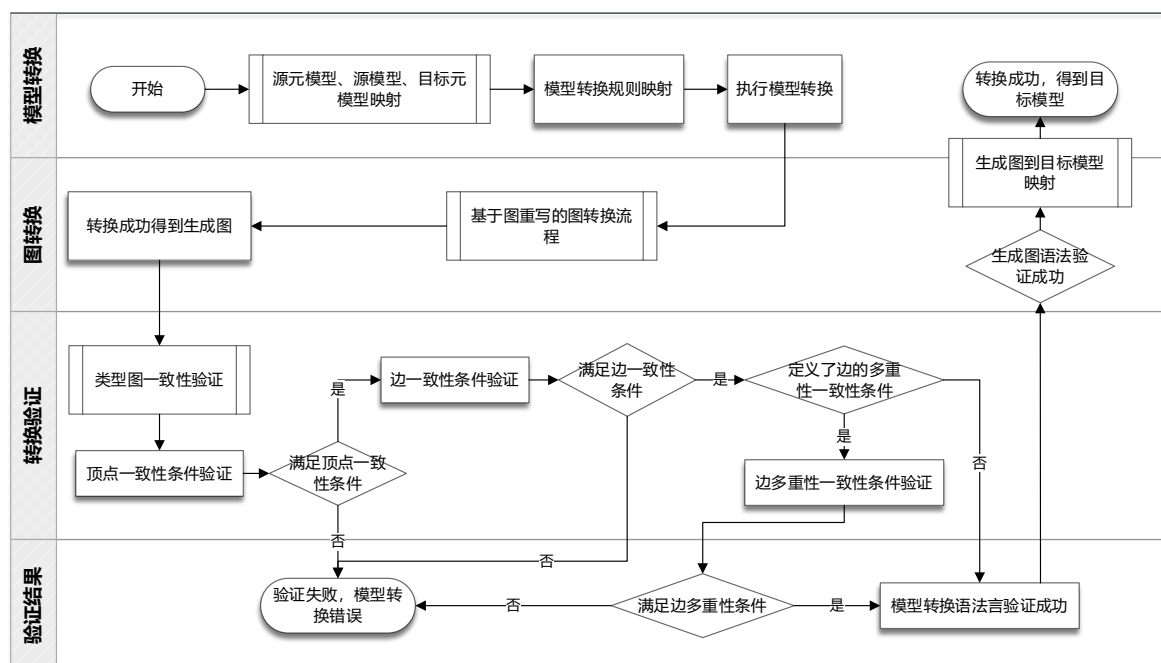


图 4.7 模型转换正确性验证流程

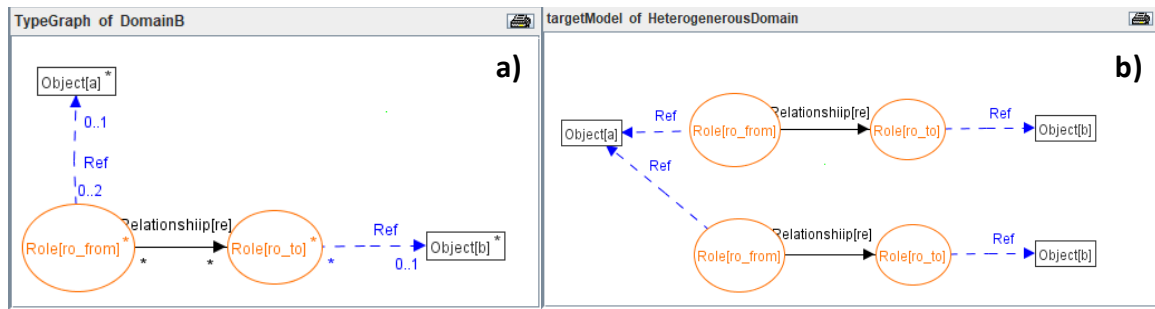
在 3.5 节架构模型转换流程的基础上增加模型检验步骤, 针对图转换执行后的生

成图进行检验，检验通过的生成图映射为目标模型，否则认为生成的模型无效，模型转换发生错误。检验的总体思路如下，根据类型图一致性验证条件，验证生成图的有效性，验证有效表示模型转换可以得到正确的目标模型，否则模型转换失败，此次转化不能得到正确的目标模型。验证过程的输入为生成图和对应的类型图，生成图由图转换生成，对应的类型图由模型转换的目标元模型映射得到。

类型图的一致性验证流程如下：首先验证图中所有顶点是否满足顶点一致性条件 c_1 ，若不满足则表示验证失败，无需继续验证；否则验证图中边的一致性条件 c_2 ，不满足条件表示验证失败，否则验证边的多重性一致性条件 c_3 、 c_4 ；由于某些情况下边未定义多重性或使用默认多重性（对连接数量不进行限制），因此在验证前需检查是否需要验证，若无需验证多重性，则验证成功，否则分别对边起始多重性和目标多重性一致性条件进行验证，只有 c_3 、 c_4 都满足的情况下才证明验证成功。

若得到类型图一致性验证成功的生成图，则证明模型转换语法正确，并通过生成图到 GOPRR-E 模型的映射得到目标模型，否则模型转换失败，并予以错误反馈。

基于提出的验证流程，对 3.5 节中的模型转换案例进行检验。该模型转换为跨域模型转换，目标元模型对应的类型图 TG_{target} 如图 4.8-a 所示，在输入模型对应的起始图上应用转换规则，得到的生成图 G_{target} 如图 4.8-b 所示。模型转换正确性的条件为生成图符合对应的类型图。按照上述的类型图一致性验证流程，分析如下：



a) 目标元模型映射的类型图 TG_{target} b) 图转换的生成图 G_{target}

图 4.8 模型转换正确性验证实例

(1) 验证是否满足顶点一致性条件。生成图 G_{target} 包含三个对象顶点，其类型分别为 $Object[a]$ 和 $Object[b]$ ，四个角色类型顶点，其类型分别为 $Role[ro_from]$ 和 $Role[ro_to]$ ，在类型图 G_{target} 中以定义对应的对象类型，因此图 G_{target} 满足顶点一致性条件。

(2) 验证是否满足边一致性条件。生成图 G_{target} 包含 *Ref* 和 *Relationship[re]* 两种类型边, *Ref* 边可以连接 *Role[ro_from]* 和 *Object[a]* 类型顶点, 也可以连接 *Object[ro_to]* 和 *Object[b]* 类型对象; *Relationship[re]* 边可以连接 *Role[ro_from]* 和 *Role[ro_to]* 类型对象。图 G_{target} 中每条边的连接与 TG_{graph} 所定义的边连接相符, 因此图 G_{target} 满足边一致性条件。

(3) 验证边的多重性一致性条件。定义在 *Role[ro_from]* 和 *Object[a]* 类型顶点之间的 *Ref* 边, 其目标多重性为[0..1], 表示同一个 *Role[ro_from]* 类型顶点最多只能连接一个 *Object[a]* 类型顶点; 其起始多重性为[0..2], 表示同一个 *Object[a]* 类型对象最多能被 2 个 *Role[ro_from]* 类型对象连接。对于定义在 *Role[ro_to]* 和 *Object[b]* 类型顶点之间的 *Ref* 边, 其目标多重性为[0..1], 表示同一个 *Role[ro_from]* 类型顶点最多只能连接一个 *Object[a]* 类型顶点; 其起始多重性未作限制。对于 *Role[ro_from]* 和 *Role[ro_to]* 类型顶点之间的 *Relationship[re]* 边, 其起始多重性和目标多重性都未做限制。基于上述分析, 可知悉图 G_{target} 的所有边满足多重性条件。

综上, 经过图转换生成的图 G_{target} 满足顶点一致性条件、边一致性条件以及多重性一致性条件, 因此 G_{target} 符合类型图 TG_{target} , 满足类型图一致性条件, 经过模型转换可得到正确的模型。

4.5 本章小结

本章提出了一种模型转换正确性验证方法, 以保证对于一个有效的源模型, 经过模型转换后生成的目标模型也是有效的。首先给出了模型转换正确性定义及其形式化描述, 本文提出的模型转换正确性验证主要针对生成模型的语法正确性; 然后基于类型图一致性理论, 将模型转换正确性验证问题转变为类型图一致性验证问题, 并提出类型图一致性条件及其形式化描述; 最后提出模型转换自动验证流程, 检测转换过程中可能生成的错误模型, 保证模型转换的正确性, 避免生成缺陷模型。模型转换正确性验证是模型转换语言设计的前提, 为模型转换语言的设计奠定基础。

第5章 模型转换统一描述语言设计

5.1 引言

基于图论中图转换的模型转换方法实现了跨域架构模型之间的相互转换，在工程应用中进行模型转换的关键步骤是通过制定模型转换规则表达源模型和目标模型之间的对应关系，而模型转换规则是基于模型转换语言描述的。模型转换语言是基于某种范式（文本式或图形式）、具有良好语法和语义结构的，用于描述如何将源模型转换为目标模型的计算机语言。本文在 KARMA 语言基础上，提出支持描述多领域架构模型转换的模型转换统一描述语言，该语言采用基于文本的声明式语言范式，以模型模式实现对模型转换规则 LHS 和 RHS 的直观描述。本章节以第 3、4 章节研究内容为理论基础，结合计算机编程语言设计方法，研究模型转换语言的具体语法和抽象语法设计，旨在提出面向 MBSE 领域工程师的、能够统一描述架构模型转换的模型转换语言，减轻系统工程师学习多种模型转换语言的负担，提高系统开发效率。

5.2 模型转换语言抽象语法

本文设计的模型转换语言在 KARMA（Combination of ARchitecture Model specificAtion）语言基础上进行扩展，KARMA 语言是一种以 GOPPRR-E 元建模方法为基础面向对象的文本式统一系统建模语言，能够形式化 GOPPRR-E 元元模型、元模型以及模型，从而实现对多个领域、多种特定域建模语言的统一描述^[84]。KARMA 语言分为多个模块，其中模型形式化模块是语言的基础，实现模型底层数据的统一描述，并提供元模型及模型元素的相关操作接口（API），实现模型数据的增删查改。本章节的模型转换语言在 KARMA 语言基础上增加模型转换功能，模型转换的源模型、源元模型、目标元模型及目标模型都以 KARMA 语言的形式描述，通过 API 对模型进行创建元素、删除元素、修改属性等模型数据操作，最大程度重用 KARMA 语言的统一描述能力。本章节基于已有的 KARMA 语言形式化描述规范，扩充模型转换语言规范，使 KARMA 语言具备模型转换描述功能。

模型转换语言按照功能可以分为三个部分：模式描述语言、规则描述语言、转换辅助语言。模式描述语言用于指定模型结构（模型元素之间连接的拓扑结构）以及模

型属性；规则描述语言定义规则的 LHS 和 RHS 及两者之间的映射关系，同时描述规则的执行顺序；转换辅助语言实现函数定义，源模型、源元模型指定、目标元模型指定，流程控制等辅助功能，模型转换语言的抽象语法如图 5.1 所示。

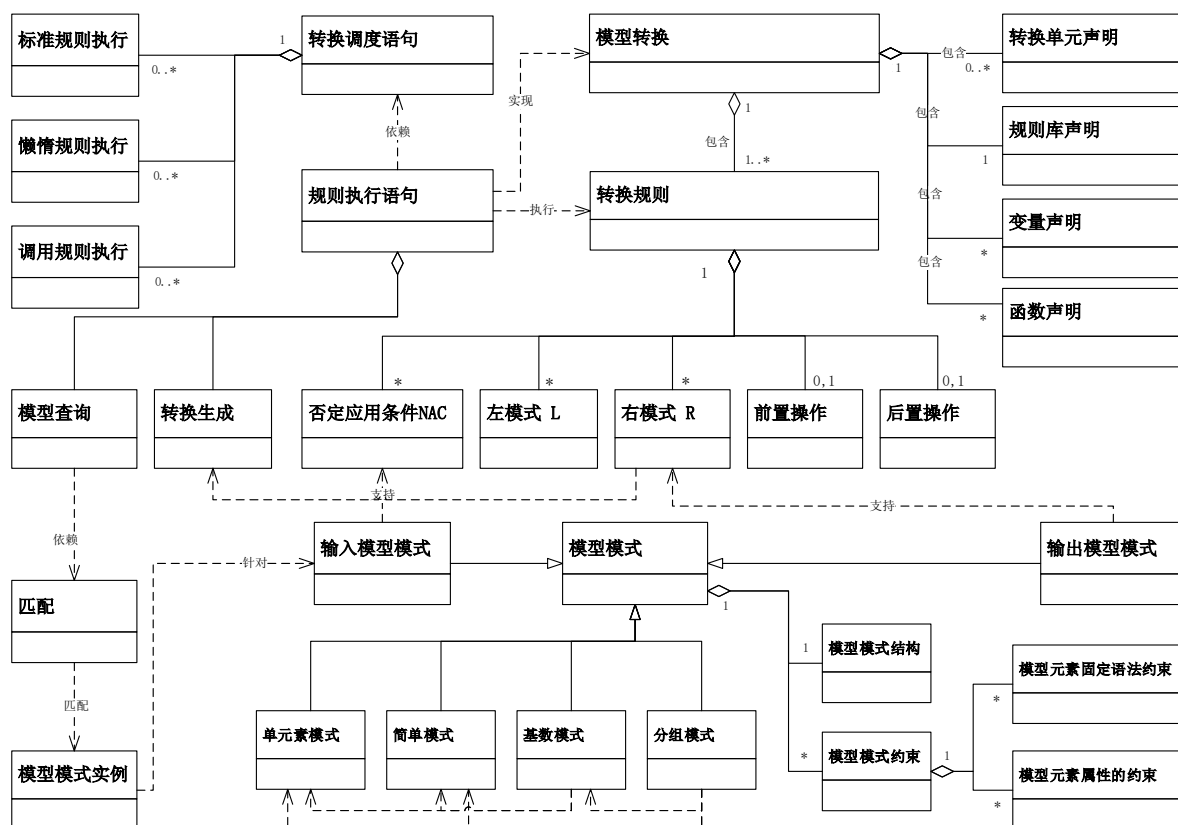


图 5.1 模型转换语言抽象语法类图描述

5.2.1 模式描述语言

模型模式是由某种建模语言的元模型元素的实例组成的模板^[85], 用于定义某个模型中模型元素实例之间的关联关系和属性约束, 由模型结构和模型约束两部分组成。模型结构, 定义源模型中模型元素实例(对象、关系、角色、点)之间的连接拓扑关系, 如图 5.2 所示, 模型模式定义了两个对象(O_1 、 O_2)通过关系 e_1 连接, 并限定了关系绑定的角色类型 Ro^{from} 、 Ro^{to} ; 在关联关系的基础上, 可以对模型元素实例的属性值以及固定语法进行约束, 例如限定某个对象整数类型的属性值大于某个数值。根据定义的模型模式, 在源模型中识别到一个与该模式具有相同模型结构的元素集, 称为一个模型模式实例。为了便于表示模型模式, 下文通过 $\{(T1, E1), (T2, E2), \dots\}$ 形式模型

模式实例，如图 5.2 所示，在模型实例中存在两个模型模式实例，分别记作： $\{(O_1, Obj_1), (ro_1, Ro_2), (e_1, R_1), (ro_2, Ro_5), (O_2, Obj_2)\}$ 和 $\{(O_1, Obj_1), (ro_1, Ro_1), (e_1, R_2), (ro_2, Ro_6), (O_2, Obj_3)\}$ 。

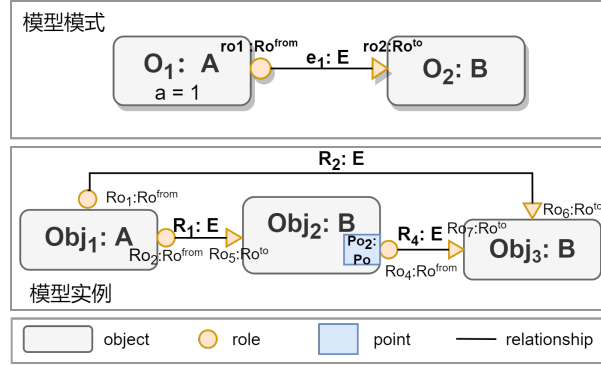


图 5.2 模型模式示意图

模型转换规则的 LHS 和 RHS 是两个模型模式，分别称为输入模型模式和输出模型模式，输入模型模式用于指定如何匹配源模型中的元素，输出模型模式指定如何生成目标模型元素。两种模式在模型约束描述方面含义有所不同，对于输入模型模式，模型约束表示对源模型元素属性的限制，对于输出模型模式，模型约束表示对生成的模型元素的属性进行赋值，图 5.2 中，模型模式作用于输入模型表示对象 O_1 的匹配项属性必须满足 $a = 1$ ，作用于目标模型表示对象 O_1 的属性 a 赋值为 1。

针对 GOPPRR-E 模型可能存在的拓扑关系，总结了单元素模式、简单模式、基数模式以及分组模式四种基本的模型模式^[86]。基于四种基本的模型模式可描述复杂的 GOPPRR 模型拓扑结构，以满足不同应用场景下的模型结构描述需求。

5.2.1.1 单元素模式

单元素模式中只包含一个对象实例，以指定模型中的对象实例的类型和属性。如图所示，单元素模式包含一个 A 类型的对象实例，在模型中存在一个模型模式实例 $\{(O_1, Obj_1)\}$ 。



图 5.3 单元素模式示例

5.2.1.2 简单模式

简单模式中列举了源模型中模型元素实例（对象、点、关系、角色）以及它们之间关联关系，其中关联关系是通过二元关系描述的（如表 5.1 所示）。根据对象之间的绑定关系，存在直接绑定和间接绑定两种形式的简单模式，直接绑定简单模式是指对象实例直接和关系上的角色绑定实现连接，间接绑定简单模式是指对象实例通过其上的点实例和关系上的角色绑定实现连接。基于两种基本的模式，其他情形的连接也可清晰描述（例如，通过某关系连接的两个对象一端为直接绑定，另一端为间接绑定）。

表 5.1 模型元素之间存在的二元关系

	<i>Graph</i>	<i>Object</i>	<i>Relationship</i>	<i>Role</i>	<i>Point</i>
<i>Graph</i>	-	containIn	containIn	-	-
<i>Object</i>	include	connectWith	source target	bindWith	containIn
<i>Relationship</i>	include	source target	-	startFrom endTo	-
<i>Role</i>	-	bindWith	startFrom endTo	-	bindWith
<i>Point</i>	-	include	-	bindWith	-
<i>Property</i>	have	have	have	have	have

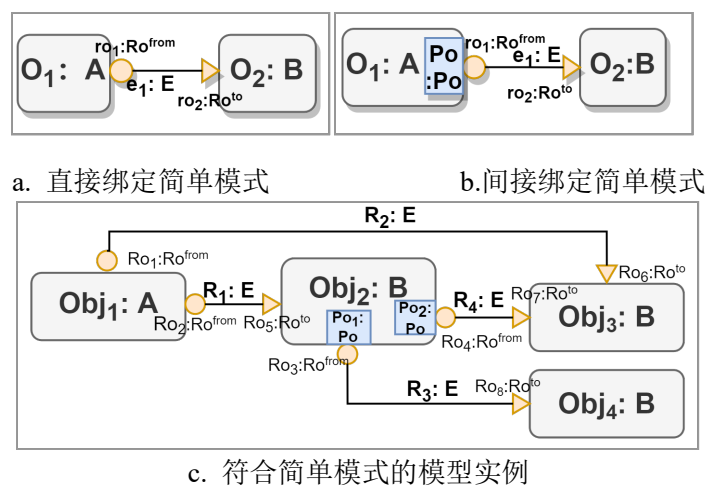


图 5.4 简单模式示例

如图 5.4 所示，模型实例 c 中存在两个直接绑定简单模式 a 的模型实例，分别记作 $\{(O_1, Obj_1), (ro_1, Ro_2), (e_1, R_1), (ro_2, Ro_5), (O_2, Obj_2)\}$ 和 $\{(O_1, Obj_1), (ro_1, Ro_1), (e_1, R_2), (ro_2, Ro_6), (O_2, Obj_3)\}$ ；同样存在两个间接绑定简单模式 b 的模型实例，记作 $\{(O_1, Obj_2),$

$(Po, Po_2), (ro_1, Ro_4), (e_1, R_4), (ro_2, Ro_7), (O_2, Obj_3)$ 和 $\{(O_1, Obj_2), (Po, Po_1), (ro_1, Ro_3), (e_1, R_3), (ro_2, Ro_8), (O_2, Obj_4)\}$ 。

5.2.1.3 基数模式

针对模型模式中某些相似的模型结构多次出现的场景，使用简单模式描述较为繁琐。例如模式中某个对象实例出现 n 次，则需要在简单模式中列举出 n 个对象元模型，采用基数模式可简化上述描述。基数模式通过基数指定某个（些）模型结构中对象、点、角色、关系的实例在模型实例中出现的次数。基数是个具体数字 n 或数字范围 (x, y) ， n 表示在模型实例中，基数模式中的元素严格出现 n 次；范围表示基数模型中元素出现的次数不少于 x 但不多于 y 。基数模式可以指定在对象、点、关系实例的次数，也可以对定义子模式（下文详细介绍）出现的次数。

对象基数模式描述了模型结构中对象实例出现的次数，如图 5.5 所示，模式 a 指定对象实例（具有 A 类型） O_1 出现次数为 1，对象实例 O_2 出现次数为 4，对象 O_1 和对象 O_2 通过 E 类型关系连接，在模型实例 d 中存在模型模式实例 $\{(O_1, Obj_1), (O_2, \{Obj_2, Obj_3, Obj_4, Obj_5\}), (ro_1, \{Ro_1, Ro_2, Ro_3, Ro_4\}), (ro_2, \{Ro_1', Ro_2', Ro_3', Ro_4'\}), (e_1, \{R_1, R_2, R_3, R_4\})\}$ 。

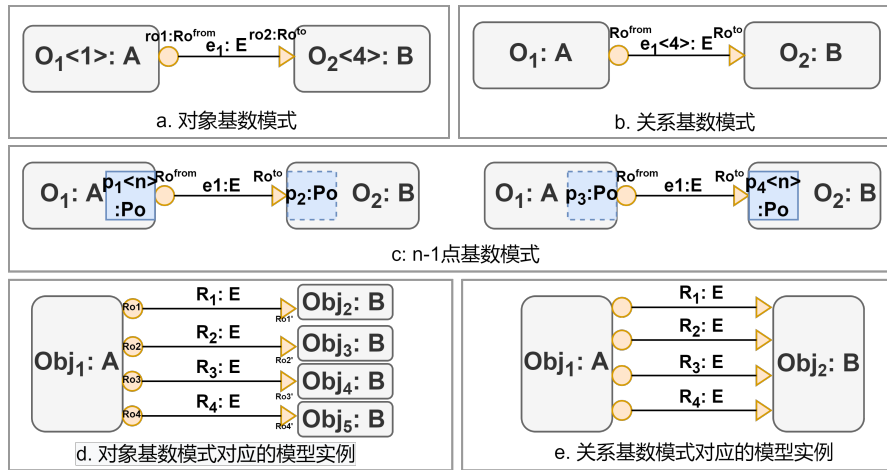


图 5.5 基数模式示例

关系基数模式指定模型结构中某种类型关系实例出现的次数，如图 5.5-b 所示模式指定 E 类型关系在对应对象实例之间出现的次数为 4，在模型实例 e 中存在模型模式实例 $\{(O_1, Obj_1), (O_2, Obj_2), (e_1, \{R_1, R_2, R_3, R_4\})\}$ 。

点和角色的绑定存在多种情况，通过基数模式可描述 $n-1$ 绑定场景。 $n-1$ 点基数模式表示对象实例上存在 n 个点实例，且通过这 n 个点和另一个对象（或点）实例连接（或被连接），如图 5.5-c 所示模式定义 2 个 Po 类型的点实例 p_1 通过指定的关系和角色连接到另一对象 O_2 ，模型实例图 5.6-d 中存在模型模式实例 $\{(p_1, \{p_4, p_5\}), (O_2, Obj_4)\}$ 。

5.2.1.4 分组模式

为描述更加复杂的模型结构，引入了分组模式的概念。分组模式将以一个模式通过子模式的方式分为若干个部分，上述三种模式都可以作为子模式的形式存在，分组模式以分层的方式将复杂的模型结构化解为多个简单结构模式，有利于复杂结构的分解。分组模式包含一个主模式和子模式，主模式中可嵌套定义若干个子模式。如图 5-6 所示主模式 a 包含两个子模式 pattern1 和 pattern2，pattern1 属于简单模式，记作 $P(Po1, ro)$ ；pattern2 属于基数模式，记作 $P(Po2<n>, O2)$ 。分组模式的典型应用场景是 1-n 和 n-n 点连接的无歧义描述。

如图 5.6-b 模式描述了一个 Po 类型的点实例通过指定类型的关系、角色和 2 个 B 类型的对象实例连接，其连接的角色、关系和对象通过子模式 pattern 指定，通过子模式的基数指定连接个数。模型图 5.6-d 存在模型模式实例 $\{(p, p_6), (O_2, \{Obj_5, Obj_6\})\}$ 。

$n-n$ 点连接是指对象实例 O_1 包含 n 个点实例，对于每一个点实例，都通过指定的角色、关系和其他对象（或点）实例连接。如图 5-6 所示模式 c 通过分组的方式描述 $n-n$ 点的连接，其中子模式描述了对象 O_1 上的每一个 Po 类型的点 p ，都通过对应角色以及关系 $e1$ 与另一个对象连接，并指定子模式的基数表示点-对象连接重复次数，模型图 5.6-d 存在模型模式实例 $\{(O_1, Obj_0), (pattern, \{(p, p_1), (O_2, Obj_1)\}, \{(p, p_2), (O_2, Obj_2)\}, \{(p, p_3), (O_2, Obj_3)\})\}$ 。

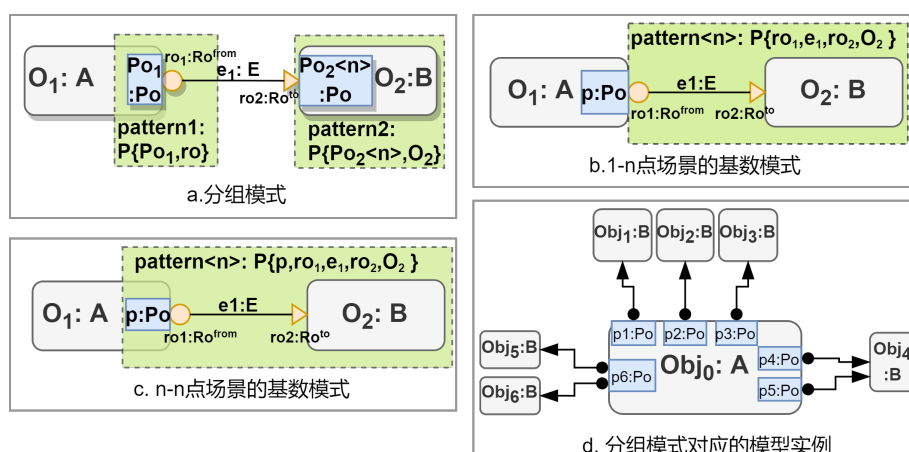


图 5.6 分组模式和点基数模式示例

5.2.2 规则描述语言

模型转换规则是模型转换语言的核心，其抽象语法如图 5.7 所示（省略了部分抽象语法，完整抽象语法见图 5.1），一个模型转换规则的主要组成部分为左模式声明、右模式声明、否定应用条件声明。为了在规则应用之前和规则应用之后执行某些操作，规则中可定义前置操作和后置操作语句。模型转换通常包括多条模型转换规则，在模型转换执行时需要按照某种顺序逐条应用规则，称为规则调度，规则的调度有显式调用和隐式调用两种，为满足规则的调度，有三种规则执行方式：标准规则执行、懒惰规则执行和调用规则执行。

标准规则执行是一种隐式规则调用，根据规则声明的顺序逐条应用规则。懒惰规则执行和调用规则执行是显示规则调用。

懒惰规则的执行与声明的顺序无关，只有在其他规则中显式调用时才执行，且每调用一次便执行一次，类似于函数的调用。

调用规则也是一种显示调用规则，和懒惰规则不同之处在于调用时可传入参数，而本身不能含有左模式定义，即调用规则本身不能实现匹配操作，在调用时需要传入已经匹配的模型模式实例。调用规则中有两类特殊情况，分别为标注了 *entrypoint* 和 *endpoint*，在开始转换时由模型转换引擎直接调用执行一次，*entrypoint* 在所有标准转换规则之前调用，*endpoint* 在所有标准转换规则之后调用。

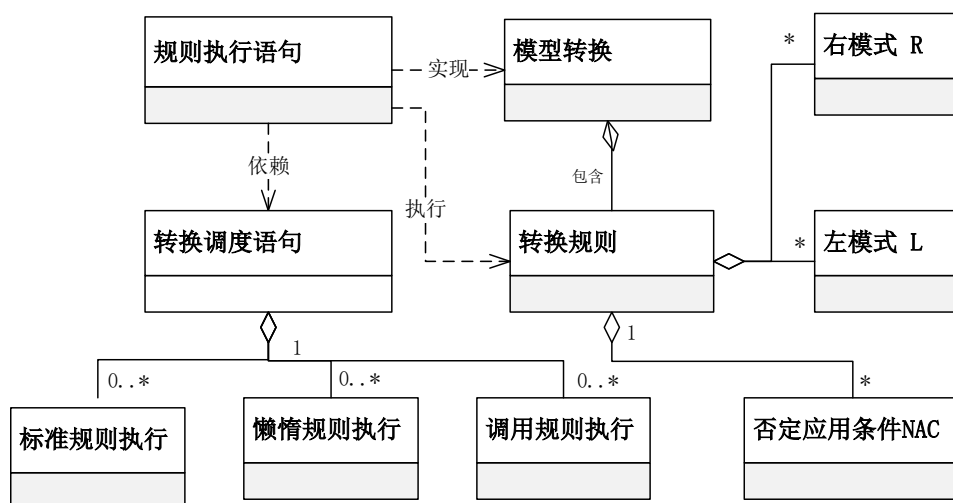


图 5.7 模型转换规则抽象语法

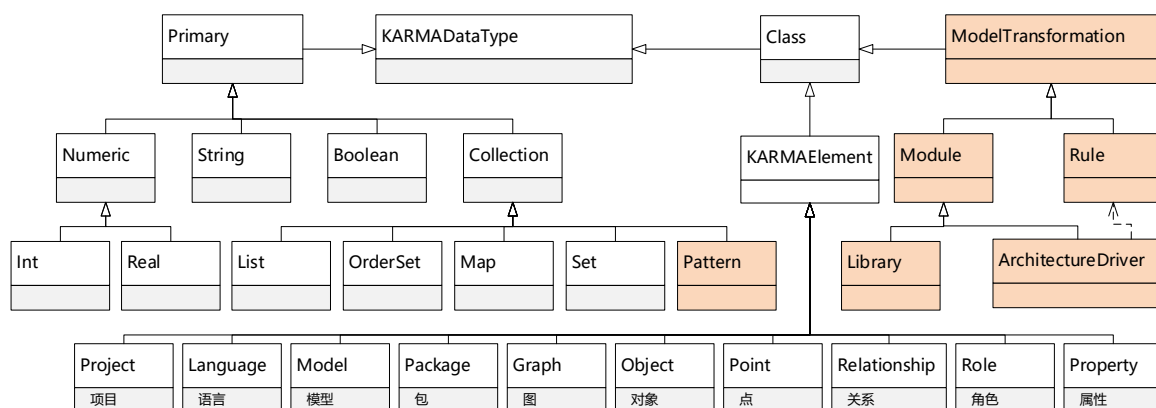
5.2.3 转换辅助语言

为实现模型转换的完整描述，模型转换语言提供其他声明性语句，例如转换单元声明、规则库声明、变量声明等。

转换单元声明用于表示面向对象语言中的一个“模型转换对象”，模型转换单元是一段包含规则以及相关声明的代码片段，通常通过文件区分模型转换单元，一个文件能且只能定义一个模型转换单元。在模型转换单元作用域内，可以定义变量、函数、规则等其他声明语句，这些变量以及语句作用范围只限于该转换单元。源模型、目标元模型的声明是模型转换单元最重要的声明，指定模型转换必需的源模型、源元模型以及目标模型，由于源元模型可通过源模型间接获得，故无需显式声明。

规则库声明是指将一个模型转换单元指定为规则库，在规则库中只能定义标准规则、调用规则和函数，不能含有变量或其他声明语句。在规则库定义好的规则可在其他模型转换单元中通过导入规则的方式调用，以实现库中的函数、规则的重用。

与面向对象的编程语言类似，模型转换语言支持声明变量和函数，定义在模型转换单元的变量称为全局变量，定义在规则中前置操作中的变量或定义在函数中的变量称为局部变量，全局变量在整个模型转换单元内共享，局部变量的作用范围只限定于规则或函数内部。模型转换语言是一种强类型语言，定义变量时需要指定变量类型，作为 KARMA 语言的扩展，模型转换语言继承 KARMA 语言的数据类型，并有相应扩展，数据类型如所示。



5.3 模型转换语言具体语法

5.3.1 模型转换语言关键字

表 5.2 模型转换语言关键字

5.3.2 语言具体语法设计

模型转换单元是一个代码段，由关键字 *architectureDriver* 标识一个模型转换单元开始并由 *end* 关键字标识结束，*modelTransformerName* 为该模型转换单元命名，与模型转换单元文件命名保持一致。规则、函数和变量等相关声明都位于代码片段中，其语法如下：

2) 变量定义和赋值

模型转换语言的变量声明和赋值继承自 KARMA 语言，在 KARMA 语言支持的数据类型上增加 *Pattern* 类型（下文单独介绍），以单独定义模型模式。支持的数据基本数据类型有基本数据类型 *Int*、*Real*、*Boolean*、*String*，集合数据类型 *List*、*OrderSet*、*Map*、*Set*、*Pattern*。模型转换语言为了直接操作 GOPRR 模型数据，将 GOPRR-E 模型元素抽象为模型相关数据类型，包括 *Project*、*Language*、*Model*、*Package*、*Graph*、*Object*、*Point*、*Relationship*、*Role*、*Property*。变量声明和赋值的具体语法如下，*type* 表示类型，*varName* 表示变量名，*value* 表示变量值。

```
1. type varName;
2. type varName = value;
```

例如基本数据类型变量声明赋值如下：

```
3. Int i;
4. Int j = 0;
5. Real c = 2.0;
6. Boolean b = false;
7. String st = "hello, world";
```

集合类型变量声明赋值如下所示：

```
1. List(Int) list = List{1, 2, 3}; //声明并初始化 List
2. Set(Int) set = Set{1, 2, 3};
3. Set(Int) set = Set{}; //声明初始化一个空 Set
4. OrderSet(Int) orderSet = OrderSet{1, 2, 3};
5. Map(String, Int) map = Map{"eve":13, "john":20}; //声明并初始化一个 map，键为字符串类型，值为整数类型
```

模型相关数据类型有所不同，对于 GOPRR-E 实例化后的模型元素需要指定对应的元模型，有通过 *id* 和 *localName* 两种方式指定对应的元模型。*elementType* 表示对应的 GOPRR 类，*id* 表示元模型元素的唯一标识，*localName* 表示元模型的 *localName*，*eleName* 表示变量名，变量声明语法如下：

```
1. elementType[id] eleName;
```

```
2. elementType["localName"] eleName;
```

例如以下模型相关变量声明：

```
1. Graph[sysml_UseCaseDiagram] g1; //通过 id 方式指定元模型声明模型
2. Graph["用例图"] g2; //通过 localName 方式指定元模型声明模型
3. Object[object_Usecase] obj1; //声明对象类型变量
4. Point[point_id] point1; //声明点类型变量
5. Role["role_name"] rol; //声明角色类型变量
6. Relationship[rel_id] rel; //声明关系类型变量
```

3) 规则库导入

导入模型转换规则库，导入后规则库中的规则和当前模型转换单元中规则等同。库中定义的规则通过“.规则名”直接引用，使用 *import* 关键字标识导入规则库，*ruleLibraryName* 为规则库名。

```
1. //在模型转换的脚本中引用库
2. import ruleLibraryName;
```

4) 源模型指定声明

用于指定转换单元操作的源模型、目标元模型，*create* 和 *from* 关键字分别创建目标模型和指定源模型，后具体语法如下：

```
1. create targetModel from sourceModel; //指定源模型为 sourceModel 目标模型为 targetModel
2. //创建目标模型模型，其元模型为 sysml_RequirementDiagram，通过模型 id 的方式指定源模型
3. create Graph[sysml_RequirementDiagram] targetDiagram from Graph(id =
    "model_PackageDiagram");
```

5) 函数

模型转换单元内部可自定义函数，封装某些操作流程，使用关键字 *func* 标识。使用函数时，通过函数名进行调用。函数可以带有参数和返回值，通过 *return* 关键字来返回值，当返回类型为 *void* 时，则不需要返回值，其语法如下：

```
1. func [returnType/void] funcName (parameters){
2. // funcBody
3. return variableName }
```

其中的 *parameters* 包括参数名和参数类型，一个简单的函数示例如下所示，在函数体 *funcbody* 中，可以使用多种 KARMA 支持的操作语句（赋值语句、if 语句、for、while 循环语句等）。

```
1. // 定义一个函数，输入为 String 类型变量，返回 String 类型结果
2. func String function_test (String a) {
3.     if(a == "female")
4.         return "lady";
5.     else
6.         return "gentleman";
7.     end
8. }
```

6) 规则

模型转换规则是由关键字 *rule* 标识的代码块，*ruleName* 表示规则名称，在模型转换规则代码块中定义左模式（由 *from* 关键字标识），右模式（由 *to* 标识），否定应用条件（由 *neg* 标识），前置操作（由 *using* 标识）以及后置操作（由 *do* 标识）。其中使用 *[]?* 中内容是可选的，标准模型转换规则语法如下：

```
1. rule ruleName {
2.     from      // 输入模式声明语句或模式调用语句
3.     [using { // 变量声明语句
4.         Int i = 1; //...
5.     }]?
6.     to      // 输出模式声明语句
7.         out_type1 out_var [{
8.             bindingExp
9.         }], //...
10.    [neg /*输入模式声明语句*/]?
11.    [do { /* 赋值等 statements*/ }]?
12. }
```

懒惰规则由 *lazy* 关键字标识，具体语法如下：

```
1. lazy rule lazyRuleName{
2.     from
3.         Point[point_id] a; // 输入模式声明语句或模式调用语句
4.     to
```

```

5.      // 输出模式声明语句
6.      String b(
7.          b <- thisModule.templateA (a);
8.      )
9.  }

```

调用规则需要定义参数且不能定义输入模式，即不包含 *from* 部分，两类特殊调用规则分别用 *entrypoint* 和 *endpoint* 关键字标识，其语法如下：

```

1.  [entrypoint | endpoint]? rule rule_name(parameters){
2.      [using {type var = init_exp; }]?
3.      to // 输入模式声明语句或模式调用语句
4.      [do { /*statements*/ }]?
5.  }

```

7) 模式

模式由若干条模式声明语句组成，模式声明语句包括模式变量声明语句、约束声明语句和二元关系声明，变量声明由模型数据类型 *elementType*，变量名列表 *variableDeclaratorList* 以及约束条件 *filterExpression* 组成；模型模式约束声明语句由 *constraint* 关键字标识，约束条件 *conditionalExpression* 可以是任意逻辑表达式；二元关系语句用于描述模型模式中 *element* 和 *anotherElement* 之间的连接关系，*binaryName* 为二元关系操作符，具体语法如下。

```

1.  elementType variableDeclaratorList ([filterExpression]?) //模式变量声明
2.  constraint ( conditionalExpression ) //模型模式约束声明语句
3.  element.binaryName(anotherElement);

```

有如下实例：

```

1.  //指定模型中的一个对象，其元模型 id 为 ablock，并对对象的 name 属性进行约束
2.  Object[ablock] obj1(obj1.Property["name"].value == "add_block");
3.  //指定模式中从关系 rel 到对象 obj1 之间的连接结构
4.  rel.source(obj1);
5.  constraint( obj1.id == "objectIns_block3" ); //对对象 obj1 的 id 进行约束

```

通常由多条模式声明语句才能描述一个完整的模型模式，也可以通过由 *pattern* 标

识的模式单元封装模式，在 `pattern` 中定义若干模式声明语句以描述模式结构和约束，模式单元可类似于函数定义输入参数，在调用模式单元时传入实际参数。

```
1. pattern patternUnitName (parameters) { /*模式声明语句*/ }
```

例如如下模型模式声明描述了图 5.4-a 的简单模式：

```
1. Object[A] o1; Role[RO_from] ro1; Role[Ro_to] ro2; Relationship[E] e1; Object[B] o2;
2. o1.bindWith(ro1)
3. e1.startFrom(o1);
4. e1.endTo(o2);
5. o2.bindWith(ro2);
6. constraint(value(o1.property) > 0 && value(o2.name) == "o2");
```

基数模式中的基数指定通过 `cardinality` 关键字标识，具体语法如下：

```
1. cardinality( obj2 ) = 3; //指定对象 obj2 出现的次数为 3
```

下例所示模型模式单元描述了图 5.5-a 的基数模式：

```
1. Object[A] o1; Role[RO_from] ro1; Role[Ro_to] ro2; Relationship[E] e1; Object[B] o2;
2. o1.bindWith(ro1);
3. e1.startFrom(o1);
4. e1.endTo(o2);
5. o2.bindWith(ro2);
6. cardinality(o2) = 2;
```

分组模式需要将子模式定义为一个模式单元，在主模式中通过 `match` 关键字调用，并传入子模式参数，分组模式也可以通过 `cardinality` 关键字指定基数，具体语法如下：

```
1. match subPatternName(parameters)
```

例如如下模型模式声明描述了图 5.6-c 的分组模式：

```
1. pattern pointCardinalityNTon(Object[A] o1){
2.   Point[Po] p; Object[B] o2; Role[RO_from] ro1; Role[RO_to] ro12; Relationship[E] e1;
3.   o1.include(p);
4.   match subPatternPointToObject(p,o2,ro1,ro2,e1);
```

```

5.   cardinality(subPatternPointToObject) = 2; }
6.
7.   pattern subPatternPointToObject(Point[Po] p, Object[B] o2, Role[RO_from] ro1,
   Role[RO_to] ro2, Relationship[E] e1){
8.     p.bindWith(ro1);
9.     e1.startFrom(ro1);
10.    e1.endTo(ro2);
11.    ro2.bindWith(B);
12. }

```

输入模式主要用于匹配，输出模型主要用于生成，因此，输出模式的有额外的绑定语句。绑定语句类似于赋值语句，不同的是绑定语句可以指定模型元素之间的绑定关系，*var* 表示变量，*value* 表示值，具体语句如下：

```

1. var <- value;
2. rel.source <- ann; //将 ann 对象和 rel 关系绑定，ann 对象为 rel 的起始端

```

规则中的绑定语句用法如下：

```

1. rule ObjecttoAddblock{
2.   from //输入模式省略
3.   to
4.     Object[object_annotation] ann, //输出模式指定生成对象 ann
5.     Relationship[relsys] rel( //指定生成关系 rel
6.       rel.source <- ann; //指定关系的起始端通过默认的角色与对象 ann 绑定
7.     )
8. }

```

5.4 模型转换引擎开发

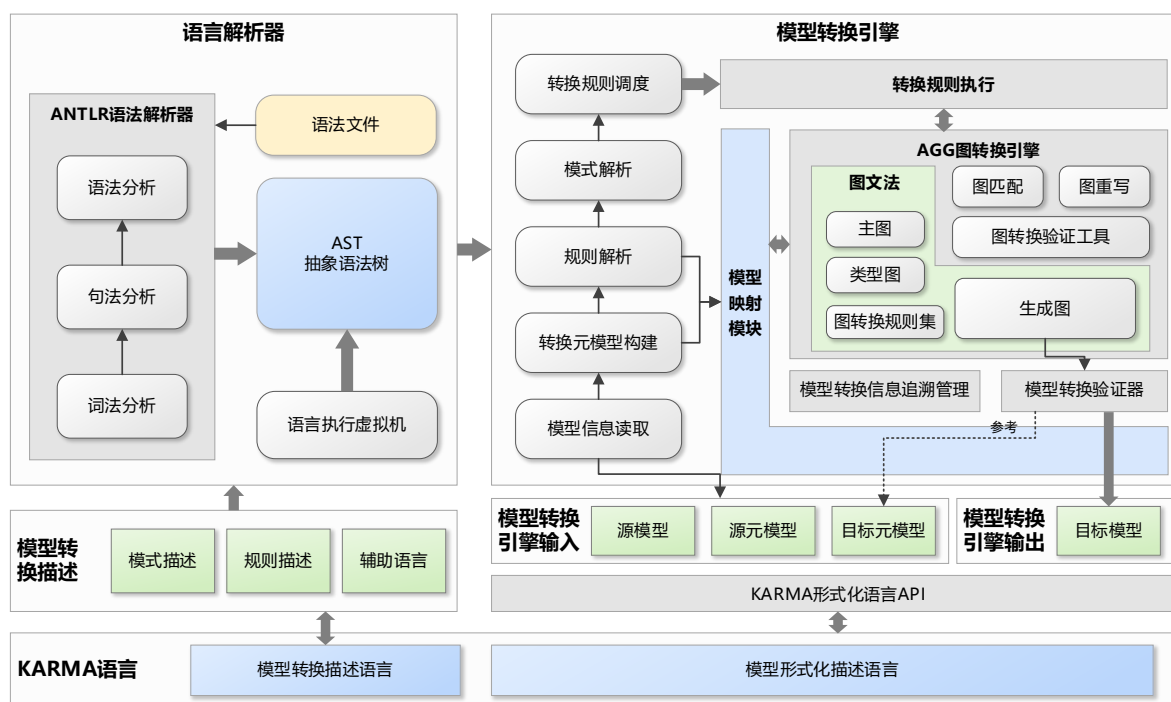


图 5.9 模型转换引擎开发架构

模型转换引擎包含模型转换描述语言解析器、模型转换引擎两个核心模块。其中语言解析器借助了开源语法解析器工具 ANTLR（Another Tool for Language Recognition），模型转换引擎基于 AGG 图转换引擎开发。

5.4.1 基于 ANTLR 的语言解析器

模型转换语言解析器主要功能是解析模型转换描述语言，将模型转换语言解析为虚拟机可执行的抽象语法树，解释执行。首先需要将模型转换语言解析为抽象语法树，语言执行虚拟机根据抽象语法树进行语义解析，由虚拟机调用模型转换引擎实现转换功能。其中语法解析器借助 ANTLR 工具，ANTLR 是一个开源模型的语法解析器，能够根据定义好的语法文件自动解析输入语言，自动生成抽象语法树，并生成语法树遍历框架，提供针对语法树的操作接口。模型转换语言的语法文件见附录 B。

5.4.2 基于 AGG 图转换引擎的模型转换引擎

模型转换引擎是实现转换功能的核心，本文的模型转换引擎基于 AGG 图转换引

擎实现, AGG 是柏林工业大学 Gabriele 教授等人开发的图转换系统, AGG 基于代数图变换理论实现代数图重写, 其环境由图形用户界面和图转换引擎组成, 包括几个可视化编辑器、解释器和一组验证工具。AGG 转换引擎支持多种图转换验证, 包括图解析、图一致性检查和基于关键对分析方法的转换规则冲突检测, 因此能够实现图转换本身正确性验证。模型转换的主要流程如下:

根据模型转换语言的描述读取源模型、源元模型以及目标元模型, 并根据第 3 章方法构建转换元模型, 模型映射模块将源模型、源元模型以及目标元模型映射为图文法中的相关图、类型图。

解析模型转换规则, 映射为图转换规则, 分别解析模型转换规则的左模式、右模式以及否定应用条件, 将解析后的模式映射为图转换规则的 LHS、RHS 以及 NAC, 并将映射后产生的图转换规则添加到图文法的图转换规则集中。

模型转换调度是指根据指定的规则执行顺序, 调用 AGG 图转换引擎执行对应的图规则, 对输入模型映射产生的主图进行重写, 规则执行结束后得到生成图。

由图转换引擎产生的生成图需要经过模型转换验证器进行正确性验证, 保证生成的模型符合目标元模型。最后, 经模型转换验证器验证正确的生成图, 将其映射为目标模型, 完成转换。上述模型以及元模型相关数据都通过 KARMA 形式化描述语言统一描述, 并通过 API 完成读取修改等操作。

5.5 本章小结

本章在前几章研究基础上提出模型转换统一描述语言, 该语言继承现有的 KARMA 语言, KARMA 语言能够实现基于 GOPPRR-E 元建模方法的异构模型的统一形式化表达。本章从模型转换语言的抽象语法设计(第 5.2 节)、具体语法设计(第 5.3 节)两个方面对介绍, 并引入模型模式的概念, 针对 GOPPRR-E 模型的特点, 提出单元素模式、简单模式、基数模式和分组模式四种典型的模型模式。最后开发模型转换语言解析器和模型转换引擎, 实现系统架构模型转换的描述能力。

第6章 面向无人平台系统架构设计的模型转换验证

6.1 引言

本章节以上述架构模型转换方法、模型转换正确性验证方法以及模型转换统一描述语言等研究为基础，对无人平台系统级架构模型进行开发，验证上述方法的有效性。案例按照需求-功能-逻辑-物理设计流程构建无人平台系统级架构模型，并通过模型转换方法实现系统级的需求到功能架构、功能架构到逻辑架构、功能架构到物理架构模型的自动转换，最终得到各个系统层级的架构模型。

6.2 无人平台架构设计背景

无人平台是一种可自行行驶的，能够根据远程指令执行作战任务的复杂武器装备系统，无人平台融合了机械、信息、电子和人工智能等多个学科，能够适应更加严苛极端的作战环境。无人平台的研制是一项复杂的系统工程，采用 MBSE 方法开发系统有利于管理复杂性，提高开发效率。

采用 MBSE 方法的架构设计是无人平台开发的基础，架构设计通过建立顶层模型对无人平台进行概念设计，对无人平台涉及到的概念以及概念之间的关系进行建模，为详细设计提供依据。无人作战平台的系统架构设计具有多层级、多领域的特点。从系统层级的维度看，架构设计涉及到不同层级系统的协同研制，包括系统级、分系统级和部件级等，在不同层级上需要建立不同粒度的架构模型，反复迭代，确保架构在不同颗粒度上的可行性与可靠性。从涉及学科的维度看，无人平台架构设计涉及到机械、电气、液压、控制等多领域多学科的交叉融合，需要通过建立不同学科的架构模型，从多个视图上描述系统整体。因此，系统工程师往往采用需求、功能、逻辑和物理架构，即 RFLP 流程实现正向设计，如图 6.1 所示。

在传统的 RFLP 设计流程中，需要在不同系统层级上按照需求、功能、逻辑、物理的顺序手动建立架构模型。采用模型转换方法可通过定义模型转换规则，根据上层模型自动生成下层模型，在生成模型的基础上进行模型精化，实现模型数据的追溯，提高建模效率。因此，本章节以无人平台的总体架构设计为例，采用模型转换的方法半自动生成建模过程中的部分模型，提高模型数据的可追溯性。

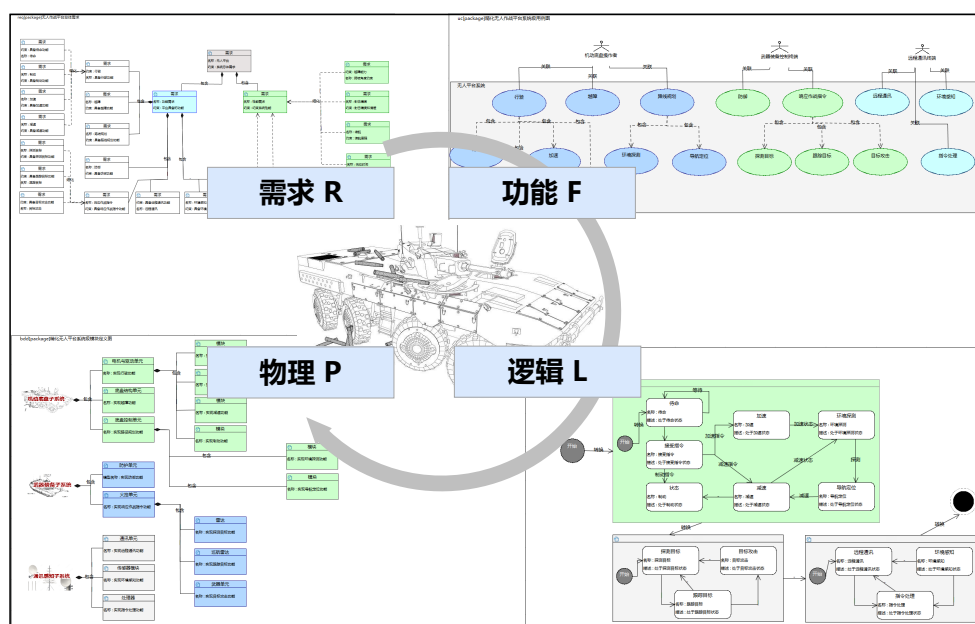


图 6.1 基于 RFLP 设计流程的无人平台架构设计

6.3 无人平台架构模型转换流程

根据 RFLP 设计流程，无人平台架构设计按照从系统、分系统以及部件三个系统层级进行架构设计，其设计流程如图 6.2 所示。对于每一个系统层级，在早期根据定义明确的利益相关方需求建立完备的需求集合是十分重要的，在后期架构设计中，需求的变更将造成项目成本几何级的增长。其中系统需求是进行架构设计的基础，在分析利益相关方业务需求的基础上对系统需求进行建模，并基于系统需求进行系统定义及架构设计，需求的实现即表示了系统的实现过程。接着，在得到系统需求模型的基础上，建立该系统层级的功能架构。功能架构是将需求分析阶段获得的系统需求转化为可执行的模型，是系统黑盒分析的过程，通常使用 SysML 中的用例图、活动图、时序图、状态图等描述功能架构。以系统需求作为输入条件，通过顶层功能定义、功能分解与层级定义，输出系统的功能需求以及功能架构等。逻辑架构关注的是系统内部接口信息交互关系，将系统的功能分配给不同的系统组成。从上一步骤功能-非功能分析细化的结果出发，将系统分解为若干个逻辑组件，该设计过程是一个由上至下的分解分配过程。物理架构设计属于较低系统层级的架构设计，主要对系统的软硬件组件以及组件之间的关系进行建模，一旦完成这一步，系统便可以交给“更低”层级进行开发。

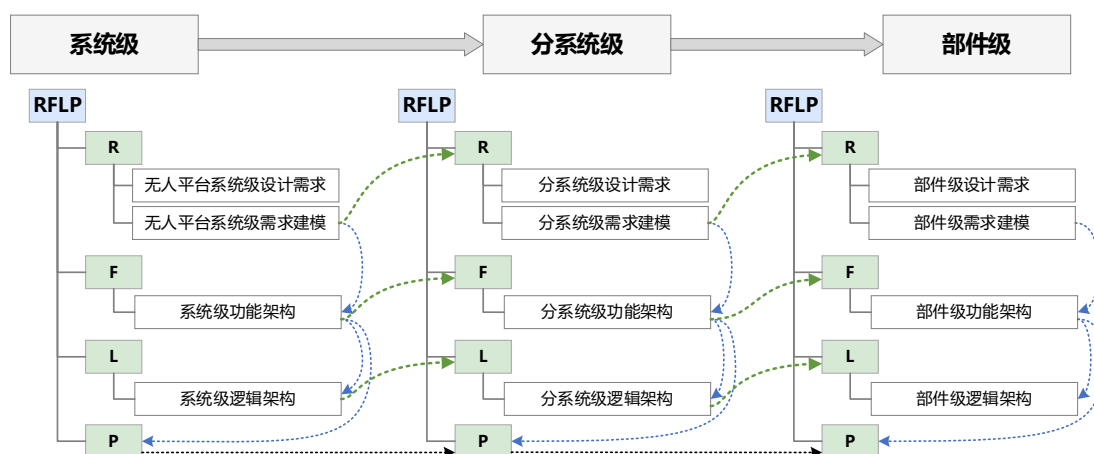


图 6.2 基于 RFLP 流程的多系统层级架构设计

基于上述 RFLP 分析流程，本案例以 GOPRR-E 元建模框架构建流程中需求、功能、逻辑和物理模型。由于在工程中，这几种模型通常基于 SysML（系统建模语言）构建，为利用本文提出的模型转换方法实现架构模型的转换，首先需要通过 GOPRR-E 元建模方法构建 SysML 语言，即 GOPRR 六种元元模型定义 SysML 九种图元模型，在此基础上通过构架好的 SysML（系统建模语言）图元模型建立各系统层级的需求、功能、逻辑和物理模型，对系统的功能架构、逻辑架构和物理架构进行描述。

SysML 系统建模语言是一种针对系统工程领域，可描述复杂系统架构的图形化建模语言，SysML 建模语言的图模型如图 6.3 所示，SysML 分为需求图、行为图和结构图三类，定义了九种模型视图，从不同视角对整个系统架构进行描述。

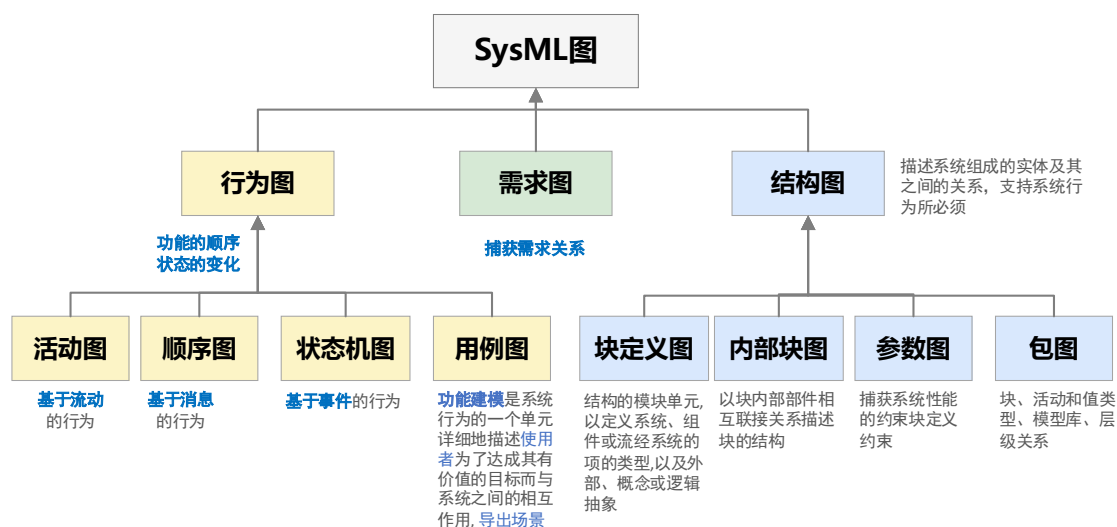


图 6.3 SysML 系统建模语言的九种图

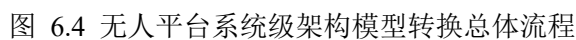
如表 6.1 所示，系统需求建模主要通过需求图构建，需求图是 UML 类图扩展而来的，用于对系统需求和需求的可跟踪性进行建模。需求的可跟踪性可通过包含、跟踪、继承、改善、满足和验证六种需求关系描述；功能架构是描述系统包含的功能以满足需求分析阶段定义的功能性需求，通常通过用例图、包图和活动图构建。用例图用于描述系统提供的功能以及功能和利益相关者之间的关系，包图可对功能进行划分；逻辑架构通过序列图、状态机图和活动图构建，活动图用于描述功能的动态行为，序列图可以表达运行过程中用例的动态变化行为，描述参与者之间的信息交互。物理架构通过模块定义图、内部模块图、包图和参数图构建，模块定义图和内部模块图用于描述系统物理结构包含的组件以及组件之间的关联关系，参数图描述组件的参数，包图对组件进行逻辑上的划分。

表 6.1 RFLP 设计流程与 SysML 视图对应关系

设计阶段	模型图表示
需求分析	需求图
功能分析	用例图、包图、活动图
逻辑分析	序列图、状态机图、活动图
物理架构设计	模块定义图、内部模块图、包图、参数图

在上述基于 RFLP 流程的无人平台架构设计场景中，提出了如图 6.4 所示的无人平台系统级架构模型转换总体流程（总体流程图中内容在下文中有详细介绍）。以系统级的 RFLP 设计流程为例，①在系统需求模型基础上，建立需求模型到用例模型的转换规则，将需求模型半自动转换为用例图描述的功能架构模型，实现需求到功能架构的转换。之所以是半自动转换，是因为由于需求模型信息不足以包含所有用例图语义，生成的用例图并不完整，需要在生成用例图的基础上手动添加执行者等元素，精化模型，得到完善的用例图；②建立用例图到状态机图的转换规则，实现功能架构到逻辑架构的半自动转换；③建立用例图到模块定义图的转换规则，生成系统架构模型的接口，由功能架构半自动转换为物理架构。硬件或软件开发工程师在生成的架构模型上添加模型细节，得到最终（或分配到分系统、部件级）物理架构模型，完成物理架构设计。

由于 SysML 建模语言划分为九种图，九种图虽属于同一种建模语言，但遵循的元模型不完全相同。例如每种图包含不同类型的建模元素，相同固定语法的建模元素



6.4 架构模型转换分析

基于 RFLP 方法开发系统架构过程中，需求是整个架构设计的输入条件，对系统架构进行约束，因此首先需要对无人平台的用户需求进行分析，建立系统层级的需求模型。无人平台从实现功能的角度上由三个部分组成，分别是机动底盘子系统、武器装备子系统和感知通信子系统。机动底盘子系统用于承载整个平台，在控制指令下实现平台移动和越障等功能；武器装备子系统用于响应作战指令和平台防御，对作战目标进行精准打击、实现目标拦截防御等功能；感知和通信子系统用于感知探测环境，和远程操控终端通讯并将处理接受的指令，将指令下达给相应的子系统。

通过对上述三个子系统的利益相关方需求分析，基于 SysML 需求图构建了如图 6.5 所示的无人平台系统总体需求模型，系统总体需求包括功能性需求和性能需求，功能性需求指无人平台系统必须具备的功能，性能需求对无人平台的机动性能、打击精度性能、越障性能等性能进行约束。

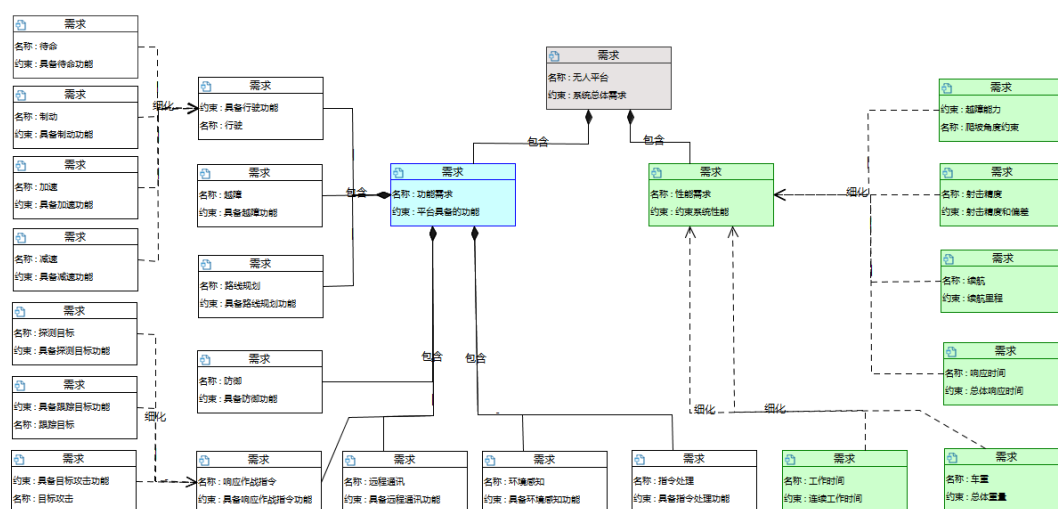


图 6.5 无人平台系统总体需求模型

6.4.1 需求到功能架构的转换

无人平台系统总体需求通过需求图描述，系统级功能架构主要通过用例图描述，因此，以需求图到用例图的转换为例，说明需求到功能架构的转换。转换的目的是将需求图中的功能性需求转化为用例图中的用例，以描述系统为了满足需求应该具备的功能。在如图 6.5 所示的需求模型中，功能性需求包含“行驶”、“越障”等多个子需求。

求, 并通过需求细化的方式对子需求进行分解, 例如“行驶”子需求可细化为“待命”、“加速”、“减速”和“制动”子需求。在转换过程中, 包含于功能性需求的子需求 (及其细化子需求) 转换为用例图中的用例, 子需求之间细化关系应转换为用例之间的包含关系, 对应关系如表 6.2 所示。在分析对应关系的基础上, 设计如图 6.6 所示意的转换规则。

表 6.2 需求图到用例图转换的元素对应关系

输入模型的元素	输出模型的元素
需求图	用例图
需求对象	用例对象
需求的包含关系	—
需求的细化关系	用例的包含关系

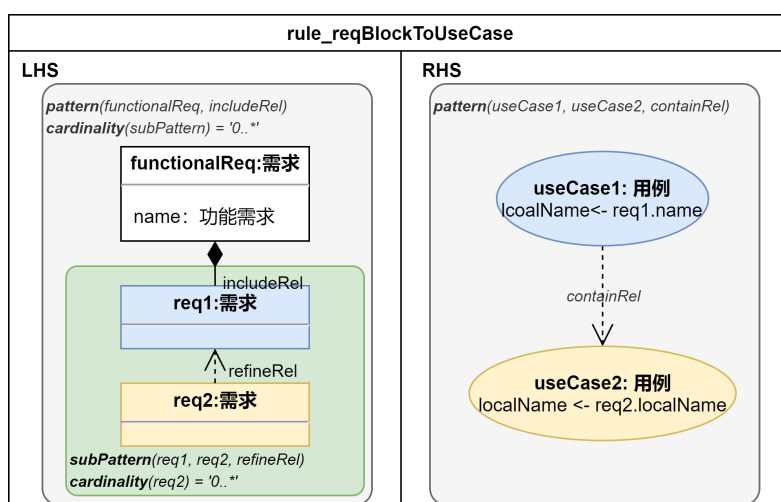


图 6.6 需求图到用例图转换规则示意图

规则的 LHS 中 *functionalReq* 对象对 *name* 属性进行限制以匹配功能需求, 忽略非功能需求 (图 6.5 中绿色表示的需求), *req1* 对象用于匹配 *functionalReq* 直接包含的对象, *req2* 对象用于匹配和 *req1* 通过细化关系连接的元素, 为了清晰描述 *req1* 和 *req2* 之间的细化关系, 采用分组模式 *subPattern* 描述。由于需求图中 *functionalReq* 匹配项直接包含多个 *req1* 匹配项, 因此采用基数模式将 *subPattern* 的基数定义为 $[0..*]$ 。同理在子模式中 *req1* 匹配项可能被多个 *req2* 匹配项细化, 定义 *req2* 的基数为 $[0..*]$ 。

规则的 RHS 中根据 LHS 匹配到的结果创建两个 *useCase1* 和 *useCase2* 两个用例, 其中 *useCase1* 的 *localName* 由 *req1* 匹配项的 *name* 属性赋值, *useCase2* 的 *localName*

由 *req2* 匹配项的 *localName* 赋值，最后在两个用例之间添加包含关系 *containRel*。

无人平台的系统总体需求图到用例图转换的模型转换语言描述如下：

```

1. architectureDriver reqToUsecase
2.   // 创建空的用例图 userCaseGraph, 其图元模型为 sysml_UseCaseDiagramSysML
3.   create Graph[sysml_UseCaseDiagramSysML] userCaseGraph from
      Graph.model_sysml_RequirementDiagram;
4.   pattern subPattern(Object[objectIns_object_Requirement91] req1, req2, Relationship[Refine]
      refineRel){
5.     refineRel.source(req1);
6.     refineRel.target(req2);
7.     cardinality(req2) = '0..*';
8.   }
9.
10.  rule reqBlockToUsecase{
11.    from
12.      Object[objectIns_object_Requirement91] functionalReq;
13.      Object[objectIns_object_Requirement91] req1, req2;
14.      Relationship[Contain] includeRel;
15.      Relationship[Refine] refineRel;
16.      constraint(functionalReq.Property["name"].value == "功能需求");
17.      includeRel.source(functionalReq);
18.      includeRel.target(req1);
19.      match subPattern(req1, req2, refineRel);
20.      cardinality(subPattern) = '0..*';
21.    to
22.      Object[object_Usecase] useCase1(useCase1.localName <- req1.Property["name"].value),
23.      Object[object_Usecase] useCase2,(useCase2.localName <- req2.localName)
24.      Relationship["GenerateRel"] containRel(
25.        containRel.source <- useCase1;
26.        containRel.target <- useCase2;
27.      )
28.  }
29. end reqToUsecase

```

经过模型引擎执行后生成如图 6.7 所示用例图。生成的用例图需要经过精化，添加执行者元素和执行者与用例之间的关联关系，最终得到如图 6.8 所示完整的无人平台系统级用例图。

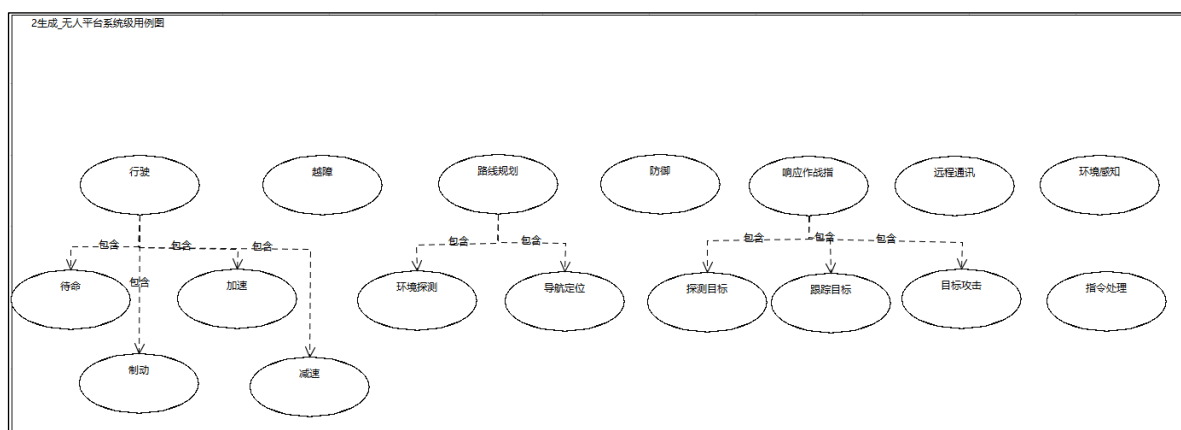


图 6.7 模型转换生成的用例图

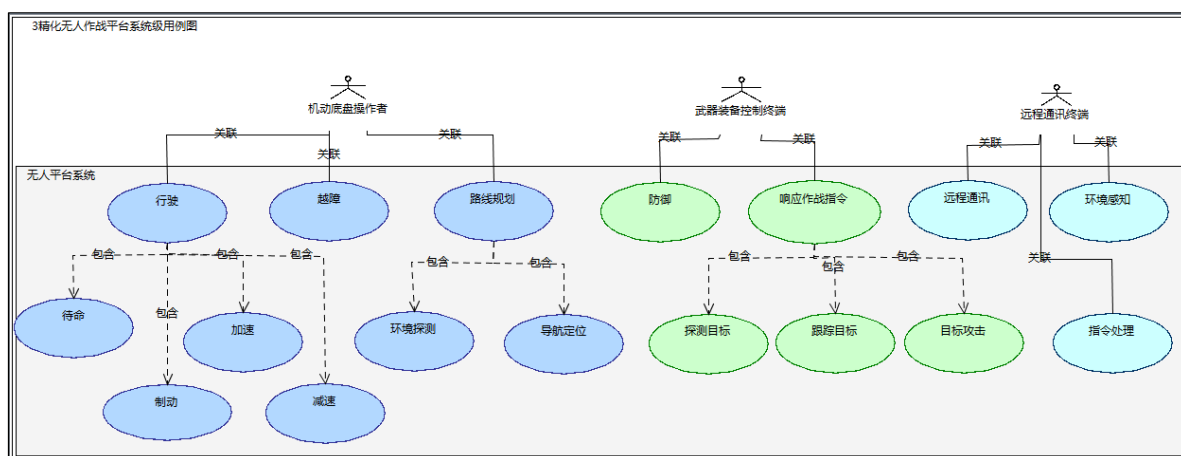


图 6.8 无人平台系统级用例图

无人平台需求到功能架构的模型转换中，以如图 6.5 所示的需求模型为源模型，目标模型为如所图 6.7 示用的例模型。转换结果显示，根据定义的模型转换规则，需求图中的“功能需求”下的所有子需求对象，都正确转换为用例图中的用例对象，且根据需求对象的 *name* 属性为用例的 *localName* 属性赋值，例如“行驶”需求对象转换为“行驶”用例；子需求之间的“细化”关系正确转换为用例对象之间的“包含”关系，例如待命子需求“细化”行驶子需求，转换为行驶用例“包含”待命用例。在模型转换生成目标模型基础上，添加“机动底盘操作者”、“武器装备控制终端”和“远程通讯终端”外部执行者，并建立关联关系，得到完整的用例模型。模型转换半自动生成用例模型，需求模型更改后，重新应用模型转换规则即可生成新的用例模型，无需从头构建用例模型，提高模型开发效率。

6.4.2 功能架构到逻辑架构的转换

无人平台系统级逻辑架构主要通过状态机图描述，因此，以用例图到状态机图的转换为例，说明功能架构到逻辑架构的转换。转换的目的是将用例图中的用例转化为状态机图中的状态，以描述系统在实现某功能过程中其状态发生的变迁。在上步骤生成并精化产生的图 6.8 所示的用例图中，主要包含“机动底盘操作者”、“武器装备控制终端”和“远程通讯终端”三个执行者，分别与描述对应功能的用例关联，功能的分解通过用例之间的包含关系描述。在转换过程中，根据执行者创建对应的子状态，对于与执行者关联的用例，将其包含的用例转换为状态，并添加到相应的子状态中，描述实现子功能时系统状态的变迁，对应关系如表 6.3 所示。在分析对应关系的基础上，设计如图 6.9 所示意的转换规则。

表 6.3 用例图到状态机图转换的元素对应关系

输入模型的元素	输出模型的元素
用例图	状态机图
执行者对象	子状态对象
用例对象	状态对象
执行者和用例的关联关系	—
用例之间的包含关系	—

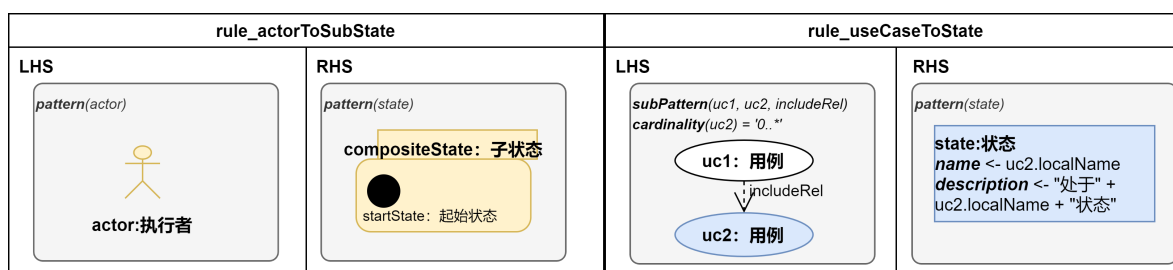


图 6.9 用例图到状态机图转换规则示意图

用例图到状态机图的转换需要两个转换规则完成，*actorToSubState* 规则实现执行者对象到子状态对象的转换，*useCaseToState* 规则实现用例对象到状态对象的转换。*actorToSubState* 根据 LHS 匹配到的 *actor* 对象在生成模型中创建 *compositeState* 对象，由于每一个子状态需要一个起始对象，因此规则的 RHS 创建了 *startState* 对象。*useCaseToState* 规则的 LHS 描述了用例 *uc1* 和 *uc2* 之间通过包含关系 *includeRel* 连接，

由于用例图中 *uc1* 匹配项可包含多个 *uc2* 匹配项, 因此采用基数模式将 *uc2* 的基数定义为 $[0..*]$ 。 *useCaseToSubState* 规则的 RHS 根据 LHS 匹配到的 *uc2*, 创建 *state* 状态对象, *state* 对象的 *name* 属性由 *uc2* 匹配项的 *localName* 赋值, *description* 属性由 *localName* 经过字符串运算之后赋值。例如对于匹配到的“加速”用例对象, 转换之后的状态对象的 *description* 属性变为“处于加速状态”。

无人平台的系统级用例图到状态机图转换的模型转换语言描述如下。其中为了创建状态机中的开始状态和终止状态, 定义 *entrypoint* 类型的调用规则 *createStartState*, 用于创建状态机中的起始状态对象; 定义 *endpoint* 类型的调用规则 *createEndState*, 用于创建状态机中的终止状态对象。此外, 在生成每个子状态的规则 *actorToSubState* 也调用了 *createStartState* 规则以创建子状态的起始状态对象。

```

1. architectureDriver useCaseToStateMachine
2.   create Graph[sysml_StateMachineDiagramSysML] smGraph from
   Graph.model_sysml_UseCaseDiagramSysML2;
3.   // 创建起始状态对象
4.   entrypoint rule createStartState(){
5.     to
6.       Object[object_Start] startState;
7.   }
8.   //将执行者对象转换为子状态对象, 并生成子状态的起始对象
9.   rule actorToSubState{
10.    from
11.      Object[object_Actor] actor;
12.    to
13.      Object[object_CompositeState] compositeState,
14.      createStartState();
15.  }
16.  //将用例对象转换为状态对象
17.  rule useCaseToState{
18.    from
19.      Object[object_Usecase] uc1, uc2;
20.      Relationship[relationship_Dependency2] includeRel;
21.      includeRel.source(uc1);
22.      includeRel.target(uc2);
23.      cardinality(uc2) = '0..*';
24.    to
25.      Object[object_State] state(

```

```

26.     state.Poperty["name"].value <- uc2.localName;
27.     state.Poperty["description"].value <- "处于" + uc2.localName + "状态";
28.   )
29. }
30. //创建结束状态对象的规则
31. endpoint rule createEndState(){
32.   to
33.     Object[object_Stop2] compositeState
34. }
35. end useCaseToStateMachine

```

经过模型引擎执行后生成如图 6.10 所示状态机图。生成的状态机图需要经过精化，添加额外的状态以及状态之间的转移关系，最终得到如图 6.11 所示完整的无人平台系统级状态机图。

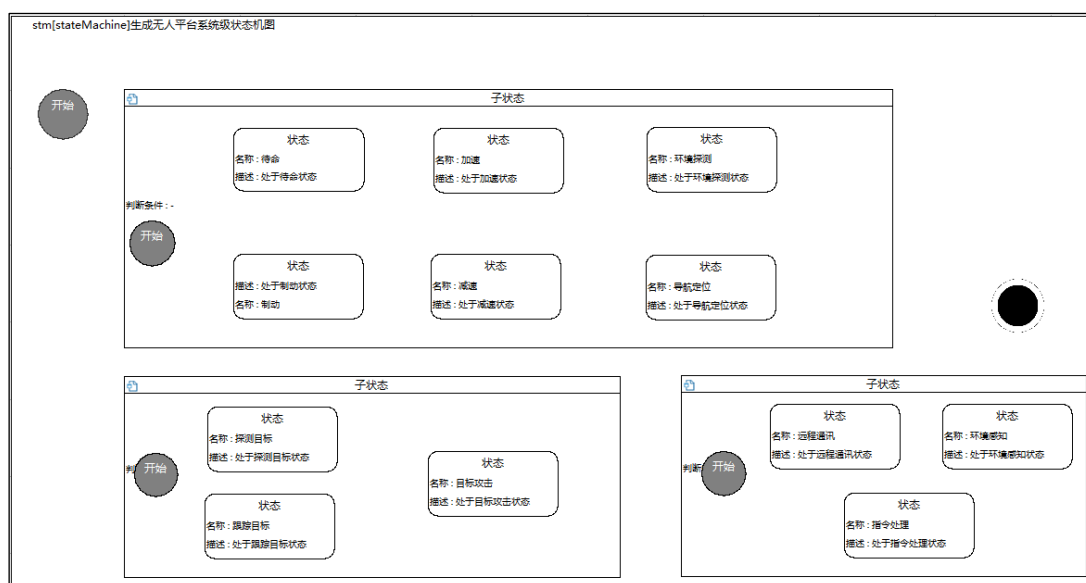


图 6.10 模型转换生成的状态机图

无人平台功能架构到逻辑架构的模型转换中，以如图 6.8 所示的用例模型为源模型，目标模型为如图 6.10 所示的状态机模型。转换结果显示，根据模型转换规则 *actorToSubState*，用例图中执行者正确转换为子状态，例如“机动底盘操作者”转换为图 6.10 左上方的子状态；根据规则 *useCaseToState*，将通过包含关系连接的用例对象转换为相应的状态对象，并通过用例的 *localName* 为状态对象的 *description* 属性赋值；根据 *createStartState* 和 *createEndState* 规则自动为状态机图创建“起始”和“结束”对

象。在目标模型基础上，手动添加状态间的转移关系，得到完整的状态机模型。该转换通过用例图到状态机图的转换实现功能架构到逻辑架构的半自动生成。

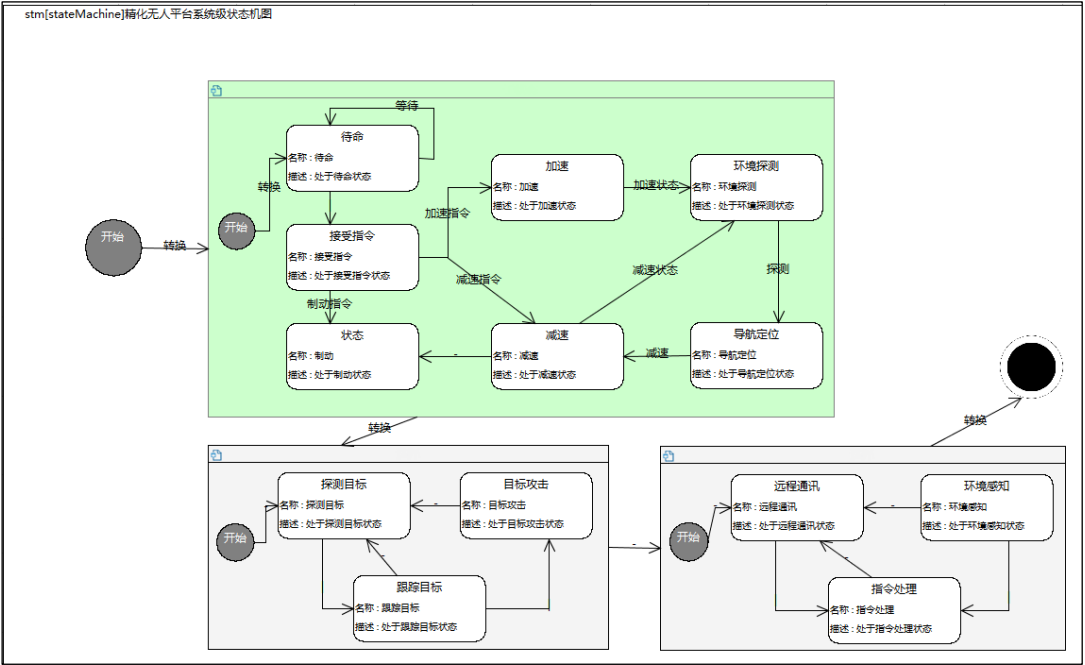


图 6.11 无人平台系统级状态机图

6.4.3 功能架构到物理架构的转换

无人平台系统级物理架构通过模块定义图描述，以用例图到模块定义图的转换为例，说明功能架构到物理架构的转换。转换目的是将用例图所述功能分配给系统物理组件，具体来说，以模块定义图中的模块描述物理组件，根据用例图中的用例对象创建模块定义图中的模块对象，以描述要实现某些功能，系统应具备的物理组件以及组件之间的关联关系。模型转换的输入模型为图 6.8 所示用例图，在转换过程中，根据执行者创建对应的功能包，将用例对象转化为模块对象，执行者和用例之间的关联关系转换为包和模块之间的包含关系，用例之间的包含关系转换为模块之间的包含关系，对应关系如表 6.4 所示。在分析对应关系的基础上，设计图 6.12 示意的转换规则。

表 6.4 用例图到模块定义图转换的元素对应关系

输入模型的元素	输出模型的元素
用例图	模块定义图

输入模型的元素	输出模型的元素
执行者对象	包对象
用例对象	模块对象
执行者和用例的关联关系	包和模块之间的包含关系
用例之间的包含关系	模块之间的包含关系

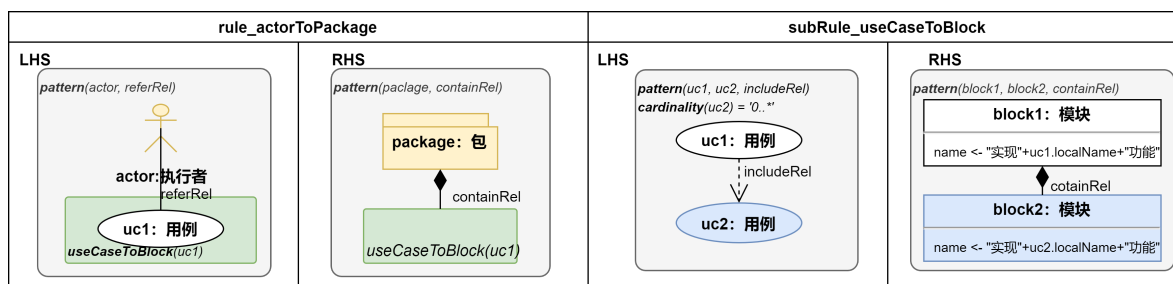


图 6.12 用例图到模块定义图转换规则示意图

用例图到模块定义图的转换需要两个转换规则完成，*actorToPackage* 规则将执行者对象转换为包对象，*useCaseToBlock* 规则将用例对象转换为到模块对象。由于转换生成的包对象和模块对象之间存在包含关系，因此 *actorToPackage* 规则调用了子规则 *useCaseToBlock*。*actorToPackage* 规则的 LHS 匹配到通过 *referRel* 连接的 *actor* 对象和 *uc1* 用例对象，并以 *uc1* 为参数，调用子规则 *useCaseToBlock* 做进一步匹配。对于调用规则，子规则先于主规则执行，因此首先根据子规则的 RHS 生成元素，接着 *actorToPackage* 规则的 RHS 根据匹配到的 *actor* 创建 *package* 对象，并将子规则 RHS 生成的元素通过 *containRel* 与 *package* 对象连接。

规则 *useCaseToBlock* 的 LHS 描述了用例 *uc1* 和 *uc2* 之间通过包含关系 *includeRel* 连接，采用基数模式将 *uc2* 的基数定义为 $[0..*]$ ，以描述用例图中 *uc1* 匹配项可包含多个 *uc2* 匹配项，*useCaseToBlock* 规则的 RHS 根据 LHS 匹配到的 *uc1* 和 *uc2*，创建 *block1* 模块和 *block2* 模块，以及两模块之间的包含关系 *containRel*，并分别将 *uc1* 和 *uc2* 的 *localName* 属性经过运算后赋值给 *block* 的 *name* 属性。例如对于匹配到的“加速”用例对象，转换之后的模块对象的 *name* 属性变为“实现加速功能”。

无人平台的系统级用例图到模块定义图转换的模型转换语言描述如下。

```

1. architectureDriver useCaseToBlockDefine
2.   create Graph[sysml_BlockDefinitionDiagram] bddGraph from
      Graph.model_sysml_UseCaseDiagramSysML2;

```



```

3.    // 将 actor 对象转换为 package 对象, 并调用子规则 useCaseToBlock
4.    rule actorToPackage{
5.        from
6.            Object[object_Actor] actor;
7.            Relationship[relationship_Association2] referRel;
8.            Object[object_Usecase] uc1;
9.            referRel.source(actor);
10.           referRel.target(uc1);
11.        to
12.            Object[object_Package2] package,
13.            Relationship[relationship_Association2] containRel(
14.                containRel.source <- package;
15.                containRel.target <- useCaseToBlock(uc1);
16.            )
17.    }
18.    // 调用规则, 将 useCase 对象转换为 block 对象, 参数类型为 Object[object_Usecase]
19.    rule useCaseToBlock(Object[object_Usecase] uc1){
20.        from
21.            Object[object_Usecase] uc2;
22.            Relationship[relationship_Dependency2] includeRel;
23.            includeRel.source(uc1);
24.            includeRel.target(uc2);
25.            cardinality(uc2) = '0..*';
26.        to
27.            Object[object_Block1] block1(
28.                block1.Poperty["name"].value <- "实现" + uc1.localName + "功能";
29.            ),
30.            Object[object_Block1] block2(
31.                block2.Poperty["name"].value <- "实现" + uc2.localName + "功能";
32.            ),
33.            Relationship[relationship_Association2] containRel(
34.                containRel.source <- block1;
35.                containRel.target <- block2;
36.            )
37.    }
38. end useCaseToBlockDefine

```

经过模型引擎执行后生成如图 6.13 所示模块定义图。生成的模块定义图需要经过精化, 根据具备的功能命名模块, 并添加包名, 最终得到如图 6.14 所示完整的无人平台系统级模块定义图。

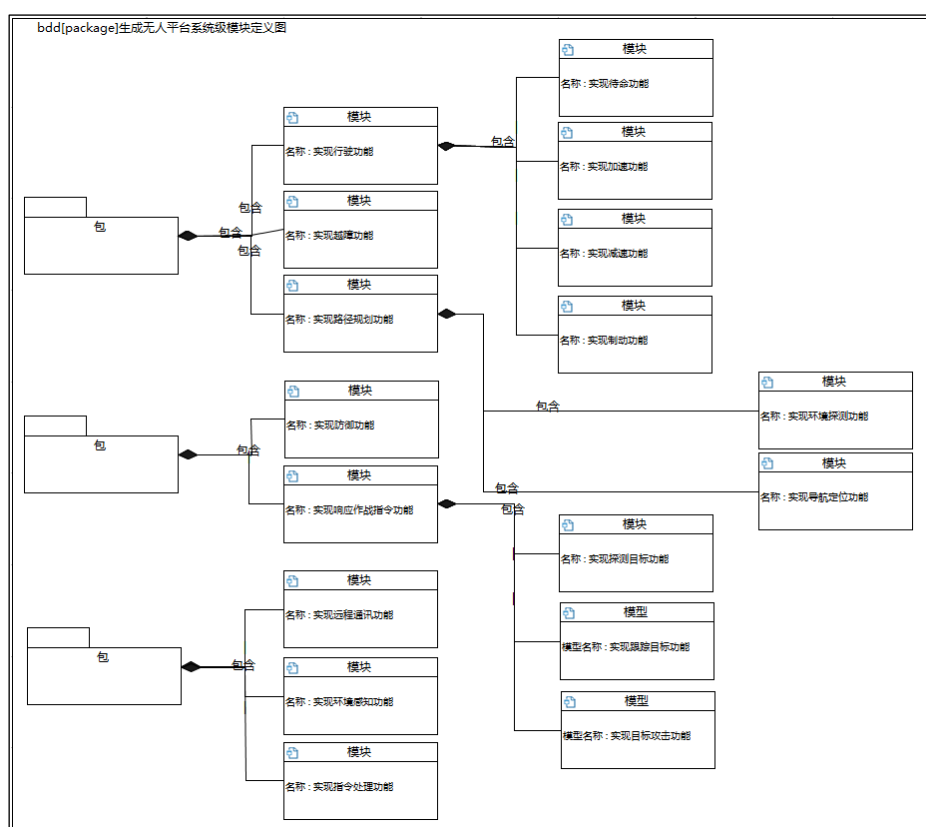


图 6.13 模型转换生成的模块定义图

无人平台功能架构到物理架构的模型转换中，以如图 6.8 所示的用例模型为源模型，目标模型为如图 6.13 所示的模块定义图。转换结果显示，根据模型转换规则 *actorToPackage*，用例图中“执行者对象关联用例对象”转换为模块定义图的“包对象包含模块对象”；对于每个和用例关联的执行者，通过调用子规则 *useCaseToBlock*，将用例图中“用例对象包含用例对象”转换为模块定义图的“模块包含模块”。例如，用例图中“机动底盘操作者关联行驶用例”、“行驶用例包含加速用例”，转换为模块定义图的“机动底盘包包含行驶模块”、“行驶模块包含加速模块”。在目标模型基础上，添加包名，并根据模块具备的功能重新命名模块，得到完整的模块定义图。该转换通过用例图到模块定义图的转换实现功能架构到物理架构的半自动生成。

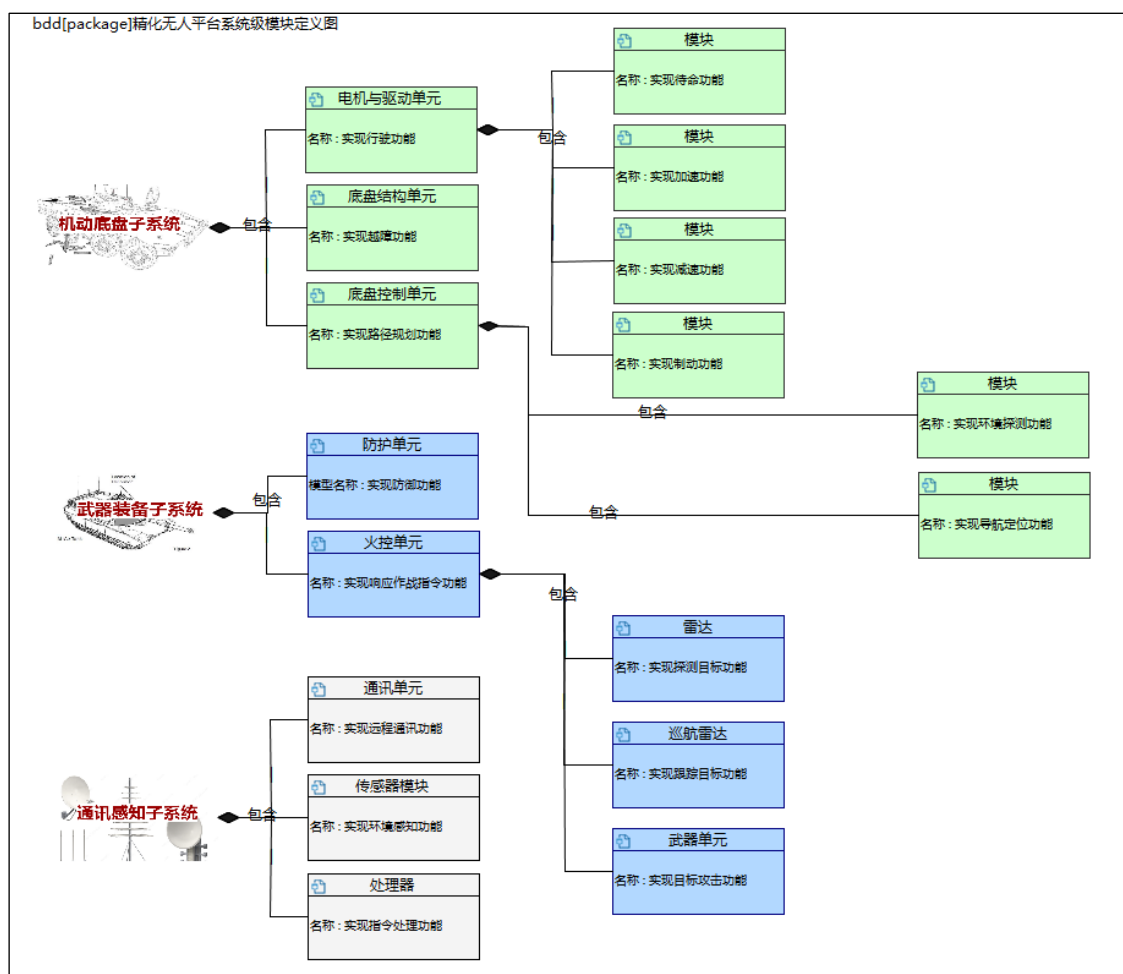


图 6.14 无人平台系统级模块定义图

6.5 架构模型转换结果分析

面向无人平台系统架构设计的模型转换验证过程中,按照需求-功能-逻辑-物理设计流程构建无人平台系统级架构模型,在建模语言方面,采用 SysML 构建设计流程中涉及到的各阶段模型。和传统方法不同的是,本文首先基于 GOPRR-E 元建模框架构建 SysML 建模语言,以实现九种图之间的模型转换。在模型转换方面,以系统需求模型为输入,通过所提出的模型转换方法和模型转换语言,实现无人平台系统级需求图到用例图、用例图到状态机图、用例图到模块定义图的半自动转换,并在转换生成的目标模型基础上手动添加模型细节,最终得到各个系统层级的架构模型,指导无人平台系统的整体架构设计。针对各场景,根据所提建模语言概念描述架构模型的转换,无人平台转换案例中涵盖了提出了四种模式和三类规则,应用情况如表 6.5 所示,目

前模型转换引擎对所提语言概念的支持情况如表 6.6 所示。结果表明，所提出的模型转换语言能够涵盖系统架构模型转换的多种场景，实现模型转换的统一描述。

表 6.5 无人平台架构模型转换过程中应用的模型转换语言概念

架构模型转换	规则	规则类型	用到的模式
需求图到用例图	objectToAddblock	标准规则	简单模式、基数模式、分组模式
	createStartState	懒惰规则	
用例图到状态机	actorToSubState	标准规则	单元素模式、简单模式、基数模式
图	useCaseToState	标准规则	
	createEndState	懒惰规则	
用例图到模块定义图	actorToPackage	标准规则	简单模式、基数模式
	useCaseToBlock	调用规则	

表 6.6 目前模型转换引擎对模型转换语言概念支持情况

模型转换语言概念	目前模型转换引擎支持情况
标准规则	支持
懒惰规则	支持
调用规则	支持
单元素模式	支持
简单模式	支持
基数模式	部分支持, 支持固定基数和[0..*]两种类型的基数, 以下几种不支持: 1. [x, +]类型基数模式, “+”表示元素个数为“一个以上”, x 表示元素个数为 x, 该模式一般用于指定模型元素的下限。 2. [?]类型基数, [?]表示元素个数为“一个或零个”, 该模式用于指定元素包含有无两种状态。
分组模式	部分支持, 对于分组模式嵌套 (即子模式 p1 中定义了其他子模式 p2), 以下情况几种情况尚未支持: 1. p1 为简单 (或单元素) 模式, p2 为基数模式 2. p1 为基数模式, p2 为基数模式

6.6 本章小结

本章节以无人平台复杂系统架构设计为案例，对本文提出的支持复杂系统架构设计模型转换方法与各研究内容进行实例应用和验证。首先分析了无人平台架构设计中多系统层级应用 RFLP（需求-功能-逻辑-物理）方法的设计流程，在此基础上制定模型转换流程，实现无人平台系统级的需求到功能架构、功能架构到逻辑架构、功能架构到物理架构的自动转换。针对三种具体的转换场景，分析输入模型和目标模型元素之间的映射关系，在此基础上应用提出的模型模式定义模型转换规则，并给出对应的模型转换语言描述。案例结果表明，本文所提出的方法和模型转换语言能够支持复杂系统架构设计流程，从需求出发，在每一个系统层级按照 RFLP 流程实现模型转换和模型信息传递，最终得到各个系统层级的架构模型，提高建模效率和模型信息可追溯性；此外，分析模型转换过程可知，模型转换语言提出的三种规则以及四种模型模式，能够实现复杂系统架构模型转换的无歧义描述。

总结与展望

工作总结

针对复杂系统架构模型开发过程中缺乏支持多领域异构模型转换的通用模型转换方法,以及缺乏能实现统一描述的模型转换语言的问题,本文提出基于图论的复杂系统架构模型转换方法以及模型转换统一描述语言。论文的主要研究工作总结如下:

(1)分析复杂系统架构模型构建方法和目前模型转换相关理论,提出模型转换框架。首先分析基于 GOPPRR-E 元建模框架的复杂系统架构模型构建原理,实现多领域异构模型的底层同构,为跨域转换奠定基础。其次总结目前已有的模型转换方法,选择基于图论中的图转换实现架构模型的转换,并提出面向 GOPPRR-E 架构模型的模型转换框架,以满足复杂系统开发场景下多领域跨域模型转换需求。

(2)研究基于图论的系统架构模型转换方法。首先根据复杂系统开发场景下模型转换特点定义了跨域模型转换概念;然后,面向 GOPPRR-E 架构模型,基于图论将 GOPPRR-E 元建模框架中模型及元模型形式化为等价的图概念。提出架构模型到图的映射、元模型到类型图的映射、模型转换规则到图转换规则的映射等映射方法,通过图转换实现异构架构模型的转换。最后,提出架构模型转换流程,实现架构模型转换的自动化。

(3)研究架构模型转换正确性的形式化检验方法。首先给出模型转换正确性的定义及其形式化描述,本文所提模型转换正确性基于目标模型的语法正确性定义。其次,基于图论中类型图一致性理论,将模型转换验证问题转变为类型图验证问题,并提出类型图验证的一致性条件。最后,提出模型转换自动验证流程,检验模型转换过程中可能出现的缺陷模型,实现模型转换正确性验证的自动化。

(4)研究模型转换统一描述语言。在上述研究基础上,提出模型转换统一描述语言,设计语言的具体语法和抽象语法。重点引入模型模式的概念,以描述模型转换规则中 LHS 和 RHS 模型元素拓扑结构,并根据 GOPPRR-E 模型结构特点提出四种典型的模型模式。最后,开发模型转换语言解析器和执行引擎,实现模型转换语言的描述能力。

(5)在上述理论研究的基础上,以无人平台系统架构设计为案例,对本文提出的模型转换方法和研究内容进行实例应用和验证。首先分析了无人平台架构设计流程,

在此基础上制定架构模型转换流程，实现无人平台系统级的需求到功能架构、功能架构到逻辑架构、功能架构到物理架构模型的自动转换。针对三种具体的转换场景，定义模型转换规则，通过模型转换语言描述相应的转换规则，在模型转换引擎支持下实现模型的半自动转换。案例结果表明，本文提出的模型转换方法能够实现复杂系统架构设计过程中架构模型（符合不同元模型）之间的转换，并通过模型转换语言实现转换的统一描述，提高建模效率和模型信息可追溯性。

基于上述总结，论文的创新点主要有以下几点：

（1）提出了基于图论的系统架构模型转换方法。在 GOPRR-E 元建模框架的基础上研究架构模型转换方法，GOPRR-E 元建模框架能够实现多领域系统架构模型的统一描述，因此该模型转换方法可支持多领域跨域模型转换。在此基础上，基于图论将 GOPRR-E 元建模框架中的相关元素映射为图论概念，具体包括架构模型到图的映射，元模型到类型图的映射，模型转换规则到图转换规则的映射，将模型转换问题转变为图转换问题，进一步研究基于图的架构模型转换提供了新的研究途径。

（2）提出了基于规则的模型转换统一描述语言。在 KARMA 语言基础上提出模型转换统一描述语言，KARMA 语言能够实现 GOPRR-E 架构模型数据的统一描述，模型转换语言扩展了 KARMA 语言使其具备模型转换描述能力。模型转换语言通过直观的声明式规则表达模型转换，实现多领域异构模型转换的统一描述，减轻架构师学习语言的负担。

展望

论文的研究工作围绕复杂系统架构设计过程的模型转换展开，重点对模型转换方法、模型转换验证方法以及模型转换语言方面进行研究，以下方面还存在改进空间。

（1）本文以图转换解决模型转换问题，重点研究模型相关概念到图文法的映射原理，而对图转换理论本身并未涉及太多研究，因此，图的重写主要应用了默认的一次推出法，一次推出法相比二次推出法转换条件较为宽松，易产生错误模型，本文中是通过模型检查避免错误模型的产生。在后续研究中，可以对图转换理论本身做进一步研究，研究二次推出法，保证生成图的正确性。

（2）在模型转换验证方面，本文主要关注生成模型的语法正确性。在实际架构模型开发过程中，某些情况下架构师在现有建模语言基础上，根据业务需求自定义模型

的语义约束，因此生成的模型不仅要保证模型语法正确性，还要保证生成模型的语义满足语义约束，模型转换的语义验证需要做进一步研究。

(3) 目前已经开发的模型转换引擎对于提出的几种模型模式尚未完全支持，对于基数模型目前仅支持固定基数和 $[0..*]$ 两种类型，对于分组模式尚未支持复杂的模型嵌套结构，需要进一步开发转换引擎以实现所提模式的完全支持。

参考文献

- [1] 朱静, 杨晖, 高亚辉等. 基于模型的系统工程概述[J]. 航空发动机, 2016, 42 (04) : 12-16.
- [2] Ramos A. L., Ferreira J. V., Barcelo J. Model-Based Systems Engineering: An Emerging Approach for Modern Systems[J]. IEEE transactions on systems, man and cybernetics. Part C, Applications and reviews, 2012, 42 (1) : 101-111.
- [3] 蒋严冰, 邢春晓. 模型驱动的体系结构研究综述[J]. 南京大学学报: 自然科学版, 2005, 41 (z1)
- [4] 王崑声, 袁建华, 陈红涛等. 国外基于模型的系统工程方法研究与实践[J]. 中国航天, 2012 (11) : 52-57.
- [5] 张德芬, 李师贤, 古思山. MDA中的模型转换技术综述[J]. 计算机科学, 2006, 33 (10) : 228-230.
- [6] Batchkova Idilia, Antonova Iskra. Improving the software development life cycle in process control using UML/SysML[J]. IFAC Proceedings Volumes, 2011, 44 (1) : 14133-14138.
- [7] Hoffmann Hans Peter. 5.3.3 SysML - Based Systems Engineering Using a Model - Driven Development Approach[J]. INCOSE International Symposium, 2006, 16 (1) : 804-814.
- [8] . Architecture Analysis & Design Language (AADL)[S]: 国外-国外学协会-美国机动车工程师协会 US-SAE
- [9] 刘畅. 模型驱动飞机座舱显示控制应用软件 开发方法研究与实现[D]: 上海交通大学
- [10] Scheer A. W. Architektur integrierter Informationssysteme[J]. Springer Berlin Heidelberg, 1992
- [11] Bergmann G. A. Bor, Ujhelyi Zolt A. N., R A Th Istv A. N. et al, A Graph Query Language for EMF Models[M], 2011: 167-182.
- [12] Dori Dov, Object-Process Methodology - A Holistic Systems Paradigm[M], 2002
- [13] Pohjonen Risto, Metamodeling made easy - metaedit+ (tool demonstration)[M]. 1ère éd Berlin, Heidelberg: Springer-Verlag, 2005: 442-446.
- [14] 麻志毅, 刘辉, 何啸等. 一个支持模型驱动开发的元建模平台的研制[J]. 电子学报, 2008, 36 (04) : 731-736.
- [15] Wang H., Wang G., Lu J. et al, Ontology Supporting Model-Based Systems Engineering Based on a GOPRR Approach[M]: New Knowledge in Information Systems and Technologies, 2019
- [16] 曾一, 周吉, 孙政等. 支持MDA的设计模式建模与模型转换方法研究[J]. 计算机工程与应用, 2012, 48 (01) : 76-80.

- [17] 吉鸣, 黄志球, 祝义等. 基于MDA的实时软件资源建模与模型转换的方法[J]. 计算机科学, 2011, 38 (08) : 136-141.
- [18] Appukuttan B. K., Clark T., Evans A. et al. QVT-Partners Revised Submission to MOF 2.0 Query/View/Transformations RFP[J], 2003
- [19] Biehl M. Literature study on the state of the art in model transformation technology[J]. KTH, 2010
- [20] Kurtev I. State of the Art of QVT: A Model Transformation Language Standard[J]. Springer, Berlin, Heidelberg, 2007
- [21] Gogolla M. Object Constraint Language[J]. Springer US
- [22] 秦亭亭, 刘淑芬, 吴菲等. 一种模型转换分类方法及其在QVT中的应用[J]. 计算机应用与软件, 2008 (08) : 27-29.
- [23] Fettke P., Loos P. Model driven architecture (MDA)[J]. Business and Information Systems Engineering the international journal of Wirtschaftsinformatik, 2003, 45 (5) : 555-559.
- [24] Gomes C., Barroca B., Amaral V., Classification of Model Transformation Tools: Pattern Matching Techniques[M], 2014
- [25] Kalnins A., Barzdins J., Elms E. C. The model transformation language MOLA[J], 2005
- [26] Foundation E. EMF: The Eclipse Modeling Framework[J], 2010
- [27] 李少芳. MDA软件范型中模型的形式化与转换研究[D]: 宁夏医科大学, 2018.
- [28] 邓刘梦, 沈国华, 黄志球等. 扩展SysML支持需求追踪模型的自动生成[J]. 计算机科学与探索, 2019, 13 (06) : 950-960.
- [29] 王宁波. 基于QVT的MOF模型转换技术的研究与实现[D]: 中国石油大学(华东), 2014.
- [30] 曾一, 周吉, 孙政等. 支持MDA的设计模式建模与模型转换方法研究[J]. 计算机工程与应用, 2012, 48 (01) : 76-80.
- [31] 李燕. 基于MDA的领域模型到HBase的模型转换研究[D]: 广西大学, 2014.
- [32] Chang Chih-Hung, Lu Chih-Wei, Yang Wen Pin et al, A SysML Based Requirement Modeling Automatic Transformation Approach[M]: IEEE, 2014: 474-479.
- [33] Jouault Frédéric, Allilaire Freddy, Bézivin Jean et al. ATL: A model transformation tool[J]. Science of Computer Programming, 2008, 72 (1-2) : 31-39.
- [34] Agrawal A. GReAT: A Metamodel Based Model Transformation Language[J]. Institute for Software Integrated Systems, 2003: 386-387.

- [35] Grunske L., Geiger L., Zundorf A. et al, Using Graph Transformation for Practical Model Driven Software Engineering[M]: Using Graph Transformation for Practical Model Driven Software Engineering, 2005
- [36] Cetinkaya D., Verbraeck A., Metamodeling and model transformations in modeling and simulation[M]: IEEE, 2011: 3043-3053.
- [37] Balasubramanian D., Narayanan A., Buskirk C. V. et al. The Graph Rewriting and Transformation Language: GReAT[J]. DBLP, 2007
- [38] Mens T., Gorp P. V., A taxonomy of model transformation[M], 2006: 125-142.
- [39] Jouault Frédéric, Kurtev Ivan. On the interoperability of model-to-model transformation languages[J]. Science of Computer Programming, 2007, 68 (3) : 114-137.
- [40] 何啸, 麻志毅, 邵维忠等. 一种针对模型转换的图形化建模语言[J]. 计算机研究与发展, 2015 (9) : 2145-2162.
- [41] 王学斌. 软件工程中基于模型驱动架构的模型转换技术研究[D]: 国防科学技术大学, 2006.
- [42] Jézéquel Jean-Marc, Combemale Benoit, Barais Olivier et al. Mashup of metalanguages and its implementation in the Kermeta language workbench[J]. Software and systems modeling, 2015, 14 (2) : 905-920.
- [43] Group O. M. Meta Object Facility (MOF) 2.0 Core Specification[J], 2003
- [44] Kern Heiko, Hummel Axel, Kühne Stefan, Towards a comparative analysis of meta-metamodels[M]: ACM, 2011: 7-12.
- [45] Sprinkle J., Rumpe B., Vangheluwe H. et al. Metamodelling: State of the Art and Research Challenges[J], 2014
- [46] Dimitrieski V., Elikovi Milan, Ivanevi Vladimiret al, A Comparison of Ecore and GOPPRR through an Information System Meta Modeling Approach[M], 2012
- [47] Biehl M. Literature study on the state of the art in model transformation technology[J]. KTH, 2010
- [48] Tratt Laurence. Model transformations and tool integration[J]. Software & Systems Modeling, 2005, 4 (2) : 112-122.
- [49] Kleppe Anneke G., Warmer Jos B., Bast Wim, MDA explained: the model driven architecture : practice and promise[M]. Boston: Addison-Wesley, 2003: {}.
- [50] Mens Tom, Van Gorp Pieter. A Taxonomy of Model Transformation[J]. Electronic Notes in

Theoretical Computer Science, 2006, 152: 125-142.

[51] Batot Edouard, Sahraoui Houari, Syriani Eugene et al, Systematic mapping study of model transformations for concrete problems[M]: SCITEPRESS, 2016: 176-183.

[52] Guo Jiangmin, Wang Guoxin, Lu Jinzhiet al. General Modeling Language Supporting Model Transformations of MBSE (Part 2)[J]. INCOSE International Symposium, 2020, 30 (1) : 1460-1473.

[53] Visser E. A Survey of Rewriting Strategies in Program Transformation Systems[J]. Electronic Notes in Theoretical Computer Science, 2001, 57: 109-143.

[54] 王宁波. 基于QVT的MOF模型转换技术的研究与实现[D]: 中国石油大学(华东), 2014.

[55] 朱彬. 基于QVT的模型转换技术研究与实现[D]: 中南大学, 2005.

[56] 张震波. 模型转换引擎的研究与设计[D]: 江苏大学, 2006.

[57] 魏佳璇. MDA在科学 workflow 建模中的应用研究[D]: 兰州大学, 2009.

[58] Westfechtel Bernhard, Nagl Manfred, Graph Transformations for Evolving Domain Knowledge[M]. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012: 341-355.

[59] Lano Kevin, Kolahdouz-Rahimi Shekoufeh, Yassipour-Tehrani Sobhanet al. A survey of model transformation design patterns in practice[J]. Journal of Systems and Software, 2018, 140: 48-73.

[60] Syriani Eugene, Vangheluwe Hans, Küster Jochen et al, ECEASST De-/Re-constructing Model Transformation Languages[M]

[61] Karsai Mpvgv. Applying a Model Transformation Taxonomy to Graph Transformation Technology[J]. Electronic Notes in Theoretical Computer Science, 2006

[62] Wimmer Manuel, Kusel Angelika, Retschitzegger Werner et al, ECEASST Reusing Model Transformations across Heterogeneous Metamodels[M]

[63] Varr D., Varr G., Pataricza A. Designing the automatic transformation of visual languages[J]. Science of Computer Programming, 2002, 44 (2) : 205-227.

[64] Grunske L., Geiger L., Zundorf A. et al, Using Graph Transformation for Practical Model Driven Software Engineering[M]: Using Graph Transformation for Practical Model Driven Software Engineering, 2005

[65] Andries Marc, Engels Gregor, Habel Annegret et al. Graph transformation for specification and programming[J]. Science of Computer Programming, 1999, 34 (1) : 1-54.

[66] Corradini A., Montanari U., Rossi F. et al, Algebraic Approaches to Graph Transformation - Part I:

Basic Concepts and Double Pushout Approach[M], 1997

[67] 韩秀清, 曾晓勤, 邹阳等. 图文法综述[J]. 计算机科学, 2008, 35 (08) : 10-16.

[68] Biermann Enrico, Ermel Claudia, Taentzer Gabriele. Formal foundation of consistent EMF model transformations by algebraic graph transformation[J]. Software & Systems Modeling, 2012, 11 (2) : 227-250.

[69] Ehrig Hartmut, Ehrig Karsten. Overview of Formal Concepts for Model Transformations Based on Typed Attributed Graph Transformation[J]. Electronic Notes in Theoretical Computer Science, 2006, 152: 3-22.

[70] Ehrig Hartmut, Prange Ulrike, Taentzer Gabriele, Fundamental Theory for Typed Attributed Graph Transformation[M]. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004: 161-177.

[71] Isenberg Tobias, Steenken Dominik, Wehrheim Heike, Bounded Model Checking of Graph Transformation Systems via SMT Solving[M]. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013: 178-192.

[72] Grunske L., Geiger L., Zundorf A.et al, Using Graph Transformation for Practical Model Driven Software Engineering[M]: Using Graph Transformation for Practical Model Driven Software Engineering, 2005

[73] Ehrig H., Heckel R., Korff M.et al, Algebraic Approaches to Graph Transformation - Part II: Single Pushout Approach and Comparison with Double Pushout Approach[M], 1997

[74] Ehrig Hartmut, Ehrig Karsten. Overview of Formal Concepts for Model Transformations Based on Typed Attributed Graph Transformation[J]. Electronic Notes in Theoretical Computer Science, 2006, 152 (1-2) : 3-22.

[75] Mierlo S. V., Vangheluwe H. Adding Rule-Based Model Transformation to Modelling Languages in MetaEdit+[J]. electronic communications of the easst, 2012

[76] de Lara Juan, Guerra Esther, Boronat Arturet al. Domain-specific discrete event modelling and simulation using graph transformation[J]. Software & Systems Modeling, 2014, 13 (1) : 209-238.

[77] Oakes B. J. A Symbolic Execution-Based Approach to Model Transformation Verification using Structural Contracts[J], 2018

[78] Biermann Enrico, Ermel Claudia, Taentzer Gabriele. Formal foundation of consistent EMF model transformations by algebraic graph transformation[J]. Software & Systems Modeling, 2012, 11 (2) : 227-

250.

[79] Buettner F., Egea M., Cabot J. et al, Verification of ATL Transformations Using Transformation Models and Model Finders[M], 2012

[80] 刘峥峥. 使用图转换理论的模型转换一致性研究[D]: 中国科学技术大学, 2010.

[81] Rensink A., Schmidt A., Varro D., Model Checking Graph Transformations: A Comparison of Two Approaches[M], 2004

[82] Becker B., Beyer D., Giese H. et al, Symbolic invariant verification for systems with dynamic structural adaptation[M], 2006

[83] Asztalos M., Lengyel L., Levendovszky T., Towards Automated, Formal Verification of Model Transformations[M], 2010

[84] Lu Jinzhi, Wang Guoxin, Ma Junda et al. General Modeling Language to Support Model-based Systems Engineering Formalisms (Part 1)[J]. INCOSE International Symposium, 2020, 30 (1) : 323-338.

[85] Varró G., Friedl K., Varró D. Adaptive Graph Pattern Matching for Model Transformations using Model-sensitive Search Plans[J]. Electronic Notes in Theoretical Computer Science, 2006, 152 (none) : 191-205.

[86] Guo J, Lu J, Ding J, et al. A Model Query Language for Domain-Specific Models[C]//2020 5th International Conference on Mechanical, Control and Computer Engineering (ICMCCE). IEEE, 2020: 1207-1213.

攻读学位期间发表论文与研究成果清单

参与科研项目

1. 国家重点研发计划

发表论文

- [1] Guo J, Wang G, Lu J, et al. General Modeling Language Supporting Model Transformations of MBSE (Part 2)[J]. INCOSE International Symposium, 2020,30(1):1460-1473.
- [2] J. Guo, J. Lu, J. Ding and G. Wang, A Model Query Language for Domain-Specific Models[C], 2020 5th International Conference on Mechanical, Control and Computer Engineering (ICMCCE), 2020, pp. 1207-1213.

学科竞赛

1. 2019 年 5 月，北京理工大学研究生科技创新项目校级重点团队（负责人）
2. 2020 年 9 月，北京理工大学第十七届“世纪杯”学生创业竞赛银奖（2/7）
3. 2020 年 9 月，第六届中国国际“互联网+”大学生创新创业大赛（北京赛区复赛）一等奖（3/8）

附录 B. 模型转换语言具体语法

```

grammar modelTrans;
modeltransUnit
:
    modeltomodelUnit
    | libraryUnit
;
modeltomodelUnit
:
    'architectureDriver' IDENT
classBody 'end'
;
classBody
:
    classBodyDeclaration*
;
classBodyDeclaration
:
    modelInitialDeclaration
    | ruleDeclaration
;
modelInitialDeclaration
:
    CREATE targetModel FROM
sourceModel ';'
;
targetModel
:
    graphType IDENT
;
sourceModel
:
    elementPointed
;
elementPointed
:
    modelElementTypes '(' 'id=='
String ')'
;
/**
    * Define the rule expression for
    model transformation.
    */
ruleDeclaration
:
    matchedRule
    | calledRule
;
matchedRule
:
    normalMatchedRule
    | lazyMatchedRule
;
normalMatchedRule
:
    RULE IDENT normalRuleBlock
;
lazyMatchedRule
:
    LAZY RULE IDENT normalRuleBlock
;
normalRuleBlock
:
    '{' normalMathedruleBody '}'
;
/**
    * The matching part can not be
    defined in the callable rule body
    */
calledRuleBlock
:
    '{' calledRuleBody '}'
;
normalMathedruleBody
:
    matchingPattern inPointedGraph?
usingPart? bindingPattern doPart?
;
calledRuleBody

```

```

:
    usingPart? bindingPattern
doPart?
;
calledRule
:
    calledRuleType? RULE
methodDeclarator calledRuleBlock
;
calledRuleType
:
    ENTRYPOINT
    | ENDPOINT
;
inPointedGraph
:
    'in' elementPointed(',',
elementPointed)* ';'
;
matchingPattern
:
    FROM sourceElementExpr
;
sourceElementExpr
:
    patternBody
;
/*
 * Lexical Structure
 */
literal
:
    REAL
    | IntegerLiteral
    | BooleanLiteral
    | String
;
dataType
:
    primitiveType
    | karmaModelType
;
primitiveType
:
    numericType
    | textualType
    | 'Boolean'
;
textualType
:
    'String'
;
numericType
:
    integralType
;
integralType
:
    'Number'
    | 'Int'
;
referenceType
:
    collectionType
;
collectionType
:
    collectionTypePrefix
typeParameter?
;
collectionTypePrefix
:
    'Set'
    | 'OrderedSet'
    | 'Map'
    | 'List'
;
elementType
:
    modelElementTypes
;
elementSysName

```

```

:                                statementList
    IDENT                        :
;                                statement+
modelElement_roleType          ;
:                                ruleDefine
    ROLE                        :
;                                'rule' IDENT ruleClause* 'end'
modelElemets_propertyType      IDENT
:                                ;
    PROPERTY                   ruleClause
;                                :
metaModel_pointType            IDENT '(' metaElement ','
:                                metaElement ')' ';'
    META_POINT                 ;
;                                assingmentDeclarement
elementsWithConditions         :
:                                dataType variableDeclaratorList
    modelElementType           ;
elementsFilterExpression?      expression
;                                :
elementsFilterExpression       assignmentExpression
:                                | expression ':'
    '(' filterExpression ')'   assignmentExpression
;                                ;
filterExpression               assignmentExpression
:                                :
    conditionalExpression       conditionalExpression
;                                | assignment
model                           ;
:                                conditionalExpression
    'model' '.' IDENT          :
;                                conditionalOrExpression
localVariableDeclarationStatement ;
:                                expressionName
    fieldDeclaration           :
;                                IDENT
/** Method invocation statement */ ;
methodInvocation               primary
:                                :
    primary                     ( primaryNoNewArray
;                                | ganttCreationExpression

```

	'func' karmaContextDefinition?
collectionCreationExpression	methodHeader methodBody
templateInvocation	;
)(primaryNoNewArray_lf_primary)*	karmaContextDefinition
;	:
collectionCreationExpression	'context' dataType
:	;
collectionType	methodHeader
collectionInitializer	:
;	result methodDeclarator
basicMTStructure	;
:	result
importExpression* modeltransUnit	:
;	dataType 'void'
importExpression	;
:	methodDeclarator
IMPORT IDENT ';' ;	:
;	IDENT '(' formalParameters? ')'
libraryUnit	;
:	formalParameters
'library' IDENT classBody 'end'	:
;	formalParameter
/**Class member including global	formalParameters
field and member operation(method).	(',' formalParameter)
*/	;
classMemberDeclaration	formalParameter
:	:
fieldDeclaration	dataType IDENT
patternDeclarement	IDENT
;	;
/** Declare the assignment clause,	methodBody
all the clause which is used for	:
declaration should be included.*/	block
fieldDeclaration	;
:	block
assingmentDeclarement ';' ;	:
;	'{' blockStatements? '}'
/**The definition for	;
operation(method). */	/**The statements can be defined in
operationDeclaration	the method body. */
:	blockStatements

<pre> : blockStatement+ ; blockStatement : statement ; patternDeclarement : 'pattern' patternHeader '{' patternBody '}' ; /**Consisted of pattern statement. Both matching pattern and binding pattern are recognized.*/ patternStatements : patternBody ; patternBody : (patternBodyExpression ';')+ ; patternBodyExpression : assingmentDeclarement constraintsDeclare binaryRelationshipDeclare subPatternReference patternVariableDeclarement assignment ; patternVariableDeclarement : assingmentDeclarement '(' filterExpression? ')' ; binaryRelationshipDeclare : </pre>	<pre> expressionName methodInvocation_lf_primary ; constraintsDeclare : 'constraint' '(' conditionalExpression ')' ; patternHeader : methodDeclarator ; subPatternReference : 'match' methodInvocation ; usingPart : USING '{' usingBody? '}' ; usingBody : localVariableDeclarationStatemen t+ ; bindingPattern : 'to' outPatternElement?(',', outPatternElement)* ; callOperation : ((IDENT THISMODULE) '.')? ; outPatternElement : dataType IDENT ('in' elementPointed)? ((' bindingStatement* '))'? ; /**The bindings statement is similar </pre>
--	--

to assignment operation with	<i>DIGITS</i> <i>'.'</i> <i>DIGITS</i>
difference of operation sign.*/	;
bindingStatement	<i>IDENT</i>
:	:
assignment ';'	<i>NONDIGITAL</i> (<i>DIGIT</i>
;	<i>NONDIGITAL</i>)*
doPart	;
:	fragment
'do' '{' karmaMTstatements? '}'	<i>DecimalIntegerLiteral</i>
;	:
<i>IntegerLiteral</i>	<i>DIGIT</i>
:	;
<i>DecimalIntegerLiteral</i> +	fragment
;	<i>NUMBER</i>
<i>BooleanLiteral</i>	: <i>DIGITS</i> (<i>'.'</i> <i>DIGITS</i>)?
:	;
'true' 'false'	fragment
;	<i>NONDIGITAL</i>
<i>String</i>	:
:	[a-zA-Z] '_'
''' (.)*? '''	;
; //String type	fragment
/**It seems that this is the	<i>DIGITS</i>
duplicate rule, please unify the	:
rule. */	<i>DIGIT</i> +
<i>INT</i>	;
:	fragment
<i>DIGIT</i> +	<i>DIGIT</i>
;	:
/**Real type variable */	[0-9]
<i>REAL</i>	;
:	

致谢

执笔于此，在北理工的三年研究生生涯即将结束，三年的研究生时光恍如白驹过隙，那个对未来充满憧憬、满怀信心的稚嫩少年踏入梦寐以求学府的场景仿佛就在昨天。三年中有成长、有收获、也有遗憾，感谢一路以来遇到的每一位倾心指导的老师，热心相助的同学，志趣相投的朋友，背后默默支持的亲人。

首先要感谢我的导师王国新老师，在科研上为我指明方向，耐心指导。王老师对待科研和工作严谨认真、一丝不苟的态度始终影响着我。王老师对基于模型系统工程有着独到的见解，自入学之初，王老师就从课程安排、课题选择和学术研究方面给与详细的指导，毕业论文也是在王老师的悉心指导下顺利完成的。王老师积极关心学生的生活，及时帮助解决生活中遇到的困难。王老师也是我的人生导师，在求职过程中，在我对未来感到迷茫时指引人生方向。

感谢鲁金直博士在我道路中给与的无私帮助，很荣幸在刚步入研究生就受到鲁博士的详细指导，帮助我顺利步入科研道路。鲁博士对科研充满热情，对学术认真严谨，对学生耐心，是我学习的榜样，犹记得鲁博士带领我逐字逐句修改小论文的日子。

感谢同课题组的阎艳老师、郝佳老师、明振军老师和姚丽亚老师。在小组会上耐心指导，倾囊相授，使我受益良多。各位老师也为课题组的同学们提供更高的平台，多次举办学术活动，请校内外学者来课题组交流，开拓学生眼界，营造出浓厚的学术氛围。

感谢工业与智能系统工程研究所的胡耀光老师、薛庆老师、敬石开老师、官琳老师、张发平老师、闻敬谦老师、刘敏霞老师，老师们为我们的毕业论文提供宝贵的意见，在提问中让我们积极思考，帮助顺利毕业。

感谢同课题组王儒师兄、黄思翰师兄、贾良跃师兄、马君达师兄、牛红伟师兄、鲍衍地师兄几位博士师兄在科研生活上给与的指导和帮助；感谢已经毕业的杨念师姐、陈舒婷师姐、王宏伟师兄在科研和生活中给与的帮助；感谢同届的余士严、霍阳阳、万斯来、周梦莹、靳益利、曾梦梦、龙辉、陈雪健同学，我们在科研上互相鼓励共同进步；感谢同小组的陈婧琦、栗寒、丁洁、吴绶玄、李子航师弟师妹们在学术上的支持；感谢工业工程全实验室的同学们留下的美好集体回忆。

感谢家人和女友的一路陪伴，默默付出。他们永远是我的坚强后盾和舒适的港湾，

特别感谢我的女友，在我人生的低谷中不离不弃，鼓励我重拾信心，走出困境，在我撰写论文过程中为提供后勤保障，保驾护航。

感谢三位舍友余士严、霍阳阳和万斯来，让我在繁忙的科研中感受到家人般的关怀，在科研中互相鼓励，互相监督。

最后感谢各位专家于百忙之中抽出时间评阅本文，并提出宝贵意见。

停笔于此，求学之路暂时告一段落，现在的我仍然满怀信心，对未来的职业生涯充满憧憬，在工作岗位上为社会做出贡献。

二零二一年六月

于北京理工大学 1 号教学楼